

DSWoK: Классический ML и метрики качества

Учебное пособие для самостоятельного изучения

Перевод и адаптация открытой базы знаний

DSWoK — Data Science Well of Knowledge

github.com/Erlemar/dswok

Содержание

Раздел 1. A/B-тестирование и снижение дисперсии через CUPED

1.1 A/B-тесты: фундамент онлайн-экспериментов

- 1.1.1 Дизайн эксперимента
- 1.1.2 Статистические основы
- 1.1.3 Снижение дисперсии
- 1.1.4 Режимы отказа
- 1.1.5 Альтернативы и расширения
- 1.1.6 Операционная практика
- 1.1.7 Применения
- 1.1.8 Типичные ошибки
- 1.1.9 Ссылки

1.2 CUPED (Controlled-experiment Using Pre-Experiment Data)

- 1.2.1 Формула
- 1.2.2 Снижение дисперсии
- 1.2.3 Выбор ковариаты
- 1.2.4 CUPAC
- 1.2.5 Связь со стратификацией
- 1.2.6 Когда CUPED работает плохо
- 1.2.7 Типичные подводные камни
- 1.2.8 Ссылки

Раздел 2. Расхождение обучения-обслуживания и холодный старт

2.1 Training-serving skew

- 2.1.1 Skew, сдвиг распределения и concept drift
- 2.1.2 Симптомы
- 2.1.3 Как возникает skew
- 2.1.4 Обнаружение и устранение
- 2.1.5 Диагностика A/B-тестов
- 2.1.6 Операционная практика на масштабе
- 2.1.7 Типичные режимы отказа
- 2.1.8 Ограничения
- 2.1.9 Ссылки

2.2 Холодный старт (cold start)

- 2.2.1 Холодный старт нового пользователя
- 2.2.2 Холодный старт нового айтема
- 2.2.3 Переход тёплый старт
- 2.2.4 Новая компания
- 2.2.5 Оценка
- 2.2.6 Типичные ловушки

2.2.7 Ограничения

2.2.8 Ссылки

Раздел 3. Калибровка вероятностей, матрица ошибок и F1-метрика

3.1 Калибровка вероятностей

3.1.1 Когда калибровка важна

3.1.2 Источники ошибок калибровки

3.1.3 Измерение калибровки

3.1.4 Методы рекалибровки

3.1.5 Prior correction для выборочных обучающих данных

3.1.6 Многоклассовая калибровка

3.1.7 Per-segment рекалибровка

3.1.8 Multi-calibration

3.1.9 Практические соображения

3.1.10 Ограничения

3.1.11 Пример кода

3.1.12 Связанные темы

3.2 Матрица ошибок (confusion matrix)

3.2.1 Базовая структура

3.2.2 Производные метрики

3.2.3 Агрегация метрик в многоклассовой задаче

3.2.4 ROC и AUC

3.2.5 Precision-Recall Curve

3.2.6 Ссылки

3.3 F1-метрика

3.3.1 Вариации

3.3.2 Ссылки

Раздел 4. Линейные модели, градиентный бустинг и кластеризация

4.1 Линейная регрессия

4.1.1 Простая линейная регрессия

4.1.2 Множественная линейная регрессия

4.1.3 Подбор коэффициентов

4.1.4 Предположения

4.1.5 Метрики оценки модели

4.1.6 Проверка гипотез

4.1.7 Доверительные интервалы

4.1.8 Расширения и регуляризация

4.1.9 Моделирование сложных зависимостей

4.1.10 Пример кода

4.1.11 Ссылки

4.2 Логистическая регрессия

4.2.1 Предположения

4.2.2 Maximum Likelihood Estimation (MLE)

4.2.3 Градиентный спуск для логистической регрессии

4.2.4 Интерпретация коэффициентов

4.2.5 Достоинства

4.2.6 Ограничения

4.2.7 Пример кода

4.2.8 Ссылки

4.3 Градиентный бустинг

4.3.1 Как это работает

4.3.2 Достоинства

4.3.3 Недостатки

4.3.4 Классификация vs регрессия

4.3.5 Популярные реализации

4.3.6 DART (Dropouts meet Multiple Additive Regression Trees)

4.3.7 Ссылки

4.4 k-Nearest Neighbors (kNN)

4.4.1 Как это работает

4.4.2 Достоинства

4.4.3 Недостатки

4.4.4 Выбор K

4.4.5 Пример кода

4.4.6 Ссылки

4.5 k-means кластеризация

4.5.1 Как это работает

4.5.2 Метрики расстояния

4.5.3 Выбор K

4.5.4 Достоинства

4.5.5 Недостатки

4.5.6 Вариации

4.5.7 Пример кода

4.5.8 Ссылки

Раздел 5. Многорукие бандиты, SVM, снижение размерности, деревья и метрики расстояния

5.1 Многорукие бандиты (multi-armed bandits)

5.1.1 Постановка задачи

5.1.2 Основные алгоритмы

5.1.3 Контекстуальные бандиты

5.1.4 Regret против оценки

5.1.5 Применения

5.1.6 Связь с reinforcement learning

5.1.7 Типичные подводные камни

5.1.8 Литература

5.2 SVM (Support Vector Machines)

5.2.1 Математическая формулировка

5.2.2 Ядерный трюк (kernel trick)

- 5.2.3 Преимущества
- 5.2.4 Недостатки
- 5.2.5 Пример кода
- 5.2.6 Ссылки
- 5.3 Снижение размерности
 - 5.3.1 Для чего используется снижение размерности
 - 5.3.2 Основные подходы к снижению размерности
 - Линейные методы
 - Нелинейные методы
 - 5.3.3 Ковариация
 - 5.3.4 PCA и ковариация
 - Как работает PCA
 - 5.3.5 Собственные числа и собственные векторы
 - Singular Value Decomposition (SVD) и PCA
 - 5.3.6 Метрики оценки
 - 1. Доля объяснённой дисперсии (Explained Variance Ratio)
 - 2. Ошибка восстановления
 - 5.3.7 Практические соображения
 - 5.3.8 Ссылки
- 5.4 Компромисс смещения-дисперсии (bias-variance trade-off)
 - 5.4.1 Смещение (bias)
 - 5.4.2 Дисперсия (variance)
 - 5.4.3 Компромисс
 - 5.4.4 Разложение полной ошибки
 - 5.4.5 Недообучение и переобучение
 - 5.4.6 Стратегии управления компромиссом смещения-дисперсии
 - 5.4.7 Ссылки
- 5.5 Случайный лес (Random Forest)
 - 5.5.1 Как работает
 - 5.5.2 Преимущества
 - 5.5.3 Недостатки
 - 5.5.4 Предпосылки для хорошей производительности
 - 5.5.5 Ссылки
- 5.6 Решающее дерево (Decision Tree)
 - 5.6.1 Структура
 - 5.6.2 Процесс обучения
 - 5.6.3 Критерии разбиения
 - 5.6.4 Критерии остановки
 - 5.6.5 Ансамблевые методы на основе решающих деревьев
 - 5.6.6 Преимущества
 - 5.6.7 Недостатки
 - 5.6.8 Регуляризация
 - 5.6.9 Важность признаков

5.6.10 Пример вычисления энтропии

5.6.11 Ссылки

5.7 Расчёт расстояний

Раздел 6. Валидация моделей и регуляризация

6.1 Валидация (cross-validation, splits, leakage)

Назначение

Подходы к валидации

Разделение train/validation/test

Bootstrapping

Кросс-валидация

Adversarial validation

Способы разделения данных

Ссылки

6.2 Регуляризация (L1, L2, ElasticNet, early stopping и др.)

Распространённые типы регуляризации

L1-регуляризация (Lasso)

L2-регуляризация (Ridge)

Elastic Net

Типы регуляризации, специфичные для модели

Типы регуляризации, не зависящие от модели

Типы регуляризации, специфичные для нейросетей

6.3 Валидация для временных рядов

Forward Chaining

Sliding Window

Раздел 7. Метрики и функции потерь

7.1 Введение в метрики и функции потерь

7.2 Регрессионные метрики

Метрики на основе ошибок

Метрики качества подгонки

Корреляция, ковариация и R^2

7.3 Метрики рекомендательных систем

7.4 NLP-метрики

Метрики машинного перевода и генерации текста

Метрики ответов на вопросы (QA)

Метрики разметки последовательностей

7.5 Метрики кластеризации

Внутренние метрики — intrinsic (без ground truth)

Внешние метрики — требуется ground truth

Определение оптимального числа кластеров

7.6 Метрики компьютерного зрения

Метрики детекции объектов

Метрики семантической сегментации

Метрики instance-сегментации

Метрики генерации и синтеза изображений

Метрики 3D-зрения

Метрики оценки позы человека

7.7 Общие функции потерь (классификация и регрессия)

Cross-Entropy Loss

Mean Squared Error (MSE)

Kullback-Leibler Divergence (KL-дивергенция)

Регуляризационные функции потерь

7.8 NLP-функции потерь

Negative Log-Likelihood (NLL) Loss

Perplexity (перплексия)

Connectionist Temporal Classification (CTC) Loss

Triplet Loss

Contrastive Loss

Функции потерь для Reinforcement Learning from Human Feedback (RLHF)

7.9 Функции потерь для компьютерного зрения

Focal Loss

Dice Loss

IoU (Intersection over Union) / Jaccard Loss

Perceptual Loss

Adversarial Loss

SSIM (Structural Similarity Index) Loss

Классический ML и метрики качества

Раздел 1. A/B-тестирование и снижение дисперсии через CUPED

1.1 A/B-тесты: фундамент онлайн-экспериментов

A/B-тестирование (онлайн контролируемые эксперименты) случайным образом распределяет единицы наблюдения в тестовую или контрольную группу и сравнивает агрегированные исходы, относя наблюдаемую разницу к произведённому изменению. Именно рандомизация отделяет причинно-следственный эффект изменения от конфаундеров, которые иначе с ним коррелировали бы. Дальнейший текст предполагает знакомство с проверкой гипотез, р-значениями и доверительными интервалами.

1.1.1 Дизайн эксперимента

Единица рандомизации

Пользователь, сессия, устройство, запрос или временной интервал (switchback). Более мелкие единицы могут повысить статистическую эффективность, но только если анализ соблюдает предположение о независимости на этом уровне. Более крупные единицы обменивают эффективность на снижение интерференции; единица часто выбирается так, чтобы вместить спилловеры, даже если это снижает эффективность.

Метрики

Три типа метрик, определяемых до старта эксперимента:

- Первичная (primary): единственная метрика, по которой принимается решение ship/kill.
- Вторичные (secondary): диагностические метрики, помогающие интерпретировать первичную; не являются подтверждающими.
- Guardrail: метрики, которые не должны деградировать (latency, error rate, нарушения политик); они останавливают эксперимент независимо от первичной.

Overall Evaluation Criterion (OEC) — это явная комбинация метрик (взвешенная сумма или вето по guardrail), используемая, когда единственной метрики недостаточно для решения о выкатке.

Ratio-метрики, такие как $CTR = \frac{\text{clicks}}{\text{impressions}}$, не являются выборочными средними. Оценка дисперсии требует дельта-метода или bootstrap; обращение с ними как со средними занижает дисперсию и даёт слишком узкие доверительные интервалы. Перцентильные метрики и метрики задержки аналогично нелинейны; дельта-метод на них не обобщается, поэтому стандартом являются bootstrap или специализированные оценщики.

Статистическая значимость отличается от практической значимости. Малый прирост на очень большой выборке достигнет значимости при любом разумном α ; MDE (минимально

различимый эффект) следует устанавливать на уровне, оправдывающем инженерные и эксплуатационные затраты, а не только наименьшего статистически различимого эффекта. Типичные способы выбрать порог: соотношение затрат и выгод по инженерному времени, историческое распределение ранее выкаченных приростов (например, 80-й перцентиль успешных запусков) или явная бизнес-цель.

Размер выборки и минимально различимый эффект

Размер выборки определяет MDE: наименьший прирост, который эксперимент способен отличить от нуля при выбранных α (вероятность ошибки Type I) и β (вероятность ошибки Type II).

Для сравнения двух выборочных средних при равном распределении и пуловой дисперсии σ^2 на группу (хорошо аппроксимируется дисперсией контроля при малых эффектах):

$$n \approx \frac{2\sigma^2(z_{1-\alpha/2} + z_{1-\beta})^2}{\delta^2}$$

где δ — MDE, $z_{1-\alpha/2}$ и $z_{1-\beta}$ — квантили стандартного нормального распределения, а n — размер выборки на вариант. При $\alpha = 0.05$ и $\beta = 0.2$ это сводится к $n \approx 16\sigma^2/\delta^2$. Метрики редких событий (хвостовые конверсии, покупки) обладают высокой относительной дисперсией и требуют пропорционально больших выборок для того же MDE.

Для кластерной рандомизации (домохозяйства, гео, пользователи с множеством сессий) эффективный размер выборки меньше, чем сырое число единиц. Design effect равен

$$DEFF = 1 + (\bar{m} - 1)\rho$$

где $\bar{m}$ — средний размер кластера, а ρ — внутрикластерная корреляция. Эффективное n равно $n/DEFF$. Для кластеров с высокой корреляцией DEFF может превышать 100, поэтому гео- и switchback-эксперименты часто недостаточно мощны, даже когда сырая выборка кажется адекватной. Когда размеры кластеров перекошены (Нью-Йорк против Вайоминга), $\bar{m}$ должно быть взвешенным по размеру средним: невзвешенное среднее занижает DEFF.

Цели по размеру выборки также должны соответствовать календарным циклам. Проведение экспериментов кратно полным 7-дневным интервалам контролирует недельную сезонность; остановка в середине недели вносит смещение «будни/выходные», даже если целевой размер уже достигнут. Эксперименты, охватывающие праздники, дни зарплат или продукт-специфичную сезонность, могут быть искажены непредставительным трафиком; стандартное решение — закреплять окно анализа за репрезентативным периодом и исключать праздничные недели.

Распределение трафика

50/50 — значение по умолчанию. Неравные сплиты (например, 90/10) обменивают точность оценки на снижение риска при ранних накатах. Стратифицированное назначение балансирует известную ковариату по бакетам и снижает дисперсию, когда ковариата коррелирует с первичной метрикой.

Определение популяции

Определяйте популяцию для анализа до запуска: все назначенные единицы (ITT), допущенные пред-treatment единицы или triggered-единицы. Типичные исключения — боты, внутренние пользователи и сотрудники, opt-out-когорты и пользователи, никогда не входящие в trigger-окно. Исключения, определённые независимо от воздействия, увеличивают мощность; исключения, зависящие от воздействия, портят эксперимент.

Назначение и устойчивость

Назначения должны быть случайными (независимыми от поведения пользователя и состояния системы), устойчивыми (одна и та же единица остаётся в том же бакете, если только дизайн не ротирует её явно) и аудируемые (соли, namespace и правила исключений должны быть проверяемы). Типичные баги: сброс cookie, дублирование между устройствами, асимметричный учёт неаутентифицированного трафика, фильтры, применяемые после назначения, и фильтрация ботов или внутренних пользователей, работающая по-разному в разных вариантах.

1.1.2 Статистические основы

Частотный (frequentist) фреймворк

Нулевая гипотеза обычно — «эффекта нет»; р-значение — это вероятность наблюдать статистику не менее экстремальную при нулевой гипотезе. Доверительный интервал уровня $1 - \alpha$ покрывает истинный эффект в $1 - \alpha$ долях повторных экспериментов. Односторонние тесты увеличивают мощность, но требуют предварительно зарегистрированной направленной гипотезы.

- Ошибка Type I (α): выкатка неэффективного изменения.
- Ошибка Type II (β): отказ от эффективного изменения.
- Мощность равна $1 - \beta$.

Множественные сравнения

Тестирование k метрик независимо при $\alpha = 0.05$ даёт FWER, равный $1 - (1 - \alpha)^k$, приблизительно $k\alpha$ для малых k и малых α . Десять вторичных метрик при 5% каждая дают по точной формуле 40% ложноположительных результатов.

- Bonferroni: тестирование при α/k . Консервативная процедура; контролирует FWER при произвольной зависимости.
- Holm: step-down Bonferroni. Равномерно мощнее при том же контроле FWER.
- Benjamini-Hochberg: контролирует FDR (ожидаемую долю ложноположительных результатов среди отклонений). Мощнее, но даёт другую гарантию.

Прагматическое правило: используйте контроль FWER (Bonferroni, Holm), когда тестируется несколько подтверждающих первичных метрик и любое ложноположительное приводит к плохому решению о выкатке; используйте контроль FDR (Benjamini-Hochberg) для вторичных и

поисковых метрик, где важнее обнаружение гипотез, чем отдельные ложноположительные. Предварительная регистрация одной первичной метрики избавляет от поправки на множественные сравнения для подтверждающего решения — при условии, что остальные метрики трактуются как диагностические, а не подтверждающие.

Bayesian-фреймворк

Сообщайте $P(\text{treatment} > \text{control} \mid \text{data})$ из апостериорного распределения при выбранном prior. Credible-интервалы заменяют доверительные интервалы; апостериорные вероятности часто сообщаются как более интуитивно понятные стейкхолдерам, чем р-значения.

Bayesian-методы предпочтительнее, когда множество мелких экспериментов разделяют структуру (иерархические priors заимствуют силу между экспериментами), когда затраты асимметричны (теоретико-решающий фреймворк по-разному взвешивает ложноположительные и ложноотрицательные) или когда размер выборки мал настолько, что prior несёт реальный вес. При явно заданной модели, prior и решающем правиле bayesian-мониторинг может поддерживать непрерывные просмотры. Bayesian-оценки не корректируют интерференцию, асимметрию логирования или смещённое назначение.

1.1.3 Снижение дисперсии

CUPED

CUPED (Controlled-experiment Using Pre-Experiment Data) снижает дисперсию, регрессируя метрику эксперимента на пред-периодную ковариату и анализируя остаток. Снижение дисперсии масштабируется как ρ^2 , где ρ — корреляция между метрикой экспериментального периода и пред-периодной ковариатой; Deng et al. (2013) сообщают 30–70% на метриках Microsoft, которые они изучали. CUPAC (Control Using Predictions As Covariates) обобщает CUPED, заменяя линейную ковариату ML-моделью, обученной на пред-экспериментальных признаках.

Стратификация

Балансирует назначение по известным сегментам (страна, платформа, стаж). Снижает дисперсию пропорционально тому, насколько сегмент объясняет вариацию метрики. Пост-стратификация (регрессионная корректировка постфактум) асимптотически эквивалентна стратифицированному назначению при рандомизации; в конечных выборках с несбалансированными сегментами пред-стратификация надёжнее. CUPED сам по себе является частным случаем регрессионной корректировки на базовые ковариаты.

Кластерно-устойчивый вывод

Когда единица анализа мельче единицы рандомизации (рандомизируем по пользователю, анализируем по сессии), стандартные ошибки, вычисленные при IID-предположениях, неверны, а ошибка Type I раздута. Исправления: агрегировать до единицы рандомизации, использовать

кластерно-устойчивые стандартные ошибки или применять блочный bootstrap на уровне кластера.

Подрезание выбросов

Метрики дохода и времени просмотра имеют тяжёлые правые хвосты, где малая доля пользователей даёт большую часть дисперсии. Winsorizing на уровне 99-го или 99.9-го перцентиля снижает дисперсию ценой малого смещения; порог должен задаваться по пред-экспериментальным данным и фиксироваться для всех групп, так как порог, выбираемый по бакету, повторно вводит смещение. Winsorization сдвигает оценочный объект к эффекту на «обрезанном среднем», поэтому её следует считать частью определения метрики, а не безобидным трюком устойчивости.

Логарифмирование ($\log(1 + x)$) — альтернатива для метрик с тяжёлым хвостом. Сравнение средних лог-преобразованных значений измеряет геометрическое среднее, а не арифметическое — это другой оценочный объект, иногда более защитимый, но не взаимозаменяемый с непреобразованным средним.

Когда осмысленный эффект сосредоточен в самом хвосте (P95 длительности сессии меняется, а среднее — нет), инструментом является квантильная регрессия или bootstrap на интересующем квантиле. Снижение дисперсии на среднем не помогает, если решающее движение происходит в хвосте.

Trigger-based анализ

Многие фиши затрагивают только пользователей определённой кодовой ветки (клик по кнопке, просмотр элемента, вход в флюу). Ограничение анализа triggered-пользователями увеличивает размер эффекта на пользователя и устраняет разбавление за счёт пользователей, которых изменение не коснулось. Это безопасно, когда trigger определён независимо от воздействия. Когда trigger зависит от воздействия, counterfactual triggering (также называемый ghost logging или shadow logging) может восстановить несмещённый или существенно менее смещённый анализ за счёт логирования условия trigger в обеих группах независимо от того, какая ветка была пройдена, при условии, что допуск определён симметрично и логируется надёжно. См. ниже режим отказа Triggered vs intent-to-treat для случая со смещением.

1.1.4 Режимы отказа

Peeking и ранняя остановка

Многократная проверка значимости с остановкой при $p < 0.05$ раздувает ошибку Type I. Повторные peeks при номинальном $\alpha = 0.05$ толкают истинную частоту ложноположительных существенно выше 5%; инфляция растёт с числом просмотров и корреляцией теста между ними (Johari et al. 2017). Смягчения: анализ с фиксированным горизонтом, alpha-spending функции, всегда-валидные p-значения (mixture Sequential Probability Ratio Test (mSPRT), confidence sequences), bayesian-правила остановки с калиброванными priors.

Эффект новизны и primacy

- Эффект новизны: пользователи реагируют на само изменение; эффект воздействия завышен от нескольких дней до недель, затем угасает.
- Primacy (он же change aversion): пользователи медленнее вовлекаются с непривычными интерфейсами; воздействие выглядит плоским или отрицательным сначала, затем улучшается.

Построение графика эффекта воздействия во времени отличает устойчивый прирост от переходного; крупные запуски часто идут достаточно долго, чтобы наблюдать обе фазы.

Интерференция и SUTVA

Stable Unit Treatment Value Assumption требует, чтобы исход одной единицы не зависел от назначений других. Типичные нарушения, с механизмом:

- Спилловеры по социальному графу: поведение treated-пользователей меняет то, что видят и делают необработанные друзья.
- Маркетплейсы с общим инвентарём: действия treated-пользователей потребляют предложение или поднимают цены, с которыми сталкиваются и контрольные пользователи.
- Эффекты production-системы: воздействие заполняет кеш, в которые затем попадает контроль; экспозиция treatment переобучает глобальную модель, скорящую контрольных пользователей; события treatment питают bandits, распределяющих между группами.

Смягчения:

- Кластерная рандомизация: рандомизировать на уровне, где интерференция заканчивается (домохозяйство, команда, регион).
- Switchback-эксперименты: чередовать treatment и control во времени на всей популяции.
- Гео-эксперименты: назначать по географии, когда сервис сегментирован по регионам.
- Двудольная (bipartite) рандомизация: на двусторонних маркетплейсах рандомизировать обе стороны независимо и анализировать interaction-член, чтобы напрямую количественно оценить спилловер.
- Сетевые дизайны, такие как ego-cluster (назначать плотно связанные пользовательские кластеры вместе) или exposure-mapping (моделировать дозу от treated-соседей) для социальных экспериментов.

Triggered против intent-to-treat

Intent-to-treat (ITT) измеряет эффект на всех, кому назначен treatment, столкнулись они с изменением или нет. Triggered-анализ ограничивается единицами, которые действительно с ним столкнулись. Triggered-анализ безопасен, когда trigger определён независимо от воздействия (пред-treatment допуск). Когда воздействие меняет, кто попадает в trigger-окно (пост-treatment экспозиция), triggered-анализ смещён; ITT — консервативный дефолт.

Sample ratio mismatch

SRM означает, что наблюдаемые числа назначений отличаются от ожидаемых сильнее, чем объясняет случайность (например, эксперимент 50/50 даёт сплит 49.2/50.8 на большом трафике). Причина лежит выше анализа: баг назначения, баг логирования или фильтр исключения, различающийся по вариантам. Стандартная проверка — χ^2 -тест на количествах по бакетам. Сильный SRM означает, что путь рандомизации или логирования сломан настолько, что интерпретация небезопасна; $p < 0.001$ — типичный порог тревоги, хотя точный отсеочный критерий — вопрос политики. Дебажат итеративным разрезанием χ^2 по версии браузера/приложения, платформе, стране, дню недели или источнику трафика, пока не локализован затронутый сегмент.

Сбалансированные количества по бакетам не гарантируют сбалансированности базовых популяций. Проверки баланса пред-treatment ковариат (страна, устройство, стаж, историческая активность) — та же логика, применённая на уровень глубже: любой дисбаланс на пред-периодных переменных по бакетам сигнализирует о проблеме назначения или фильтрации, которую SRM мог пропустить.

Асимметрия логирования и деноминаторный дрейф

Treatment логирует событие, которое не логирует control, или наоборот, — и каждая нижестоящая метрика становится несравнимой. Тот же отказ проявляется на знаменателях ratio-метрик: treatment может изменить число показов, opportunity count или долю успешных рендеров, отчего CTR или конверсия-на-сессию двигаются по «знаменательным» причинам, а не из-за реальных изменений релевантности. A/A-тесты до запуска ловят баги симметрии логирования. SRM-проверки ловят многие баги дисбаланса бакетов. Деноминаторный дрейф ловится логированием opportunity counts как отдельной метрики и проверкой их сопоставимости до интерпретации ratio.

Парадокс Симпсона — родственный, но отдельный режим: агрегированный CTR может двигаться в направлении, противоположном CTR любого сегмента, когда treatment меняет состав сегментов. Отчёт по ключевым метрикам в разрезе сегментов рядом с агрегатом ловит этот случай.

Пропущенные данные и отложенные исходы

Покупки, подписки, отток и удержание часто имеют отложенные или цензурированные метки. Чтение результатов до созревания меток смещает оценки в сторону краткосрочного поведения. Фиксируйте окна атрибуции до запуска, согласовывайте длительность эксперимента с созреванием меток и сравнивайте ранние и зрелые оценки на одной и той же когорте, чтобы детектировать смещение.

Каннибализация

Изменение может поднять локальную метрику (клики по новому виджету), оставляя глобальную метрику плоской, потому что новое вовлечение пришло с других поверхностей, а не было приростным. Total-session, total-revenue и кросс-поверхностные guardrail ловят это. В e-commerce и рекомендательных системах прирост +10% на новом модуле при плоском доходе

сессии — предупреждающий сигнал о перетекании кликов, хотя сам по себе не диагноз. Плоские глобальные метрики могут также объясняться недостаточной длительностью, шумом измерений или отложенными нижестоящими исходами, поэтому каннибализацию следует подтвердить кросс-поверхностными метриками до того, как она станет основанием для решения о выкатке.

Гетерогенные эффекты воздействия

Средний эффект воздействия может скрывать противоположные эффекты в разных сегментах. Предварительно зарегистрированные сегментные анализы валидны для подтверждающего решения; пост-хок-нарезка «пока сегмент не станет значимым» — это p-hacking под другим именем. Drill-down ради понимания, почему изменение работает, легитимен и часто необходим, и он отличается от drill-down для решения go/no-go: нарезка ради объяснения механизма информирует следующую итерацию, но не является основанием для выкатки во вновь найденный сегмент. Causal forests и uplift-моделирование оценивают Conditional Average Treatment Effect (CATE) без поправки на множественные сравнения по сегментам, но требуют достаточно данных на сегмент.

Winner's curse и регрессия к среднему

Среди экспериментов, достигающих значимости, наблюдаемые размеры эффектов смещены вверх: победителями оказываются те, чей шум перевёл их через порог. Приросты при репликации в среднем меньше начальных. Shrinkage к prior-среднему или подтверждающий второй эксперимент — стандартные смягчения при решении о пограничной победе.

Долгосрочные и краткосрочные эффекты

Двухнедельный эксперимент захватывает две недели поведения. Long-term holdout (небольшая популяция, остающаяся на контроле бессрочно) измеряет эффекты, которые накапливаются или затухают за месяцы. Holdout используются избирательно для крупных запусков, где долгосрочное влияние неопределённо, потому что их дорого поддерживать. Глобальный holdback (когорта, удержанная от всех фиच, выкаченных за квартал) измеряет кумулятивное влияние, которое обычно меньше суммы отдельных A/B-приростов (см. Каннибализацию выше).

Экосистемные эффекты

Метрики, обращённые к пользователю, могут улучшаться, в то время как экосистема, от которой зависит продукт, деградирует: экспозиция авторов концентрируется, разнообразие предложения падает, ликвидность маркетплейса снижается. Экосистемные метрики (коэффициент Gini по показам авторов, вовлечение новых авторов, покрытие категорий) — guardrail для двусторонних платформ.

Взаимодействия между одновременными экспериментами

Когда параллельно идёт много экспериментов, эффекты воздействия могут взаимодействовать. Два изменения, каждое из которых двигает метрику на +1%, могут в сумме дать 0% или +3%.

Смягчения: ортогональные слои назначения, снижающие нежелательные паттерны пересечения и упрощающие интерпретацию (но не устраняющие подлинные продуктовые взаимодействия); эксклюзивность по доменам для высокорисковых экспериментов; плановый мониторинг взаимодействий на общих первичных метриках.

A/A-тесты

Запускайте treatment, равный control, как платформенную проверку здравомыслия. На многих A/A-тестах р-значения должны быть равномерны на $[0, 1]$, а наблюдаемая частота ложноположительных при $\alpha = 0.05$ должна быть 5%. Отклонение указывает на сломанный оценщик дисперсии, смещённое назначение или коррелированные единицы, обрабатываемые как независимые.

1.1.5 Альтернативы и расширения

Многорукие бандиты (multi-armed bandits)

Многорукие бандиты адаптируют распределение трафика к лучше работающим рукам по мере накопления свидетельств (Thompson sampling, Upper Confidence Bound (UCB), epsilon-greedy). Выбор между бандитами и A/B-тестами касается того, что эксперимент оптимизирует: бандиты минимизируют кумулятивный regret во время обучения; A/B-тесты максимизируют точность оценки конкретного контраста в конце. Используйте бандиты, когда непрерывная разведка — часть продуктового механизма; используйте A/B, когда цель — чистое решение ship/kill с точной оценкой эффекта. См. холодный старт для бандитов в cold-start-сценариях.

Sequential testing

mSPRT, group-sequential дизайны и confidence sequences позволяют непрерывный мониторинг без раздутия alpha. Они полезны, когда ожидание фиксированного горизонта дорого или рискованно. Когда верхняя граница доверительного интервала по частичной выборке уже ниже MDE, продолжение тратит слот впустую.

Switchback-эксперименты

Чередуют treatment и control во времени на всей популяции. Стандарт на маркетплейсах (ценообразование, диспетчеризация, матчинг), где user-level рандомизация нарушает SUTVA. Длина блока должна превышать время стабилизации эффекта воздействия; иначе carryover между блоками загрязняет оценку. Washout-период (отбрасывание первых нескольких минут данных в начале каждого блока) удаляет остаток carryover.

Interleaving

Для сравнений ранжирований interleaving смешивает результаты treatment и control-панкеров в одном ответе и атрибутирует клики ранкеру-источнику. Team-Draft Interleaving (TDI) — каноническая разновидность. [Chapelle et al. \(2012\)](#) и последующие исследования в Yandex и Microsoft сообщают, что interleaving обычно в 10–100 раз чувствительнее, чем user-level A/B для

ранжирующих сравнений, причём множитель меняется по поверхностям. Применимо только там, где interleaving представим (поиск, рекомендации в виде списка).

Квази-эксперименты

Когда рандомизация невозможна (регуляторная выкатка, поэтапный запуск, исторический анализ):

- Difference-in-differences (DiD): сравнить изменение исходов для treated-группы и сопоставимой untreated-группы между двумя периодами при предположении параллельных трендов.
- Synthetic control: построить взвешенную комбинацию untreated-единиц, повторяющую пред-периодную траекторию treated-единицы; пост-периодный разрыв оценивает эффект воздействия.
- Regression discontinuity (RD): когда treatment назначается по резкому порогу на непрерывной переменной, единицы чуть выше и чуть ниже порога квази-рандомизированы.

Каждый из них идентифицирует контрфактуал при предположениях, которые необходимо защищать. См. counterfactual evaluation для оффлайн-альтернатив на основе логированных данных.

1.1.6 Операционная практика

Pre-registration

Зафиксируйте первичную метрику, ключевые сегменты, окно анализа и решающее правило до просмотра данных.

Экспериментальная платформа

Стандартные возможности:

- Audit trail: кто, что, когда запустил, с каким фильтром.
- Автоматические SRM-проверки на каждом работающем эксперименте плюс периодическая A/A-валидация на уровне платформы.
- Ортогональные слои назначения, чтобы пересекающиеся эксперименты не давали коррелированных пар.
- Версионированные определения метрик. Логика метрик, правила атрибуции, фильтры ботов и обработка отложенных событий должны быть заморожены на длительность эксперимента; их изменение в середине прогона тихо ломает сопоставимость.
- Стандартные оценщики дисперсии по типу метрик (дельта-метод для ratio, bootstrap для перцентилей).

Постепенный накат

1% → 5% → 25% → 50% — типичный путь. Ранние накаты ловят интеграционные баги и SRM. Поздние накаты дают статистическую мощность. Guardrails откатывают эксперимент на один

шаг назад, а не полностью, если регрессия не сильная.

Решающий фреймворк

Выкатывайте, когда первичная метрика двигается в предварительно зарегистрированном направлении не менее чем на MDE, guardrails нейтральны, а предварительно зарегистрированные сегменты не сигналижат. Итерируйте, когда первичная плоская, но диагностическая метрика объясняет почему. Убивайте, когда guardrails деградируют или первичная двигается против направления. Пограничные победы идут на подтверждающий второй эксперимент до любого решения о выкатке.

flowchart TD

```
R[End-of-experiment readout] --> P{Primary metric<br/>direction & MDE}
P -->|Against direction| K[Kill]
P -->|Flat| D{Diagnostic<br/>explains why?}
D -->|Yes| I[Iterate]
D -->|No| K
P -->|≥ MDE, correct direction| G{Guardrails<br/>neutral?}
G -->|Regress| K
G -->|Yes| S{Pre-registered<br/>segments OK?}
S -->|Flag| K
S -->|OK| C{Borderline<br/>or robust?}
C -->|Borderline| Re[Confirmatory<br/>second experiment]
C -->|Robust| Sh[Ship]
```

Когда первичная, ключевая диагностическая и guardrail указывают в разные стороны, решение о выкатке становится оценочным суждением. Решение должно приниматься со стороны продукта.

1.1.7 Применения

Рекомендации и ранжирование

См. рекомендательные системы для полного списка подводных камней. Специфичные для ранжирования дополнения: interleaving вместо user-level A/B, экосистемные метрики для двусторонних платформ, диагностика источников кандидатов, когда A/B ранкера даёт плоскую или отрицательную первичную.

Запуск моделей

Shadow-трафик → canary → A/B → накат. Оффлайн-оценка на валидации даёт ожидаемый размер эффекта; A/B подтверждает его онлайн. Полный поток — типичный паттерн ML System Design.

Маркетплейсы и ценообразование

Unit-level рандомизация ломается, когда supply или цена взаимодействуют между популяцией. Switchback (временной) и гео-рандомизация (пространственная) — стандартные инструменты.

Спилловер сжимает наблюдаемый прирост относительно истинного эффекта.

1.1.8 Типичные ошибки

Действия практиков, на которые стоит обращать внимание, отдельно от режимов отказа выше:

- Пост-хок-нарезка до тех пор, пока какой-то сегмент не достигнет значимости, и выкатка в этот сегмент.
- Выкатка с пограничной первичной метрикой при красном guardrail из-за «большого размера прироста».
- Экстраполяция двухнедельного прироста на годовое влияние умножением на 26.
- Трактовка незначимого результата в непомощном эксперименте как свидетельства нулевого эффекта.
- Выбор ОЕС после просмотра данных.
- Переопределение первичной метрики в середине эксперимента.
- Обращение с сессиями как с независимыми при рандомизации по пользователю.
- Смешивание статистической значимости с практической значимостью на масштабе.
- Изменение логики метрик, атрибуции или фильтрации ботов в середине эксперимента.

1.1.9 Ссылки

- Kohavi, Tang, Xu — *Trustworthy Online Controlled Experiments* (Cambridge University Press, 2020).
- Microsoft ExP Platform publications.
- Deng, Xu, Kohavi, Walker — CUPED (2013).
- Johari, Koomen, Pekelis, Walsh — Always Valid Inference (2017).
- Tang, Agarwal, O'Brien, Meyer — Overlapping Experiment Infrastructure (Google, 2010).
- Chapelle, Joachims, Radlinski, Yue — *Large-scale validation and analysis of interleaved search evaluation* (ACM TOIS, 2012).
- X5 Tech — A/B-тесты в ритейле.

1.2 CUPED (Controlled-experiment Using Pre-Experiment Data)

CUPED позволяет A/B-тестам детектировать меньшие эффекты без увеличения числа пользователей. Метрика каждого пользователя за период эксперимента частично предсказуема по его пред-экспериментальному поведению; вычитание этой предсказуемой части оставляет меньший остаток, а меньшая дисперсия остатка означает меньший MDE (минимально различимый эффект).

CUPED (Controlled-experiment Using Pre-Experiment Data) снижает дисперсию в рандомизированных онлайн-экспериментах, регрессионно убирая вариацию, объясняемую пред-экспериментальным поведением пользователя. Базовая техника — control-variate-корректировка, применённая к A/B-тестам; см. A/B-тесты как родительскую тему.

1.2.1 Формула

flowchart LR

Pre[Pre-period:
observe X_{pre}] --> Assign[Random
assignment]

Assign --> Exp[Experiment period:
observe Y per arm]

Exp --> Adj["Adjust: $Y_{cuped} = Y - \theta(X_{pre} - \bar{X}_{pre})$ "]

Adj --> Cmp[Compare arms
on residual]

CUPED регрессирует метрику экспериментального периода на пред-периодную ковариату и анализирует остаток:

$$Y_{cuped} = Y - \theta(X_{pre} - \bar{X}_{pre})$$

где:

- Y — метрика экспериментального периода.
- X_{pre} — ковариата, измеренная до назначения (как правило, та же метрика или коррелированная с ней, измеренная в пред-экспериментальном окне).
- $\bar{X}_{pre}$ — пуловое среднее X по группам.
- $\theta = \text{Cov}(Y, X_{pre}) / \text{Var}(X_{pre})$ — оптимальный коэффициент control-variate.

Y_{cuped} несмещённая для среднего эффекта воздействия, потому что treatment рандомизирован, а X_{pre} измерен до назначения.

1.2.2 Снижение дисперсии

Дисперсия скорректированного оценщика масштабируется как ρ^2 , где ρ — корреляция между Y и X_{pre} :

$$\text{Var}(Y_{cuped}) = \text{Var}(Y)(1 - \rho^2)$$

Так, $\rho = 0.3$ даёт 9% снижения дисперсии; $\rho = 0.7$ — 51%; $\rho = 0.9$ — 81%.

Deng et al. (2013) сообщают о 30–70% снижении дисперсии на метриках Microsoft, которые они изучали (сессии, выручка), что соответствует ρ примерно от 0.55 до 0.84. Достигнутое снижение на других метриках зависит от конкретной структуры автокорреляции.

1.2.3 Выбор ковариаты

Стандартная ковариата — та же метрика, измеренная за пред-период. Альтернативы:

- Связанная метрика с более высокой автокорреляцией.
- Сумма нескольких пред-периодных сигналов.
- Прогноз модели для Y , обученной на пред-экспериментальных признаках (см. CUPAC ниже).

Ковариата должна быть измерима до назначения treatment. Пред-treatment ковариата по построению независима от воздействия, что сохраняет несмещённость оценщика.

1.2.4 CUPAC

CUPAC (Control Using Predictions As Covariates) обобщает CUPED, заменяя линейную ковариату ML-моделью, обученной на пред-экспериментальных признаках:

$$Y_{\text{cupac}} = Y - \theta(\hat{Y} - \bar{\hat{Y}})$$

где $\hat{Y}$ — прогноз модели для исхода экспериментального периода только по пред-экспериментальным признакам. Снижение дисперсии по-прежнему масштабируется как ρ^2 , теперь измеряя корреляцию между Y и $\hat{Y}$.

Выигрыш по сравнению с CUPED масштабируется с тем, насколько лучше ML-модель предсказывает Y , чем одиночная линейная ковариата. Предиктор должен использовать только пред-treatment-информацию; риск утечки выше, чем у CUPED, потому что гибкая модель может случайно подхватить пост-присвоенческий сигнал, если пайплайны признаков не проаудированы.

1.2.5 Связь со стратификацией

CUPED — частный случай регрессионной корректировки на базовые ковариаты. Стратифицированное назначение балансирует ковариату по бакетам в момент назначения; пост-стратификация регрессирует на ковариату постфактум. CUPED — это пост-стратификация с непрерывной ковариатой и оптимальным взвешиванием.

При рандомизации пост-стратификация асимптотически эквивалентна пред-стратификации; в конечных выборках с несбалансированными сегментами пред-стратификация надёжнее, потому что она принудительно балансирует, а не оценивает баланс.

1.2.6 Когда CUPED работает плохо

- Новые пользователи без пред-периодного поведения: ковариата не определена или равна нулю, поэтому регрессии не за что зацепиться.
- Метрики, движимые новизной или разовыми событиями: корреляция с прошлым поведением слаба.
- Метрики редких событий: пред-периодная базовая частота слишком мала, чтобы дать пригодную ковариату.
- Метрики с другой популяцией в экспериментальном окне, чем в пред-периоде (сезонный онбординг, региональные запуски).

1.2.7 Типичные подводные камни

- Включение пост-назначенческих признаков в X_{pre} . Ломает несмещённость; завышает заявленное снижение дисперсии, искажая оценку среднего эффекта воздействия (ATE).
- Вычисление θ по-бакетно вместо пуловой оценки. Среднее ковариаты $\bar{X}_{\text{pre}}$ должно быть пуловым по группам.

- Трактовка CUPED как универсального «делителя дисперсии пополам». Выигрыш полностью движется ρ^2 ; при $\rho = 0.3$ снижение дисперсии всего 9%.
- Смешение CUPED со снижением дисперсии в контрфактуальных оценках на наблюдательных данных. CUPED работает потому, что рандомизация гарантирует независимость ковариаты от treatment; применение той же математики к нерандомизированным данным таких гарантий не даёт.

1.2.8 Ссылки

- [Deng, Xu, Kohavi, Walker — CUPED \(2013\)](#). Оригинальная статья.
- Kohavi, Tang, Xu — *Trustworthy Online Controlled Experiments* (Cambridge University Press, 2020). Глава о снижении дисперсии охватывает CUPED и CUPAC с практическими рекомендациями по реализации.

Раздел 2. Расхождение обучения-обслуживания и холодный старт

2.1 Training-serving skew

Training-serving skew — это несоответствие между представлением данных, использованным для обучения и оценки модели, и представлением, доступным во время инференса. Типичный симптом: офлайн-метрики растут, а онлайн-метрики стагнируют или ухудшаются — модель обучена на распределении, которого она не увидит в продакшене, и офлайн-метрики, посчитанные на этом распределении, переоценивают онлайн-качество. Skew — один из data-dependency рисков, каталогизированных в [Sculley et al. \(2015\)](#) о системном техническом долге в ML-системах.

flowchart LR

```

L[Event logs] --> TT[Training-time<br/>feature transforms]
TT --> D[Training data] --> Fit[Model fit] --> M[(Trained model)]
R[Live request] --> ST[Serving-time<br/>feature transforms]
ST --> P[Prediction]
M --> P
P --> SL[Serving logs]
SL -->|next training set| D
TT <-.skew.-> ST

```

Пунктирное ребро отмечает, где два пайплайна могут разойтись (разные кодовые пути, разная свежесть, разные версии схемы), даже когда оба питают одну и ту же модель. Любая сторона может дрейфовать независимо. Логирование обслуженных признаков для дообучения — обратное ребро от serving-логов к обучающим данным — является самой сильной структурной защитой.

2.1.1 Skew, сдвиг распределения и concept drift

Эти три режима отказа имеют общий симптом «офлайн вверх / онлайн плоско», но разные причины и способы исправления. - Skew — инженерный: обучение и инференс получают разные представления из одного пайплайна. - Сдвиг распределения — статистический: $P(X)$ меняется по причинам, не связанным с пайплайном (популяция, сезонность, источник трафика). - Concept drift — тоже статистический: $P(Y | X)$ меняется, поэтому связь между признаками и метками эволюционирует. - Skew исправляется инженерно (общий код преобразований, логирование обслуженных признаков, point-in-time join). - Сдвиг распределения решается переобучением на свежих данных или importance weighting, когда прошлые данные ещё информативны. - Concept drift требует переобучения на свежих данных и иногда архитектурных изменений (последовательные модели, признаки свежести).

2.1.2 Симптомы

Сигналы, указывающие на skew, а не на drift или ошибку моделирования:

- Офлайн-метрики улучшаются, а A/B-тест плоский или отрицательный.
- Гистограммы одного признака различаются между обучающими входами и serving-логами.
- В продакшене появляются разрывы качества по сегментам, которых не было на отложенной офлайн-выборке.
- Распределения выходов модели расходятся с офлайн-предсказаниями при повторном проигрывании (replay) serving-входов.
- Только что добавленные признаки в первый день показывают большой skew; разрыв сокращается только по мере накопления продакшен-истории.
- Офлайн-валидация выглядит сильной, но бэктекст или shadow-оценка на залогированных продакшен-признаках хуже.

2.1.3 Как возникает skew

Расхождение реализаций

- Расходящиеся кодовые пути: обучающие и serving-пайплайны заново реализуют одно и то же преобразование. Мелкие различия в токенизации, нормализации строк, значениях по умолчанию для пропусков или округлении плавающей точки складываются в разрыв признаков.
- Двойная запись batch-streaming: один и тот же признак производится Spark batch-задачей для обучения и Flink/Kafka Streams задачей для инференса. Даже когда оба корректны, различия в окнах, семантике watermark, обработке поздних событий и поведении backfill создают skew между двумя путями.
- Несовпадение категориального кодирования: обновления словаря, изменения хэш-коллизий или правила обработки out-of-vocabulary (OOV) меняются на одной стороне без другой. Обновление словаря на обучающей стороне, добавляющее новые категории без скоординированного serving-депоя, оставляет инференс со старым

словарём. Новые категории, на которых модель обучалась, тогда уходят в OOV на serving и получают общий эмбединг, против которого модель никогда не обучалась.

- Версионирование embedding-сервиса: когда эмбединги пользователей/айтемов производятся upstream-сервисом и потребляются downstream-моделью, версия embedding-модели становится частью контракта признаков. Деплой новой embedding-модели без согласования с потребителями немедленно даёт skew векторов признаков; это часто встречается в retrieval-пайплайнах «двухбашенная модель», где эмбединги айтемов обновляются независимо от ranking-модели. ANN-индексы (HNSW, ScaNN, FAISS) требуют той же согласованности версий с embedding-моделью, которая их построила, иначе индекс возвращает несовпадающих соседей для живого query-эмбединга.
- Skew доступности признаков: признак существует во время обучения, но часто отсутствует или недоступен на serving. Типичные случаи: real-time признак тайм-аутится и тихо откатывается на default, агрегат в хранилище не имеет онлайн-аналога, или обогащение не успело завершиться для новых айтемов на момент запроса. Skew молчалив, когда значение fallback выглядит валидно: fallback 0.0, против которого модель обучалась на настоящих 0.0, не триггерит проверки распределений.
- Skew канарі или раскатки: когда новая модель обслуживает долю трафика рядом со старой, различия в векторах признаков между версиями (новая модель использует признак, которого нет у старой) загрязняют A/B-сравнение. Запуск нового признака в shadow до того, как модель его использующая выходит в live, сохраняет сравнение чистым.
- Cold-path vs hot-path serving: деплой обслуживает часть запросов через real-time path признаков, а часть — через закэшированный или предвычисленный score; два пути дают разные предсказания для одного пользователя.

Ошибки обработки времени

- Темпоральная утечка данных: значение признака на момент обучения вычислено из данных, которые ещё не были доступны в момент предсказания. Агрегаты вроде `user_7d_click_count`, наивно соединённые join'ом, втягивают клики, случившиеся после показа. Отличается от утечки целевой переменной и групповой утечки; фикс — point-in-time join, а не удаление признака.
- Lookahead-bias в скользящих агрегатах: `feature_t = mean(window ending at t)`, вычисленное против `processing_time`, а не `event_time`, даёт значения, которых не было в момент предсказания. Подвид темпоральной утечки данных с собственной отладочной сигнатурой.
- Несовпадение свежести (staleness): признак читается на одной свежести офлайн (вчерашний batch-снэпшот) и на другой онлайн (10-минутный streaming-агрегат). Обучающее распределение не совпадает с serving даже когда совпадает кодовый путь.
- First-day / backfill skew: когда добавляется новый признак, обучающие данные содержат полную историческую реконструкцию, а продакшен на момент деплоя имеет только то, что накопилось с момента запуска. Skew сокращается по мере того, как продакшен-история нагоняет.

Изменения схемы и пайплайна

- Изменения схемы и определений: схема бакетизации признака, кодирование enum, единицы измерения или логика вычисления меняются upstream без скоординированного переобучения.
- Различия в семплировании и фильтрации: обучение применяет фильтрацию негативных примеров, дедупликацию или downsampling, которых нет в serving-пути.
- Расхождение в построении меток: офлайн окно меток или правило атрибуции отличается от онлайн reward-сигнала, который модель обучена предсказывать.
- Skew от петли обратной связи: задеплоенная модель меняет данные, на которых она будет дообучаться. Рекомендер видит метки только для айтемов, которые показал; модель фрода блокирует high-risk события, поэтому будущие обучающие данные недопредставлены настоящими положительными.

2.1.4 Обнаружение и устранение

Самая сильная структурная защита — логирование обслуженных признаков для обучения. На serving логируйте точный вектор признаков и предсказание; обучайте следующую модель на этих логах. Это убирает многие ошибки офлайн-реконструкции, но не заменяет мониторинг свежести, валидацию схемы, проверки меток или replay-тесты. Правило #29 из [Rules of ML Zinkevich](#).

Полезный лог обслуженных признаков включает: версию модели, имена признаков с версиями, сырые значения признаков (до подстановки default), преобразованные значения признаков (после препроцессинга), score предсказания, источник кандидата или контекст запроса, временную метку чтения признака и индикаторы fallback/таймаута.

Паттерны ниже делятся на три группы: структурные защиты (архитектурное предотвращение), активное тестирование (диф-сравнение на уровне входов) и непрерывный мониторинг (наблюдение за продакшен-трафиком).

Point-in-time join

Когда обучающие данные реконструируются из event-логов, каждое значение признака должно соединяться по состоянию на момент предсказания, а не «сейчас». Feature store (Feast, Tecton, внутренние эквиваленты) реализуют это как корректность по точке во времени в offline store. Темпоральная утечка данных производит модели, которые хорошо подгоняются под обучающие данные и недотягивают онлайн; point-in-time join предотвращают её конструктивно.

Неправильный паттерн: предсказание в понедельник 10:00 → join последней строки user-profile из таблицы среды. Правильный паттерн: предсказание в понедельник 10:00 → join строки user-profile, видимой не позднее понедельника 10:00. Та же логика применяется к агрегатам, эмбедингам, метаданным айтемов и окнам меток.

Паритет онлайн/офлайн

Обучение и инференс должны вычислять признаки через общий код, в идеале — одну и ту же библиотечную функцию или преобразование feature store. Правило #32 из [Rules of ML Zinkevich](#) рекомендует совместно использовать код преобразований где только возможно. Когда паритет невозможен (разные рантаймы, разные бюджеты latency), unit-тесты на фиксированном входном векторе ловят расхождение рано.

Регрессионное тестирование на основе replay

Более сильная версия unit-тестов: взять выборку недавних serving-запросов, пересчитать их признаки через оба пути (обучения и инференса) и сравнить поэлементно по тысячам векторов. Replay (повторное проигрывание) ловит skew кодовых путей, который пропускают тесты с фиксированным входом, потому что тестовая выборка покрывает реальное пространство признаков, которое модель видит в продакшене.

Shadow-инференс

Запустить кандидатскую модель в shadow-режиме рядом с продакшен-моделью, логировать оба serving-предсказания на одном входе и сравнивать их. Shadow-инференс отличается от shadow-логирования: shadow-логирование захватывает признаки для будущего обучения, shadow-инференс сравнивает две модели на идентичных входах, чтобы выявить skew между версиями моделей. Полезен для отлова багов преобразований, которые валидация схемы упускает, потому что сравнение происходит на реальном продакшен-кодовом пути. Требует дублирующей serving-мощности для кандидатской модели.

Counterfactual-логирование признаков

Когда добавляется новый признак, исторических обучающих данных для него нет. Логирование признака в shadow в течение warm-up периода — вычисление и логирование без использования для предсказаний, иногда называется dark launch или shadow-режим feature-флага — генерирует обучающие данные, которые следующая модель может использовать без first-day skew.

Проверки паритета предсказаний

Для фиксированного набора примеров сравнить офлайн-предсказания модели, онлайн-предсказания модели, предсказания после сериализации/десериализации и предсказания на разных версиях CPU/GPU/рантаймов. Небольшой численный допуск приемлем. Большие различия указывают на skew препроцессинга, экспорта модели, численной точности или версии зависимостей. Экспорт модели и квантование (PyTorch → ONNX/TensorRT/TFLite, FP32 → FP16/INT8) — частые молчаливые источники численного дрейфа. Пороги допуски должны быть ужесточены для квантованных моделей, поскольку само квантование ограничивает достижимый паритет.

Sanity-проверки важности признаков

Только что добавленный признак, который доминирует в офлайн-важности, подозрителен. Исследовать, не утекает ли он будущая информация и доступен ли онлайн. Паттерны, предполагающие skew, а не настоящий сигнал:

- очень высокая офлайн-gain от признака, чьё вычисление включает join по нескольким таблицам или оконные join;
- важность сконцентрирована на чувствительных к свежести агрегатах;
- признак сильный офлайн, но часто отсутствует онлайн;
- лифт исчезает при обучении на обслуженных признаках.

Выравнивание свежести

Продакшен-системы читают признаки на смешанной свежести: real-time сигналы из online store, ежедневные агрегаты из batch-хранилища, снимки из реляционной БД. Обучающие данные должны зеркалить те же слои свежести по признакам. Правило #31 из *Rules of ML Zinkevich* покрывает случай snapshot таблицы. Запись timestamp чтения рядом с каждым значением признака делает свежесть явной по строкам.

Валидация схемы

[TensorFlow Data Validation \(TFDV\)](#) сравнивает статистики обучения и инференса с объявленной схемой и эмитирует Anomalies proto, когда распределения расходятся. Стандартные метрики расстояния — L-infinity-норма для категориальных признаков и приближённая Jensen-Shannon дивергенция для числовых, с порогами на признак. [Polyzotis et al. \(2019\)](#) описывает фреймворк. Проверки распределений имеют ложные срабатывания во время сезонных сдвигов или запуска кампаний, поэтому им нужны либо динамические пороги, либо сравнение holdout vs следующий день, либо ручной просмотр.

Валидация схемы ловит некоторые классы skew (несоответствие категориального словаря, сдвиг числового диапазона), но слепа к другим — skew кодового пути, сохраняющий маргинальные распределения, проходит проверки схемы целиком.

Измерение skew по трём окнам

Правило #37 из Rules Zinkevich: измерять skew на трёх временных окнах, чтобы отделить skew от drift.

- обучение vs офлайн holdout: ловит проблемы моделирования и разбиения.
- офлайн holdout vs данные следующего дня: ловит проблемы пайплайна данных или темпоральной реконструкции.
- данные следующего дня vs live serving-логи: ловит skew serving-пути. Это единственное окно, которое прогоняет онлайн-кодовый путь; первые два сравнивают batch-выходы с batch-выходами.

Граница, на которой впервые появляется разрыв, идентифицирует вероятный источник. Разрыв только между обучением и holdout указывает на проблемы моделирования или разбиения.

Разрыв, открывающийся на границе holdout-vs-следующий-день, указывает на проблемы пайплайна или темпоральной реконструкции. Разрыв, открывающийся только на границе следующий-день-vs-live, указывает на проблему кодового пути или рантайма.

2.1.5 Диагностика A/B-тестов

Диагностические вопросы, когда офлайн-выигрыши не воспроизводятся онлайн:

- Получила ли онлайн-модель те же значения признаков, что и офлайн-валидация?
- Различаются ли частоты missing или default онлайн против офлайн?
- Отличается ли свежесть признаков от обучающей реконструкции?
- Менялась ли схема признака или upstream-определение в ходе эксперимента?
- Было ли окно меток выровнено с онлайн-метрикой?
- Совпадает ли экспериментальная популяция с валидационной?
- Выросли ли частоты fallback/таймаута в treatment?
- Опиралась ли модель на признак, сильный только для залогированных или показанных примеров?

2.1.6 Операционная практика на масштабе

При сотнях признаков и миллионах ежедневных предсказаний skew — это проблема непрерывного мониторинга, а не разовое отладочное упражнение:

- Per-feature дашборды на расстояние распределений обучение-vs-инференс (population stability index, Jensen-Shannon дивергенция, Колмогоров-Смирнов), с порогами алертов.
- Triage-очередь для небольшого числа признаков, превышающих пороги каждый день.
- Pre-deployment проверки skew как release-gate: кандидатская модель, чьи признаки показывают свежий skew на калибровочной выборке, блокирует раскатку.
- Мониторинг skew по срезам (страна, устройство, сегмент), поскольку глобальные агрегатные метрики скрывают skew на маленьких высокоценных сегментах.
- Координированное управление изменениями при модификации общих upstream-признаков: изменение схемы признака затрагивает каждую модель, которая его потребляет.
- Окно прогрева модели после деплоя: первые часы после свежего деплоя могут показывать переходный skew от cache miss, autoscaling и прогрева пайплайна признаков. Пороги алертов должны быть ослаблены в течение определённого интервала прогрева, а не срабатывать на этот переходный шум.

2.1.7 Типичные режимы отказа

Паттерны skew

- Default-значение вроде 0 означает одновременно «отсутствует» и настоящий ноль, поэтому модель не может их различить.

- Streaming-агрегат сбрасывает счётчики при рестарте оператора, создавая молчаливый онлайн-сдвиг распределения.
- Новый признак работает для head-пользователей, но отсутствует для tail-пользователей; глобальные проверки распределения проходят, а модель проваливается на хвосте.
- Tree-модели вроде XGBoost и LightGBM учат default-направление в каждой развилке для NaN-входов; если serving-JSON опускает пропуски, а парсер дефолтит их в 0.0, дерево идёт по ветке 0.0 вместо missing-value default-направления, что может существенно ухудшить точность, пока маргинальное распределение признака выглядит нормально для валидации схемы.
- Пайплайны признаков молча падают в выходные или праздники, когда downstream batch-задачи не запускаются, поэтому модель работает на устаревших признаках, пока проверки распределений (откалиброванные против только-будничной свежести) молчат.
- Feature store гарантирует паритет схемы, но не паритет свежести, и skew свежести остаётся незамеченным.

Антипаттерны в обработке skew

- Модель переобучают на данных, произведённых serving-багом, подгоняя баг как паттерн признаков, который последующие предсказания воспроизводят.
- Калибровка используется как заплатка для дрейфа score, вызванного skew признаков, вместо исправления пути признаков.
- Офлайн replay использует только успешные запросы и пропускает serving-отказы.
- Shadow-serving сравнивает предсказания, но не значения признаков, скрывая корневую причину.
- Тестовые выборки построены из обучающих признаков (которые наследуют skew пайплайна), поэтому офлайн-метрики на тестовом разбиении не могут выявить онлайн-офлайн расхождение.

2.1.8 Ограничения

- Логирование обслуженных признаков защищает от skew преобразований, но не от шума меток или upstream-проблем качества данных.
- Per-feature паритет всё ещё может пропустить проблемы уровня взаимодействий: пара признаков, валидная независимо, может быть несогласованной совместно.

2.1.9 Ссылки

- Sculley et al. — *Hidden Technical Debt in Machine Learning Systems* (NeurIPS 2015)
- Zinkevich — *Rules of Machine Learning: Best Practices for ML Engineering* (Правила #29, #31, #32, #37)
- TensorFlow Data Validation — Skew and Drift Detection
- Polyzotis, Roy, Whang, Zinkevich — *Data Validation for Machine Learning* (MLSys 2019)
- Yan — *A Practical Guide to Maintaining Machine Learning in Production*

2.2 Холодный старт (cold start)

Холодный старт — это проблема построения полезных рекомендаций, когда у пользователя, айтема или рынка мало или нет надёжной истории взаимодействий. Сюда входят как сущности с истинно нулевой историей, так и с разреженной историей, чьего наблюдаемого поведения ещё недостаточно, чтобы доверять рекомендательной модели. Формализация для новых айтемов и метрики качества рекомендаций восходят к [Schein, Popescul, Ungar, Pennock \(SIGIR 2002\)](#). Та же структурная проблема (отсутствие исторического сигнала для свежей сущности) встречается в ранжировании, классификации и онлайн-экспериментировании всякий раз, когда новая сущность входит в пайплайн, обученный на залогированном поведении.

Три обычно различаемых случая:

- Новый пользователь: нет прошлых взаимодействий для персонализации.
- Новый айтем: нет кликов или оценок, из которых можно выучить коллаборативный сигнал.
- Новое сообщество: компания новая, поэтому нет ни истории пользователей, ни истории айтемов.

Холодный старт отличается от исследования-эксплуатации (exploration-exploitation). Холодный старт — это отсутствие сигнала для конкретной сущности; исследование (exploration) — это политический выбор того, как балансировать обучение против полезности. Они связаны через один механизм: холодная фаза — это когда исследовательский член бандита наибольший, потому что апостериорная дисперсия наибольшая для невиданных сущностей.

flowchart LR

CS[Cold start] --> NU[New user]

CS --> NI[New item]

CS --> NC[New community]

NU -->|context, onboarding, bandits, inductive embeddings| WU[Personalized def

NI -->|content features, creator priors, guaranteed exposure| WI[Surfaceable c

NC -->|editorial curation, content bootstrap, imported signal| WC[Bootable sur

2.2.1 Холодный старт нового пользователя

Подходы, грубо от наименее до наиболее персонализированных:

- Popularity-baseline по умолчанию: показывать глобальный топ айтемов, опционально стратифицированный по базовым сигналам (страна, язык, устройство, возрастная группа). Покрывает широкую голову предпочтений; теряет точность, как только важна вкусоспецифичная персонализация. popularity-prior также — естественная отправная точка для бандитских политик.
- Контекстная персонализация: признаки, доступные на первом запросе (referrer, время, локация, устройство, query, поля формы регистрации). Некоторые платформы хранят теневой профиль, привязанный к device ID, cookie или fingerprint, который накапливает сигнал между до-регистрационными визитами и сливается с зарегистрированным пользователем при регистрации. Оба подхода ограничены политикой конфиденциальности.

- Импорт сигнала с других поверхностей: пользователь, холодный на одной поверхности (видео), часто тёплый на другой поверхности той же платформы (поиск, соцсеть, маркетплейс). Общий эмбединг пользователя или кросс-поверхностные поведенческие признаки переносят сигнал через границу. Поведение на одной поверхности может неверно представлять намерение на другой.
- Демографическая кластеризация: назначить нового пользователя ближайшему кластеру существующих пользователей (по атрибутам регистрации или контексту) и рекомендовать топ айтеров кластера. Полезно, когда регистрационные данные есть, но скудны.
- Опрос пользователей в онбординге: оценить или выбрать айтемы, жанры или темы при регистрации, обычное дело в музыкальном и видеостриминге. Более длинная регистрация увеличивает drop-off, а оценки на малом наборе обобщаются неравномерно. Активное обучение может таргетировать айтемы, максимизирующие информацию на оценку.
- Предпочтения, выведенные LLM: свободнотекстовый онбординг («люблю жёсткую sci-fi и стратегии») картируется LLM в узлы таксономии или извлекает эмбединги. Превращает неструктурированный регистрационный ввод в пригодный холодный профиль без длинного потока оценивания.
- Sequential session models: новый пользователь без кросс-сессийной истории всё равно производит сигнал в первой сессии (просмотренные айтемы, dwell time, глубина скролла, скippy). Sequential-модели семейства SASRec или BERT4Rec производят пригодное in-session представление из этих событий. Отличается от контекстной персонализации (которая использует метаданные запроса) и от онбординга (который использует явный опрос); сигнал здесь неявный — поведение в сессии.
- Бандиты над кандидатскими политиками: политика бандита (многорукий бандит) трактует кандидатские айтемы, сегменты или рекомендательные стратегии как arms; per-user апостериор обновляется по мере взаимодействия пользователя. Контекстные варианты вроде LinUCB и нейронных контекстных бандитов обуславливают выбор arm на наблюдаемом контексте, что нацелено на укорочение холодной фазы.
- Индуктивные embedding-модели: модель, картирующая контекстные признаки в эмбединг пользователя (вместо поиска per-user эмбединга), производит пригодное представление для невиданного пользователя. Общая техника — ID dropout: случайно маскировать ID-токен в OOV во время обучения, заставляя модель проводить backprop через ветвь контента/признаков. У DropoutNet две ветви: коллаборативное/preference-представление и контентное представление, с dropout, применённым к коллаборативной ветви, так что модель производит хорошие эмбединги из одного контента, когда предпочтения отсутствуют. MeLU использует мета-обучение в стиле MAML для быстрой адаптации по нескольким оценкам.

2.2.2 Холодный старт нового айтема

У нового айтема нет кликов, оценок или других коллаборативных сигналов. Подходы:

- Content-only retrieval: закодировать айтем из его контента (текстовое описание, изображение, теги, таксономия) и сделать извлекаемым для пользователей, чей профиль или эмбединг недавней сессии близок к контентному эмбедингу айтема.

Классический content-based маршрут, часто запускаемый как отдельный candidate source рядом с коллаборативной фильтрацией.

- Item-content tower'ы в двухбашенных моделях: «двухбашенная модель» retriever, чья item-башня потребляет контентные признаки, обобщается на невиданные айтемы во время инференса.
- Инициализация ID-эмбединга из контента: в модели с обученными per-item ID-эмбедингами ID-эмбединг нового айтема случаен до накопления градиента. Инициализировать его из контентного эмбединга айтема, центроида ID-эмбедингов похожих айтемов или иерархического fallback (эмбединг бренда или категории, когда ID айтема OOV).
- Реляционное распространение: распространять сигнал через граф знаний или со-встречаемости (новый фильм известного режиссёра наследует аудиторию режиссёра; новый SKU наследует соседство бренд-категории). Индуктивные графовые модели вроде GraphSAGE и PinSAGE вычисляют эмбединг нового узла на лету из эмбедингов его соседей, без переобучения. Отличается от контентных эмбедингов (реляционные, а не описательные) и от creator-prior'ов (структурированные графом, а не одинокий скаляр).
- Creator или seller prior'ы: когда признаки айтема слабы, прошлое поведение создателя, продавца или издателя (CTR канала, conversion rate продавца, прошлое engagement контента) часто предсказывает производительность нового айтема.
- LLM-bootstrapped сигнал: эмбединги айтемов, выведенные LLM из текста или метаданных, и сгенерированные LLM синтетические взаимодействия дают нетривиальный prior для айтемов, чьих ручных контентных признаков недостаточно. Трактуйте синтетические взаимодействия как prior или данные для предобучения, иначе модель подгоняется под предпочтения LLM, а не пользовательской популяции.
- Бюджет гарантированной экспозиции: выделить фиксированный срез показов новым айтемам независимо от score модели, чтобы система строила коллаборативный сигнал, который иначе никогда бы не собрала. Варианты включают per-creator квоты, бусты с возрастным затуханием и бид-бусты новых кампаний или резервированные доли explore-бюджета в рекламных аукционах. Без гарантии экспозиции candidate retrieval часто замораживается вокруг айтемов, которые фаза warmup уже вынесла на поверхность.
- Эффективное распределение исследования: где тратится explore-бюджет, имеет такое же значение, как и его размер. Thompson sampling над новыми айтемами и таргетирование экспозиции на пользователей, чья обратная связь наиболее информативна (активные, разноплановые пользователи), строит коллаборативный сигнал быстрее, чем равномерно случайная экспозиция при том же бюджете. Логируйте propensities, slot, candidate source и правила eligibility для каждого исследовательского показа; без этого последующая оценка не сможет разделить качество айтема от политики экспозиции.
- Canary-экспозиция на тёплом подмножестве: показать новый айтем малому рандомизированному подмножеству тёплых пользователей, исключив его из основной поверхности, затем собрать метки до более широкого продвижения. Отличается от чистого shadow-scoring (вычисление признаков и score без экспозиции), который ловит

баги пайплайна, но не собирает меток пользовательских откликов; только реальная экспозиция производит метки.

2.2.3 Переход тёплый старт

Подходам холодного старта нужен механизм переключения на нормальные модели после накопления достаточных данных. Типичные подходы:

- Count-based интерполяция: смешать холодный prior и тёплый score с весом, затухающим по мере накопления взаимодействий. Распространённая форма — $\alpha = \frac{n}{n+k}$, где n — счётчик взаимодействий, а k — настраиваемая сила prior'a; обслуживаемый score — $(1 - \alpha) \cdot \text{cold} + \alpha \cdot \text{warm}$.
- Байесовская усадка: трактовать холодный prior как априорное распределение над эмбедингом пользователя или айтема и усаживать обученный эмбединг к нему, с усадкой, затухающей по мере падения апостериорной дисперсии.
- Прогрев эмбединга (warm start): инициализировать обученный ID-эмбединг из контентного эмбединга (или центроида эмбедингов похожих айтемов), чтобы представление было непрерывным через переход, а не прыгало от случайного к обученному.
- Confidence-gated маршрутизация: маршрутизировать на холодный путь, пока оценка уверенности (счётчик взаимодействий, апостериорная дисперсия, калиброванная неопределённость предсказания) ниже порога, смешивая через границу вместо переключения на ней.

Холодные и тёплые score должны быть калиброваны или нормализованы на общую шкалу до смешивания; иначе score с большей шкалой доминирует независимо от предполагаемого веса уверенности.

2.2.4 Новая компания

Рекомендер, стартующий с нуля, не имеет ни истории пользователей, ни истории айтемов. Ранняя фаза опирается на любой не-взаимодейственный сигнал, который существует: редакционная курация, контентные эмбединги, демографические prior'ы, сигнал, импортированный из внешнего графа или родственного продукта, и агрессивное исследование. Импортируемый сигнал должен быть юридически пригоден, безопасен с точки зрения приватности и распределённо релевантен новой популяции; иначе он даёт обманчивый prior. В маркетплейсах это двусторонняя проблема начальной загрузки (покупатели привлекают продавцов и продавцы привлекают покупателей, при отсутствии тех и других на запуске); стандартные фиксы — субсидирование одной стороны, ручное засевание предложения и географическая раскатка, чтобы достичь ликвидности локально перед расширением. По мере накопления достаточных взаимодействий система может начать тёплое обучение, пока content-based retrieval продолжает обслуживать хвостовые айтемы. Shadow-логирование и бандитская инфраструктура, введённые в этой фазе, остаются доступны для последующей работы с холодным стартом.

2.2.5 Оценка

Агрегатные офлайн-метрики, посчитанные на полной популяции (такие как NDCG или recall@k), маскируют производительность холодного старта, потому что холодная подпопуляция мала относительно тёплой базы пользователей. Возможные подходы:

- Held-out холодные разбиения: вывести пользователей или айтемы из обучения, чтобы модель никогда не видела их коллаборативной истории, затем оценить на их отложенных будущих взаимодействиях.
- Кривые few-shot производительности: отчитывать первичную метрику как функцию доступной истории (0, 1, 3, 5, 10, 50 взаимодействий). Работающая система холодного старта должна улучшаться по мере накопления данных, а не работать только ровно на нуле или полностью тёплой.
- Декомпозировать time-to-first-interaction на три операционно различимых стадии: время до первого приемлемого retrieval (был ли айтем извлекаем вообще?), время до первого показа (вынесла ли система его на поверхность?) и время до первого положительного взаимодействия (вовлеклись ли пользователи?).
- Покрытие новых айтемов: доля только что добавленных айтемов, получивших хотя бы один показ в течение определённого окна.

2.2.6 Типичные ловушки

- Путать свежесть, холодность, покрытие retrieval и feature lag. Айтем, живой неделю с нулём показов, может быть холодным по любой из нескольких причин: плохое покрытие retrieval (фикс: retrieval), отфильтрован serving-политикой (фикс: политика), не индексирован, потому что ANN-индекс обновляется слишком медленно (фикс: свежесть индекса, инфраструктурная проблема класса training-serving skew), отсутствует exposure-бюджет (фикс: explore-бюджет), или тёплый по поведению, но холодный по признакам, потому что пайплайн ещё не сделал backfill (фикс: fast-path, синхронно вычисляющий дешёвые признаки и асинхронно backfill-ящий дорогие).
- Трактовать холодный старт как постоянный лейбл: как только взаимодействия накопились, тот же пользователь или айтем должны перейти на нормальную модель.
- Exposure-бюджеты без quality-гейта: гарантированная экспозиция новым айтемам — это и экспозиция плохих новых айтемов. Сочетать с классификатором качества контента (спам, low-effort, нарушения политик), чтобы избежать равномерной выдачи низкоценного контента.
- Гейминг буста новых айтемов: на маркетплейсах, рекламе, соцсетях и контент-платформах создатели или продавцы могут штамповать низкокачественные «новые» айтемы, чтобы захватить exposure исследования. Quality-гейты, per-creator rate limits и мониторинг злоупотреблений нужны наряду с самим бюджетом.
- Position bias, искажающий измерение холодного старта: новые айтемы, размещённые в нижних слотах, показывают худший CTR, чем тёплые айтемы в верхних слотах, даже когда базовое качество одинаково. Либо сравнивать в фиксированных слотах, либо использовать position-debiased оценщики.

2.2.7 Ограничения

- Content-only retrieval имеет потолок качества: айтемы со слабыми или дженериковыми контентными признаками получают плоский эмбединг, плохо дискриминирующий. Фильмы и книги часто несут семантически богатые описания; многие e-commerce SKU имеют шаблонные, дублированные или тонкие метаданные, поэтому контентные эмбединги схлопываются, если не обогащены таксономией, изображением, брендом и реляционными признаками.
- Митигация холодного старта обменивается с bias и петлями обратной связи: агрессивное исследование вредит краткосрочным метрикам, а отсутствие исследования имеет тенденцию схлопывать покрытие новых айтемов со временем.
- Подходы мета-обучения (MeLU и преемники) требуют эпизодического тренировочного распределения, имитирующего few-shot адаптацию. Без него офлайн-оценка переоценивает онлайн-производительность, и модель не лучше тёпло-обученной двухбашенной модели с контентными признаками.
- Холодно-сегментная популяция мала по определению. Большинство A/B-тестов холодного старта неэффективны, если не спроектированы с холодным разбиением как первичной единицей анализа с самого начала.

2.2.8 Ссылки

- Schein, Popescul, Ungar, Pennock — Methods and Metrics for Cold-Start Recommendations (SIGIR 2002)
- Li, Chu, Langford, Schapire — A Contextual-Bandit Approach to Personalized News Article Recommendation (WWW 2010)
- Volkovs, Yu, Poutanen — DropoutNet: Addressing Cold Start in Recommender Systems (NeurIPS 2017)
- Lee, Im, Jang, Cho, Chung — MeLU: Meta-Learned User Preference Estimator for Cold-Start Recommendation (KDD 2019)
- Zhang et al. — Cold-Start Recommendation towards the Era of Large Language Models: A Comprehensive Survey and Roadmap (2025)
- Cold start (recommender systems) — Wikipedia

Раздел 3. Калибровка вероятностей, матрица ошибок и F1-метрика

3.1 Калибровка вероятностей

Классификатор считается откалиброванным, если его предсказанные вероятности соответствуют наблюдаемым частотам: среди примеров, которым присвоен скор 0.7, около 70% должны принадлежать положительному классу. Формально $P(Y = 1 \mid \hat{p}(X) = p) = p$ для всех $p \in [0, 1]$.

Калибровка отличается от дискриминации. Модель с высоким AUC всё ещё может выдавать плохо откалиброванные вероятности, а идеально откалиброванная модель может плохо ранжировать примеры. Калибровка важна всегда, когда сырые скоры используются в нижестоящих моделях или в решениях, основанных на пороге.

3.1.1 Когда калибровка важна

- Вероятностные пороги, где смысл порога зависит от калибровки (правило обнаружения мошенничества, срабатывающее при $p > 0.8$, имеет смысл только если 0.8 действительно отражает 80%-ю долю положительных).
- Арифметика над несколькими скорями, например паттерн $pCTR \times pConv \times value$ в ранжировании рекламы. Плохо откалиброванные входы искажают произведение даже если у каждой модели хороший AUC.
- Комбинирование скоров между моделями: смешивание скоров двух и более моделей (взвешенное среднее или сумма) уважает заданные веса только если скоры находятся на одной вероятностной шкале. Смесь 50/50 откалиброванной и переуверенной модели будет доминирована переуверенной, хотя веса говорят об обратном.
- Решения на основе ожидаемой полезности: любые расчёты «cost-benefit», умножающие предсказанную вероятность на выигрыш или потерю.
- Распределение ресурсов: очереди ручной проверки, расследование фрода, кредитные лимиты, модерационные действия и бюджеты уведомлений часто завязаны на откалиброванный риск.
- Современные глубокие классификаторы, которые обычно переуверены по сравнению с поверхностными моделями или более ранними архитектурами (Guo et al. 2017).

Калибровка не является правильной целью для чисто ранжирующих систем. AUC, NDCG и mean average precision (MAP) зависят только от порядка скоров, поэтому любое монотонное преобразование выходов модели оставляет их без изменений. В пайплайнах рекомендательных систем стадия retrieval работает только по рангу, тогда как скоры стадии ranking участвуют в нижестоящей арифметике и нуждаются в калибровке.

3.1.2 Источники ошибок калибровки

Ошибка калибровки по семействам моделей

Разные семейства моделей искажают калибровку характерным образом:

- Логистическая регрессия: хорошо откалибрована, когда выполняется предположение о линейности log-odds и распространённость в обучении совпадает с продакшеном, поскольку log loss — строго proper scoring rule. Калибровка ломается при неверной спецификации модели, регуляризационной усадке, взвешивании классов или сдвиге распространённости.
- Градиентный бустинг, обученный с log loss (современные дефолты LightGBM, XGBoost): как правило хорошо откалиброван, так как loss — proper. Niculescu-Mizil & Caruana (2005) подтверждают это эмпирически.
- Случайный лес и бэггинг-ансамбли: недоуверены на экстремумах. Усреднение в листьях и усреднение по ансамблю одинаково оттягивают предсказания от 0 и 1.

- AdaBoost и другие margin-driven бустеры: характерная сигмоидальная ошибка калибровки. Platt scaling изначально создавался именно под эту форму.
- Глубокие нейросети: как правило переуверены, особенно при большой ёмкости, batch normalization и weight decay (Guo et al. 2017).
- Наивный Байес: переуверен, так как предположение об условной независимости doubly учитывает скоррелированные признаки.

Ошибка калибровки из-за выбора обучения и данных

Эти факторы действуют поверх тенденций семейства модели:

- Distribution shift между срезами: страна, устройство, сезон, источник трафика. Глобальный ECE (Expected Calibration Error) может скрывать тяжёлую ошибку калибровки на уровне срезов.
- Sampled negatives или class-balanced обучение: сырой выход отражает распространённость в обучении, а не в продакшене. См. negative sampling и раздел о prior correction ниже.
- Решения на этапе обучения: focal loss понижает вес лёгких примеров и может приводить к недоуверенным выходам; label smoothing ограничивает уверенность сверху на $1 - \epsilon$; mixup и cutmix обучают на интерполированных метках и требуют рекалибровки; взвешивание классов меняет эффективную распространённость в обучении.
- Отложенные и цензурированные метки: конверсии, отток, возвраты и чарджбеки приходят с задержкой. Калибровка, замеренная до созревания меток, смещена; цель калибровки должна указывать окно метки (1-day click, 7-day conversion, 30-day churn).
- Selection и exposure bias: в рекомендательных системах метки существуют только для показанных объектов. Калибровка определяется для экспонированного распределения и обусловлена политикой логирования/выдачи. Смена политики может сдвинуть распределение экспозиции и сломать калибровку даже если распределения признаков выглядят стабильно.
- Training-serving skew: преобразования признаков, дефолты для пропусков, нормализация или различающаяся свежесть между обучением и выдачей создают перекося, который калибровка может замаскировать, но не починить.
- Ограничения приватности и агрегации: когда связь «показ — метка» рвётся политикой приватности, поимпрессионная рекалибровка становится невозможной. Запасной вариант — aggregate ratio calibration по когортному окну: сравниваем суммы предсказаний и суммы реализованных событий по приватность-уважающим бакетам и применяем отношение как мультипликативную поправку.

3.1.3 Измерение калибровки

Диаграмма надёжности

Диаграмма надёжности бьёт предсказанные вероятности на M бакетов (обычно 10 или 20) и затем рисует середину бакета против эмпирической доли положительных внутри него. Идеальная калибровка лежит на диагонали $y = x$. Бакеты выше диагонали указывают на

недоуверенность; ниже — на переуверенность. Поверх графика надёжности следует накладывать счётчики бакетов или доверительные интервалы: разреженные высокоскорые бакеты шумные и их не стоит переинтерпретировать.

Для сильно несбалансированных задач (фрод, конверсия) диаграмма надёжности в линейных осях скрывает ошибки в области малых p , где как раз и принимаются ключевые решения. Помогают log-axis-варианты и наложение счётчиков бакетов. Продакшен-мониторинг должен отслеживать калибровку в области принятия решений (около порогов, top-k бакеты, аукцион-релевантные диапазоны) в дополнение к глобальной картине; низкий глобальный ECE может сосуществовать с сильно искажёнными бакетами 0.8–1.0.

Expected Calibration Error (ECE)

ECE сводит диаграмму надёжности к одному числу:

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)|$$

где M — число бакетов, B_m — m -й бакет, $|B_m|$ — его размер, n — общий объём выборки, $\text{acc}(B_m)$ — эмпирическая доля положительных внутри бакета, $\text{conf}(B_m)$ — среднее предсказанной вероятности внутри бакета.

ECE чувствителен к бинированию. Equal-width и equal-frequency схемы дают разные значения на одних и тех же предсказаниях. Слишком мало бакетов недооценивают ошибку калибровки, усредняя ошибки внутри широких бакетов; слишком много — раздувают дисперсию из-за малых выборок на бакет.

[Nixon et al. \(2019\)](#) предлагают адаптивное бинирование. [Roelofs et al. \(2022\)](#) описывают смещение оценки и рекомендуют equal-mass binning с поправкой на смещение. [Popordanoska et al. \(2022\)](#) полностью обходят бинирование, используя KDE-оценщик без бакетов.

Brier score

Brier score — среднеквадратичная ошибка между предсказанной вероятностью и исходом:

$$\text{Brier} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)^2$$

Это строго proper scoring rule: его математическое ожидание минимизируется только когда сообщённые вероятности равны истинным. Метрика раскладывается на компоненты reliability, resolution и uncertainty ([Murphy 1973](#)). Дискриминирующая, но плохо откалиброванная модель может иметь низкий Brier score, потому что высокая resolution компенсирует плохую reliability, — поэтому компоненты надо читать отдельно.

Log loss

Log loss на отложенной выборке также является строго proper scoring rule. Он более чувствителен, чем Brier, к уверенно-ошибочным предсказаниям, поскольку вклад одного

примера расходится при $p \rightarrow 0$ или $p \rightarrow 1$ на неправильной классификации. Полезен, когда приложение непропорционально штрафует уверенно-ошибочные предсказания.

3.1.4 Методы рекалибровки

Рекалибровка подгоняет одномерное отображение от сырых скоров к откалиброванным вероятностям на отложенной выборке. Три распространённых метода различаются в основном гибкостью и параметрической формой.

Platt scaling

Platt scaling подгоняет logistic regression на сырых скорых модели (Platt 1999):

$$p_{\text{calibrated}} = \frac{1}{1 + \exp(-(\alpha s + \beta))}$$

где s — сырой скор модели, α, β — скалярные параметры, подобранные на отложенной выборке минимизацией log loss. Устойчив на малых валидационных выборках, но предполагает сигмоидальную форму ошибки калибровки. Изначально предназначался для decision values SVM; также применяется к поверхностным моделям с гладкими распределениями скоров и к классификаторам типа AdaBoost, ошибка калибровки которых сигмоидальна по построению.

Две структурные причины делают logistic regression естественным калибратором. LR, обученная по log loss, имеет гарантированное свойство калибровки на обучающей выборке: в оптимуме условие нулевого градиента означает, что средняя предсказанная вероятность равна эмпирической доле положительных в разбиении по каждому категориальному признаку. Сигмоида также монотонна по сырому скору, поэтому откалиброванные вероятности сохраняют порядок входов, а ранговые метрики вроде AUC не меняются.

Isotonic regression

Isotonic regression подгоняет непараметрическое монотонное отображение, минимизируя квадратичную ошибку при ограничении монотонности (Zadrozny & Elkan 2002). Более гибок, чем Platt scaling, точнее на несигмоидальной ошибке калибровки и сильнее переобучается на малых валидационных выборках. Полученное отображение кусочно-постоянно: экстремальные скоры могут схлопнуться в одну и ту же откалиброванную вероятность, и отображение не экстраполируется за пределы диапазона обучающих скоров.

Temperature scaling

Temperature scaling масштабирует softmax-логиты нейросетевого классификатора на единственный обучаемый скаляр T (Guo et al. 2017):

$$\mathbf{p} = \text{softmax}(\mathbf{z}/T)$$

где $\mathbf{z}$ — вектор пред-softmax логитов, $\mathbf{p}$ — откалиброванный вектор вероятностей. $T < 1$ заостряет распределение; $T > 1$ сглаживает его. Современные глубокие классификаторы обычно переуверены, поэтому подгонка даёт $T > 1$. T подгоняется на отложенной выборке

минимизацией $\log \text{ loss}$. Однопараметрическая форма переобучается меньше, чем многопараметрические альтернативы на современных глубоких классификаторах.

Внутри одного примера temperature scaling сохраняет порядок логитов, поэтому предсказанный класс не меняется. Между примерами он не сохраняет порядок уверенности: top-1-уверенность зависит от полного вектора логитов (а не только от зазора до второго места), поэтому два примера могут поменяться местами в ранжировании по уверенности при изменении T . В бинарной классификации это не возникает, так как $p = \sigma(z/T)$ монотонна по единственному логиту.

Beta calibration

Beta calibration (Kull et al. 2017) обобщает Platt scaling на несигмоидальные формы. Стандартная подгонка — logistic regression на $[\log s, \log(1 - s)]$, что даёт 2–3 параметра в зависимости от выбранного ограничения. Метод занимает промежуточное положение между Platt (слишком жёсткий для некоторых форм диаграммы надёжности) и isotonic (склонен к переобучению на малых валидационных выборках).

3.1.5 Prior correction для выборочных обучающих данных

Многие продакшен-модели обучаются на данных с распространённостью, отличной от продакшена: рекомендательные системы с negative downsampling, фронт-модели с positive oversampling, ad-CTR пайплайны с агрессивным negative subsampling. Сырой выход модели откалиброван к обучающей выборке, а не к продакшен base rate. Рекалибровка Platt или isotonic на валидационной выборке с тем же искусственным сэмплированием переносит смещение дальше; рекалибровка на несмещённой, продакшен-подобной валидационной выборке может выучить поправку на base rate напрямую.

Общая поправка сдвигает логит на разность между деплой- и training- log-odds:

$$\text{logit}(p_{\text{cal}}) = \text{logit}(p_{\text{model}}) + \log \frac{\pi_{\text{prod}}}{1 - \pi_{\text{prod}}} - \log \frac{\pi_{\text{train}}}{1 - \pi_{\text{train}}}$$

где π_{train} — распространённость положительного класса в обучающих данных, π_{prod} — распространённость в продакшене, p_{model} — сырой выход модели, p_{cal} — скорректированная вероятность.

В частном случае negative downsampling со ставкой w (доля сохранённых отрицательных примеров в обучении) поправка сводится к константному сдвигу логита:

$$\text{logit}(p_{\text{cal}}) = \text{logit}(p_{\text{model}}) + \log w$$

или эквивалентно в вероятностной форме:

$$p_{\text{cal}} = \frac{p_{\text{model}}}{p_{\text{model}} + (1 - p_{\text{model}})/w}$$

Эта поправка точна только при prior shift, когда сэмплирование зависит лишь от метки класса. Если вероятность сэмплирования зависит от признаков, сегментов объектов или пользователей,

источника кандидатов или времени, сдвиг смещён; вместо него нужно importance weighting на обучении или калибровка на несмещённой продакшен-подобной валидационной выборке.

Prior correction применяется до или одновременно с Platt/isotonic/temperature рекалибровкой. Если её пропустить, рекалибратор неявно поглотит сдвиг распространённости. Это работает на валидационных данных из обучающего распределения, но ломается, как только продакшен-распространённость дрейфует.

3.1.6 Многоклассовая калибровка

Подходы к многоклассовой калибровке отличаются от бинарной:

- Top-label calibration: среди предсказаний, где у топ-класса уверенность p , топ-класс верен с частотой p . Наиболее часто отчитываемая метрика.
- Class-wise calibration: вероятность каждого класса откалибрована отдельно, как one-vs-rest бинарная задача. Сильнее.
- Joint calibration: полный вектор вероятностей соответствует условному распределению Y при $\hat{p}(X)$. Сильнейшая, труднее всего оценить.

Методы обобщаются соответствующим образом. Temperature scaling расширяется единственным T , общим для всех классов. Vector scaling допускает индивидуальные температуру и смещение на класс. Matrix scaling применяет полное линейное преобразование к вектору логитов, гибче, но сильнее склонен к переобучению. Dirichlet calibration (Kull et al. 2019) параметризует семейство Dirichlet на симплексе вероятностей. Per-class one-vs-rest с Platt или isotonic — распространённое прагматичное приближение: каждый класс получает свой калибратор, затем выходы перенормируются в сумму единицы. Перенормировка может изменить top-1-предсказания, потому что per-class калибраторы монотонны по отдельности, но не совместно.

3.1.7 Per-segment рекалибровка

Калибровка дрейфует по срезам (страна, устройство, когорта, время суток) и внутри каждого среза со временем. Глобальная рекалибровка усредняет сегментные ошибки, что может оставить отдельные сегменты сильно искажёнными даже при малом глобальном ECE. Per-slice рекалибровка обменивает большую дисперсию (каждому срезу нужно достаточно валидационных данных) на меньшее смещение.

Когда per-slice счёт положительных мал, иерархические подходы или partial pooling, усаживающие оценки срезов к глобальному рекалибратору, удерживают дисперсию в управляемом диапазоне. Стратифицированный отбор калибровочной выборки с минимальным числом положительных на срез — более простая операционная версия.

Продакшен-мониторинг отслеживает ECE и диаграммы надёжности по срезам в дополнение к глобальному значению. Внезапный дрейф в одном срезе часто сигнализирует о проблеме в апстрим-пайплайне признаков или о популяционном сдвиге, который глобальная метрика усредняет. См. A/B-тесты — калибровка как guardrail-метрика во время раскаток.

Per-segment калибраторы могут менять межсегментный порядок, даже если каждое сегментное отображение монотонно. Это важно, когда нижестоящие системы сравнивают скоры между сегментами: глобальный порог, единая очередь проверки или аукцион, ранжирующий поперёк

типов пользователей. В таких случаях per-segment калибраторы должны оставаться согласованными на границах решений, либо стадия ранжирования должна работать на скорях до калибровки.

3.1.8 Multi-calibration

Per-segment рекалибровка эмпирична: подобрать калибратор на срез и пульти, когда срезы разрежены. Multi-calibration (Hébert-Johnson et al. 2018) формализует цель. Модель multi-calibrated, если она откалибрована для каждой подгруппы, идентифицируемой выбранным классом вычислений, включая пересекающиеся intersectional-подгруппы. Фреймворк даёт теоретические гарантии, которых простые per-slice подгонки не предоставляют.

MCGrad — недавняя масштабируемая реализация: рекурсивная процедура градиентного бустинга, которая авто-находит плохо откалиброванные подгруппы через листья деревьев, без ручной спецификации сегментов. Часто улучшает дискриминационные метрики (log loss, PRAUC) одновременно с калибровкой, а не за счёт неё.

3.1.9 Практические соображения

- Независимость калибровочной выборки: калибровочная выборка отложена от обучения и от любой селекции моделей, которая её трогала. Используйте честный трёхсторонний сплит (train, calibrate, test); переиспользование валидационной выборки одновременно для селекции и калибровки даёт оптимистичные оценки калибровки. Для time-dependent систем — сплит по времени.
- Соответствие распределений: калибровочная выборка должна соответствовать продакшену. Те же фильтры популяции, тот же путь признаков, то же окно метки, та же поправка на negative sampling (или её отсутствие).
- Каденс обновления: повторно подгоняйте на свежем окне (ежедневно, еженедельно или на релиз), когда продакшен-распределение дрейфует.
- Онлайн-ECE: streaming-бинирование с оконными или экспоненциально затухающими счётчиками ловит дрейф по ходу; оффлайн-ECE на исторических батчах ловит его постфактум.
- Continuous training: окно калибровки сдвигается вместе с обучающим окном. Отслеживание T (или Platt slope) во времени рано вскрывает дрейф. Резкий скачок подобранных параметров обычно сигнализирует о проблемах с данными или пайплайном до того, как нижестоящие метрики сдвинутся.
- Multi-task или композитные скоры: ранжировщик, комбинирующий выходы голов (например, $pCTR \times pConv \times dwell$ для ad eCPM или сумма голов по объективам), может иметь откалиброванный композит, тогда как каждая компонента не откалибрована, потому что ошибки компонент компенсируют друг друга. Per-component рекалибровка перед комбинированием ловит смещение у источника; калибровка только композита его прячет.
- Closed-loop онлайн-рекалибровка: переподгоняйте калибратор на стриминговом логе пар (prediction, realized outcome) и редеплойте на короткой каденции (например, каждые ~30 минут), с автоматическим откатом на последний known-good снапшот, если ошибка калибровки превысит порог.

- Ансамбли откалиброванных моделей: усреднение двух уже откалиброванных моделей даёт выход, который более не откалиброван. Ансамблю нужна собственная стадия рекалибровки на отложенной выборке.

3.1.10 Ограничения

- Рекалибровка подгоняется на отложенной выборке; если продакшен-распределение дрейфует, дрейфует и калибровочное отображение, поэтому его нужно периодически переподгонять на свежем окне.
- Калибровка не может починить плохо дискриминирующую модель. Константный классификатор, предсказывающий base rate, идеально откалиброван и бесполезен; рекалибровка работает только с уже выданным моделью порядком.
- Чисто ранжирующие системы не затрагиваются монотонной рекалибровкой.
- Isotonic regression может срезать экстремальные скоры в плоский плато; подобранное отображение не экстраполируется на скоры вне диапазона калибровки.
- Калибровка даёт гарантии маргинальных частот, а не покрытия для отдельного предсказания. Когда важны интервалы неопределённости на индивидуальных предсказаниях, нужно использовать conformal prediction.
- Калибровка сообщает, как часто предсказания при уверенности p верны. Решение о том, отказаться ли от предсказания на низкоуверенных входах (selective prediction) — отдельная задача: она комбинирует калибровку с ценой ошибок и ценностью верных предсказаний.
- LLM-вероятности токенов после RLHF (reinforcement learning from human feedback) или DPO (direct preference optimization) систематически искажены: переуверены на распределении политики. Любой системе, использующей LLM logprobs как скоры уверенности, нужна явная рекалибровка; диаграммы надёжности на распределениях уровня токена обычно показывают устойчивую переуверенность.

3.1.11 Пример кода

```
[!example]- sklearn CalibratedClassifierCV ```python import matplotlib.pyplot as plt from
sklearn.calibration import CalibratedClassifierCV, calibration_curve from sklearn.metrics import
brier_score_loss
```

sklearn >= 1.2 uses `estimator=`; older versions used `base_estimator=`.

```
calibrated = CalibratedClassifierCV(estimator=base_estimator, method="isotonic", cv=5)
calibrated.fit(X_train, y_train) probs = calibrated.predict_proba(X_test)[:, 1]
```

```
brier = brier_score_loss(y_test, probs) prob_true, prob_pred = calibration_curve(y_test, probs,
n_bins=10)
```

```
plt.plot([0, 1], [0, 1], linestyle="--", color="gray") plt.plot(prob_pred, prob_true, marker="o")
plt.xlabel("Mean predicted probability") plt.ylabel("Empirical positive rate") plt.title(f"Reliability
```

```
diagram (Brier = {brier:.3f}) plt.show() `` method="sigmoid" переключает на Platt scaling. cv="prefit" калибрует уже обученную модель на отдельной валидационной выборке; в новых sklearn эта опция выводится из обращения в пользу обёртки базового эстиматора во FrozenEstimator`.
```

3.1.12 Связанные темы

- Рекомендательные системы: аукционы и комбинирование нескольких скоров — частый продакшен-случай для калибровки.
- Negative sampling: вызывает сдвиг base rate, который и устраняет prior correction.
- Training-serving skew: частый драйвер дрейфа калибровки в продакшене.
- A/B-тесты: дрейф калибровки как guardrail-метрика во время раскаток и запусков.
- SVM: исходная цель для Platt scaling.
- Platt — *Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods* (1999)
- Zadrozny & Elkan — *Transforming Classifier Scores into Accurate Multiclass Probability Estimates* (KDD 2002)
- Niculescu-Mizil & Caruana — *Predicting Good Probabilities With Supervised Learning* (ICML 2005)
- Guo, Pleiss, Sun, Weinberger — *On Calibration of Modern Neural Networks* (ICML 2017)
- Kull, Filho, Flach — *Beta Calibration* (AISTATS 2017)
- Hébert-Johnson, Kim, Reingold, Rothblum — *Multicalibration: Calibration for the (Computationally-Identifiable) Masses* (ICML 2018)
- Kull et al. — *Beyond Temperature Scaling: Dirichlet Calibration* (NeurIPS 2019)
- Tax et al. — *MCGrad: Multicalibration at Web Scale* (KDD 2026)
- Nixon et al. — *Measuring Calibration in Deep Learning* (CVPR Workshops 2019)
- Popordanoska, Sayer, Blaschko — *A Consistent and Differentiable Lp Canonical Calibration Error Estimator* (NeurIPS 2022)
- Roelofs et al. — *Mitigating Bias in Calibration Error Estimation* (AISTATS 2022)
- scikit-learn — Probability calibration user guide

3.2 Матрица ошибок (confusion matrix)

Матрица ошибок (Confusion Matrix) — таблица, используемая для оценки качества классификационной модели на наборе данных, для которого известны истинные значения. Она даёт детальную разбивку того, насколько хорошо модель предсказывает классы.

3.2.1 Базовая структура

Для задачи бинарной классификации матрица ошибок — это таблица 2×2:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)

	Predicted Positive	Predicted Negative
Actual Negative	False Positive (FP)	True Negative (TN)

Где: - **True Positive (TP)**: модель корректно предсказывает положительный класс. - **True Negative (TN)**: модель корректно предсказывает отрицательный класс. - **False Positive (FP)**: модель некорректно предсказывает положительный класс (ошибка I рода). Пример: модель предсказывает, что здоровый пациент болен. - **False Negative (FN)**: модель некорректно предсказывает отрицательный класс (ошибка II рода). Пример: модель предсказывает, что больной пациент здоров.

Для многоклассовых задач матрица расширяется до размерности $n \times n$, где n — число классов.

3.2.2 Производные метрики

Accuracy

Общая корректность модели — доля корректных предсказаний от их общего числа. Подходит, когда распределение классов сбалансировано и цена неверной классификации разных классов сопоставима.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Error Rate

Общая ошибка модели.

$$\text{Error Rate} = \frac{FP + FN}{TP + TN + FP + FN} = 1 - \text{Accuracy}$$

Precision

Precision (точность) или Positive Predictive Value.

Доля действительно положительных примеров среди всех, предсказанных как положительные (сколько извлечённых элементов релевантны). Подходит, когда цена false positive высока, например при детекции спама.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

Recall (полнота) или Sensitivity, True Positive Rate.

Доля корректно классифицированных примеров среди всех положительных (сколько из релевантных элементов были извлечены). Подходит, когда цена false negative высока, например при медицинской диагностике или детекции фрода.

$$\text{Recall} = \frac{TP}{TP + FN}$$

Specificity

Specificity (специфичность) или True Negative Rate.

Доля корректно классифицированных примеров среди всех отрицательных. Подходит, когда корректное определение отрицательных примеров критично, например в медицине.

$$\text{Specificity} = \frac{TN}{TN + FP}$$

False Positive Rate

Доля некорректно классифицированных примеров среди всех отрицательных. Подходит, когда важны «ложные тревоги».

$$\text{FPR} = \frac{FP}{FP + TN} = 1 - \text{Specificity}$$

False Negative Rate

Доля некорректно классифицированных примеров среди всех положительных.

$$\text{FNR} = \frac{FN}{TP + FN} = 1 - \text{Recall}$$

F1-метрика

Гармоническое среднее precision и recall. Подходит, когда нужен баланс между precision и recall, особенно на несбалансированных датасетах, где одно может быть высоким за счёт другого.

$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

F-beta score

Обобщённая версия F1-метрики, вводящая параметр β для контроля относительной важности precision и recall. При $\beta = 2$ recall взвешивается вдвое сильнее, чем precision.

$$F_{\beta} = (1 + \beta^2) \cdot \frac{\text{precision} \cdot \text{recall}}{(\beta^2 \cdot \text{precision}) + \text{recall}}$$

Matthews Correlation Coefficient

Коэффициент корреляции между наблюдаемыми и предсказанными бинарными классификациями. Коэффициент +1 означает идеальное предсказание, 0 — не лучше случайного, -1 — полное несоответствие между предсказанием и наблюдением.

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

Cohen's Kappa

Измеряет согласованность между предсказанными и фактическими классификациями, корректируя на случайное согласие.

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

где p_o — наблюдаемое согласие, p_e — ожидаемое случайное согласие.

3.2.3 Агрегация метрик в многоклассовой задаче

Для многоклассовых матриц ошибок метрики можно агрегировать несколькими способами:

1. **Macro-усреднение:** считаем метрики для каждого класса независимо и берём невзвешенное среднее.
2. **Micro-усреднение:** агрегируем вклады всех классов для вычисления единой метрики.
3. **Взвешенное среднее:** считаем метрики для каждого класса и берём их среднее, взвешенное по support (числу примеров).

3.2.4 ROC и AUC

Кривая Receiver Operating Characteristic (ROC) откладывает True Positive Rate (recall) против False Positive Rate при различных значениях порога. Area Under the Curve (AUC) даёт единую меру качества классификатора по всем возможным порогам классификации.

- **AUC = 0.5:** качество не лучше случайного.
- **AUC = 1.0:** идеальный классификатор.
- **AUC > 0.8:** обычно считается хорошим качеством.

3.2.5 Precision-Recall Curve

Для несбалансированных датасетов кривая precision-recall часто даёт более информативное представление, чем кривая ROC. Площадь под этой кривой, известная как Average Precision (AP), суммирует компромисс precision-recall.

3.2.6 Ссылки

- [Scikit-learn documentation on confusion matrices](#)
- [Google ML Crash Course on Classification](#)
- [Confusion matrix page on Wikipedia](#)
- [Precision & Recall at mlu-explain](#)

3.3 F1-метрика

F1-метрика — метрика оценки классификационных моделей, объединяющая precision и recall в единое значение и дающая сбалансированную оценку качества модели. Особенно полезна на несбалансированных датасетах, где одна ассурасу может вводить в заблуждение.

F1-метрика вычисляется как гармоническое среднее precision и recall:

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

F1-метрика принимает значения от 0 до 1, где 1 соответствует идеальным precision и recall. F1 использует гармоническое среднее вместо арифметического, чтобы штрафовать экстремальные значения. Precision 1 и recall 0 дают арифметическое среднее 0.5, но F1 равен 0.

F1 обычно используется для несбалансированных датасетов, когда важны и precision, и recall. Но метрика игнорирует TN, поэтому её нельзя применять, когда TN важны. И требуется настройка порога классификации.

3.3.1 Вариации

F-beta score — обобщённая версия F1-метрики, вводящая параметр β для контроля относительной важности precision и recall:

$$F_{\beta} = (1 + \beta^2) \cdot \frac{\text{precision} \cdot \text{recall}}{(\beta^2 \cdot \text{precision}) + \text{recall}}$$

При $\beta = 2$ recall взвешивается вдвое сильнее, чем precision.

В многоклассовых сценариях **macro F1** усредняет F1-метрики, посчитанные независимо для каждого класса:

$$\text{Macro F1} = \frac{1}{n} \sum_{i=1}^n F1_i$$

Micro F1 агрегирует вклады всех классов для вычисления единой F1-метрики:

$$\text{Micro F1} = \frac{2 \cdot \sum_{i=1}^n \text{TP}_i}{2 \cdot \sum_{i=1}^n \text{TP}_i + \sum_{i=1}^n \text{FP}_i + \sum_{i=1}^n \text{FN}_i}$$

3.3.2 Ссылки

- [Scikit-learn documentation on F1 score](#)
- [Wikipedia: F-score](#)

Раздел 4. Линейные модели, градиентный бустинг и кластеризация

4.1 Линейная регрессия

Линейная регрессия — это алгоритм обучения с учителем (или статистический метод), который учится моделировать зависимую переменную (целевую) как функцию от нескольких независимых переменных (признаков), подбирая линию (или поверхность), наилучшим образом «подходящую» под данные. Для линейной регрессии мы предполагаем, что целевая переменная непрерывна (число); для логистической регрессии — что целевая переменная дискретна (имеет конечное число классов).

4.1.1 Простая линейная регрессия

$$y = \beta_0 + \beta_1 x + \varepsilon$$

4.1.2 Множественная линейная регрессия

$$y = X\beta + \varepsilon$$

Где: - y — вектор $n \times 1$ наблюдений зависимой переменной - X — матрица $n \times (p + 1)$ независимых переменных (включая столбец единиц для свободного члена) - β — вектор $(p + 1) \times 1$ коэффициентов - ε — вектор $n \times 1$ ошибок

4.1.3 Подбор коэффициентов

OLS (метод наименьших квадратов) / Аналитическое решение

OLS — наиболее распространённый метод оценки параметров модели линейной регрессии. Цель: минимизировать сумму квадратов остатков: $\min \sum (y_i - (\beta_0 + \beta_1 x_{1i} + \dots + \beta_p x_{pi}))^2$
$$\hat{\beta} = (X'X)^{-1} X'y$$

Где: - $\hat{\beta}$ — вектор оценённых коэффициентов - X' — транспонированная X - $(X'X)^{-1}$ — обратная к $X'X$

Свойства OLS-оценок при классических предположениях: 1. Best Linear Unbiased Estimator (BLUE) — наилучшая линейная несмещённая оценка 2. Состоятельность 3. Асимптотическая нормальность

Оценка σ^2

$$\hat{\sigma}^2 = \frac{ESS}{n}$$
 Валидность коэффициентов В статистике коэффициенты обычно сопровождаются их р-значениями. Эти р-значения получают из статистических тестов нулевой гипотезы: t-тесты используются для проверки того, значимо ли отличается данный коэффициент от нуля (нулевая гипотеза о том, что конкретный коэффициент β_i равен нулю), а F-тесты — для проверки того, отличаются ли *какие-либо* члены регрессионной модели значимо от нуля.

Maximum Likelihood Estimation (MLE)

Maximum Likelihood Estimation (MLE) — альтернативный OLS метод оценки параметров модели линейной регрессии. Особенно полезен при работе с ненормальными распределениями ошибок. MLE находит значения параметров, максимизирующие правдоподобие наблюдения имеющихся данных.

Функция правдоподобия для линейной регрессии В предположении нормально распределённых ошибок: $L(\beta, \sigma^2 | y, X) = \prod_i \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) * \exp\left(\frac{-(y_i - x_i'\beta)^2}{(2\sigma^2)}\right)$

Логарифм правдоподобия: $\ln L(\beta, \sigma^2 | y, X) = -n/2 * \ln(2\pi) - n/2 * \ln(\sigma^2) - 1/(2\sigma^2) * \sum_i (y_i - x_i'\beta)^2$

$$\ln L(\beta, \sigma^2 | y, X) = -\frac{n}{2} * \ln(2\pi) - \frac{n}{2} * \ln(\sigma^2) - \frac{1}{(2\sigma^2)} * \sum_i (y_i - x_i'\beta)^2$$

Сравнение с OLS - В предположении нормальности MLE и OLS дают одинаковые оценки для β - MLE может быть расширен на ненормальные распределения ошибок - MLE предоставляет каркас для проверки гипотез и выбора модели (например, тесты отношения правдоподобий)

Градиентный спуск для линейной регрессии

При работе с большими наборами данных аналитическое решение может быть вычислительно дорогим. Градиентный спуск — итеративный алгоритм оптимизации, используемый для поиска минимума функции стоимости.

4.1.4 Предположения

- Линейность: связь между X и Y линейна.
- Гомоскедастичность: постоянная дисперсия остатков ($Var(\varepsilon|X) = \sigma^2$). Это означает, что распределение ошибки одинаково для всех значений признаков, не должно быть различных закономерностей.
- Отсутствие мультиколлинеарности: независимые переменные не должны быть сильно коррелированы между собой. Это можно проверить через корреляционные матрицы или Variance Inflation Factor (VIF).
- Нормальность: остатки распределены нормально ($\varepsilon \sim N(0, \sigma^2)$). Это можно проверить через Q-Q-графики остатков, гистограммы или статистические тесты вроде теста Колмогорова — Смирнова. Релевантно только для MLE.
- Независимость: наблюдения независимы друг от друга.
- Отсутствие эндогенности: $E(\varepsilon|X) = 0$, то есть ошибки не коррелируют с предикторами.

Последствия нарушения предположений

- Нарушение линейности: смещённые и несостоятельные оценки
- Нарушение независимости: некорректные стандартные ошибки, неэффективные оценки

- Нарушение гомоскедастичности: неэффективные оценки, некорректные стандартные ошибки
- Нарушение нормальности: проверки гипотез могут быть некорректны на малых выборках
- Идеальная мультиколлинеарность: невозможно оценить уникальные коэффициенты

4.1.5 Метрики оценки модели

1. R-squared (коэффициент детерминации): $R^2 = 1 - \frac{SSR}{SST}$, где SSR — сумма квадратов остатков, а SST — общая сумма квадратов
2. Adjusted R-squared: $AdjR^2 = 1 - [(1 - R^2)(n - 1)/(n - p - 1)]$
3. MSE: $MSE = \sum \frac{(y_i - \hat{y}_i)^2}{n}$
4. RMSE: $RMSE = \sqrt{MSE}$
5. MAE: $MAE = \sum \frac{|y_i - \hat{y}_i|}{n}$

4.1.6 Проверка гипотез

1. t-тест для отдельных коэффициентов: $H_0 : \beta_i = 0 \quad t = \frac{\hat{\beta}_i}{SE(\hat{\beta}_i)}$
2. F-тест общей значимости модели: $F = (SSR/p)/(SSE/(n - p - 1)) \quad F = \frac{\frac{SSR}{p}}{\frac{SSE}{(n-p-1)}}$

4.1.7 Доверительные интервалы

Доверительный интервал для β_i : $\hat{\beta}_i \pm t(\alpha/2, n - p - 1) * SE(\hat{\beta}_i)$

4.1.8 Расширения и регуляризация

1. Ridge-регрессия (L2): $\hat{\beta}_{ridge} = \operatorname{argmin}(\|y - X\beta\|^2 + \lambda\|\beta\|^2)$
2. Lasso-регрессия (L1): $\hat{\beta}_{lasso} = \operatorname{argmin}(\|y - X\beta\|^2 + \lambda\|\beta\|_1)$
3. Elastic Net: комбинация L1- и L2-штрафов

4.1.9 Моделирование сложных зависимостей

Полиномиальная регрессия

Расширяет линейную регрессию для моделирования нелинейных зависимостей: $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_n x^n + \varepsilon$

Члены взаимодействия

Позволяют моделировать совместный эффект двух или более переменных: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 (x_1 * x_2) + \varepsilon$

4.1.10 Пример кода

```
import numpy as np

class LinearRegression:
    def __init__(self, learning_rate: float=0.01, n_iterations: int=1000):
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.weights = None
        self.bias = None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0

        for _ in range(self.n_iterations):
            prediction = np.dot(X, self.weights) + self.bias

            dw = (1 / n_samples) * np.dot(X.T, (prediction - y))
            db = (1 / n_samples) * np.sum(prediction - y)

            self.weights -= self.learning_rate * dw
            self.bias -= self.learning_rate * db

    def predict(self, X):
        return np.dot(X, self.weights) + self.bias

    def mse(self, X, y):
        y_predicted = self.predict(X)
        return np.mean((y - y_predicted) ** 2)
```

4.1.11 Ссылки

- [Explained.ai: Linear Regression](#)
- [MLcourse.ai lesson](#)

4.2 Логистическая регрессия

Логистическая регрессия — статистический метод, применяемый к задачам бинарной классификации; моделирует вероятность принадлежности объекта определённому классу. Распространённые разновидности: бинарная (два возможных исхода), мультиномиальная (более двух неупорядоченных исходов), порядковая (более двух упорядоченных исходов). Логистическая регрессия похожа на линейную регрессию, но использует поверх неё логистическую функцию (сигмоиду).

$$p(y = 1|x) = \frac{1}{1+e^{-z}} \text{ где } z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

$$\text{Логит-функция: } \log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Логит — это логарифм шансов. Логиты преобразуют вероятности (от 0 до 1) в действительные числа. Сигмоида — функция, обратная логиту. Она отображает любое действительное число в вероятность в диапазоне от 0 до 1.

Шансы (odds) представляют собой отношение вероятности наступления события к вероятности его ненаступления. Если $p = 0.75$, шансы $= \frac{0.75}{1-0.75} = 3$ — то есть событие в 3 раза вероятнее наступит, чем не наступит.

4.2.1 Предположения

1. Независимость наблюдений
2. Отсутствие или незначительная мультиколлинеарность независимых переменных
3. Линейность по логиту для непрерывных переменных

4.2.2 Maximum Likelihood Estimation (MLE)

Параметры логистической регрессии обычно оцениваются методом MLE.

Функция правдоподобия

$$L(\beta) = \prod_i p(x_i)^{y_i} * (1 - p(x_i))^{1-y_i}$$

Логарифм правдоподобия

$$ll(\beta) = \sum_i [y_i \log(p(x_i)) + (1 - y_i) \log(1 - p(x_i))]$$

4.2.3 Градиентный спуск для логистической регрессии

На практике для поиска оптимальных параметров часто используют градиентный спуск.

Правило обновления: $\beta = \beta - \alpha * \nabla J(\beta)$ Где α — скорость обучения, а $\nabla J(\beta)$ — градиент функции стоимости.

4.2.4 Интерпретация коэффициентов

- β_i : прирост логарифма шансов при увеличении x_i на единицу при фиксированных остальных переменных
- $\exp(\beta_i)$: отношение шансов (odds ratio) при увеличении x_i на единицу. Если $\exp(\beta) = 1.2$, то увеличение x на единицу умножает шансы на 1.2 (рост на 20%).

4.2.5 Достоинства

- Простая и интерпретируемая
- Даёт вероятностные оценки на выходе
- Менее склонна к переобучению по сравнению с более сложными моделями

4.2.6 Ограничения

- Предполагает линейность по логарифму шансов
- Не может улавливать сложные нелинейные зависимости без feature engineering

4.2.7 Пример кода

```
import numpy as np

class LogisticRegression:
    def __init__(self, learning_rate: float=0.01, n_iterations: int=1000, threshold: flo
        self.learning_rate = learning_rate
        self.n_iterations = n_iterations
        self.weights = None
        self.bias = None
        self.threshold = threshold

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0

        for _ in range(self.n_iterations):
            prediction = np.dot(X, self.weights) + self.bias
            y_pred = self.sigmoid(prediction)

            dw = (1 / n_samples) * np.dot(X.T, (y_pred - y))
            db = (1 / n_samples) * np.sum(y_pred - y)

            self.weights -= self.learning_rate * dw
            self.bias -= self.learning_rate * db

    def predict(self, X):
        prediction = np.dot(X, self.weights) + self.bias
        y_predicted = self.sigmoid(prediction)
        return [1 if i > self.threshold else 0 for i in y_predicted]
```

4.2.8 Ссылки

- [Explained.ai: Logistic Regression](#)

4.3 Градиентный бустинг

Градиентный бустинг — техника ансамблирования (boosting), которая последовательно объединяет слабые модели (обычно неглубокие решающие деревья) в сильную

предсказательную модель. Каждое новое дерево обучается методом градиентного спуска, чтобы исправлять ошибки, допущенные ранее обученными деревьями.

4.3.1 Как это работает

1. Инициализировать модель константой
2. Для каждой итерации при заданной функции потерь $L(y, F(x))$ и текущей модели $F_m(x)$:
 - Вычислить отрицательный градиент функции потерь $g_i = -[\partial L(y_i, F(x_i))/\partial F(x_i)]$
 - Построить регрессионное дерево $h_m(x)$ по g_i
 - Выполнить линейный поиск оптимального шага $\rho_t = \arg \min_{\rho} \sum_{i=1}^n L(y_i, \hat{f}(x_i) + \rho \cdot h(x_i, \theta))$ - Опционально вместо предыдущего шага можно вычислить оптимальное значение для каждого листа $j = 1..J$: $\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, F_{m-1}(x_i) + \gamma)$
 - Обновить модель со скоростью обучения η ($0 < \eta \leq 1$) $F_m(x) = \hat{f}(x) + \eta \cdot \rho_m h_m(x)$
3. Повторять до сходимости или достижения максимального числа итераций
4. Итоговая модель — сумма всех деревьев: $F_M(x) = F_0(x) + \sum_{m=1}^M \eta \cdot \rho_m h_m(x)$

4.3.2 Достоинства

1. Может оптимизировать различные функции потерь (например, ранжирование, регрессия Пуассона)
2. Хорошо работает со смешанными типами данных

4.3.3 Недостатки

1. Более склонен к переобучению, особенно на зашумлённых данных
2. Требуется аккуратной настройки гиперпараметров; обучение может быть долгим

Функции потерь

- Регрессия: L2 (MSE), L1 (MAE), Huber loss
- Бинарная классификация: log-loss, экспоненциальная потеря
- Многоклассовая классификация: multi-class log-loss
- Ранжирование: LambdaMART, LambdaRank

Регуляризация

1. Shrinkage (скорость обучения): масштабирует вклад каждого дерева
2. Subsampling: использовать только долю обучающих данных для каждого дерева
3. Early stopping: остановить обучение, когда ошибка на валидации перестаёт уменьшаться
4. L1/L2-регуляризация на весах листьев

Важность признаков

- Split-based importance: измеряет, как часто признак используется для разбиения данных по всем деревьям; взвешено по улучшению модели в результате каждого разбиения.
- Gain-based importance: измеряет средний gain разбиений, использующих данный признак.

Обработка данных

- Выучивает наилучшее направление (левый или правый потомок) для пропущенных значений в каждом разбиении
- Категориальные признаки: one-hot encoding неэффективно; label encoding или target encoding лучше. У CatBoost есть встроенная специализированная обработка

4.3.4 Классификация vs регрессия

- Регрессия: обычно использует MSE или MAE в качестве функции потерь
- Классификация: часто использует log-loss (бинарная) или multi-class log-loss

4.3.5 Популярные реализации

XGBoost

- Sparse-aware split finding
- Weighted quantile sketch для приближённого построения деревьев: не перебираются все возможные разбиения, а значения признаков биннингуются, и среди этих бинов ищутся наилучшие точки разбиения
- Стратегия роста дерева по уровням (BFS): дерево строится по одному уровню за раз, по всем листьям текущей глубины, прежде чем перейти к следующему уровню
- Позже был добавлен и leaf-wise рост дерева (DFS)

LightGBM

- Gradient-based One-Side Sampling (GOSS): сохраняет все объекты с большими градиентами (более важными для обучения) и случайным образом сэмплирует подмножество объектов с малыми градиентами
- Exclusive Feature Bundling (EFB): объединяет взаимно исключающие признаки (которые редко одновременно принимают ненулевые значения) в один признак.
- Leaf-wise рост дерева (DFS): дерево растёт, выбирая лист с максимальным delta loss для разбиения.
- Histogram-based алгоритм: вместо точных значений непрерывных признаков использует биннинг по дискретным интервалам.

CatBoost

- Симметричные деревья: обычно деревья строятся последовательно, что может приводить к сдвигу предсказаний по мере развития модели. Симметричные деревья строятся так, что они нечувствительны к порядку объектов выборки — каждое разбиение происходит по одному и тому же атрибуту с одним и тем же порогом
- Ordered boosting: вместо использования текущей модели для вычисления градиентов всех наблюдений используется отдельная модель для каждого примера — эти модели обучаются на подмножестве данных, идущем перед текущим примером в случайной перестановке.
- Нативная обработка категориальных признаков — комбинация one-hot encoding и продвинутой техники mean encoding

Градиентный бустинг vs AdaBoost

- Градиентный бустинг подгоняет новые модели к остаткам предыдущих моделей, AdaBoost корректирует веса неправильно классифицированных объектов.
- Градиентный бустинг более гибок в выборе функции потерь; AdaBoost изначально создавался для бинарной классификации, но может быть адаптирован для других задач.
- AdaBoost использует экспоненциальную функцию потерь

Другие типы бустинга

- Newton Boosting использует производные функции потерь как первого, так и второго порядка
- Stochastic Gradient Boosting вносит случайность, используя случайное подмножество обучающих данных для построения каждого дерева
- Extreme Gradient Boosting (как XGBoost) использует оба подхода и более регуляризованную формализацию модели для контроля переобучения

4.3.6 DART (Dropouts meet Multiple Additive Regression Trees)

DART применяет концепцию dropout, изначально пришедшую из нейронных сетей, к градиентному бустингу решающих деревьев. На каждой итерации DART случайным образом отбрасывает долю уже построенных деревьев, и новое дерево строится так, чтобы исправлять ошибки этого «прореженного» ансамбля. Для предсказания используются все деревья. DART улучшает регуляризацию и снижает переобучение.

4.3.7 Ссылки

- [Explained.ai: Gradient Boosting](#)
- [MLcourse.ai lesson](#)

4.4 k-Nearest Neighbors (kNN)

k-Nearest Neighbors (kNN) — простой непараметрический алгоритм, применяемый как для задач классификации, так и для регрессии. Он делает предсказание для нового объекта на основе K ближайших к нему объектов (соседей). Не путать с k-means. kNN — алгоритм с учителем для классификации/регрессии, ищущий ближайших соседей; k-means — алгоритм без учителя для кластеризации, ищущий центроиды кластеров.

4.4.1 Как это работает

1. Выбрать число соседей K
2. Вычислить расстояние между запрашиваемым объектом и всеми обучающими примерами
3. Отсортировать расстояния и определить K ближайших соседей
4. Для классификации: - Агрегировать метки классов K соседей - Вернуть голосование большинства как предсказание $\hat{y} = \text{mode}(y_i), i \in K \text{ nearest neighbors}$
5. Для регрессии: - Агрегировать значения K соседей - Вернуть среднее как предсказание $\hat{y} = \frac{1}{K} \sum_{i \in K \text{ nearest neighbors}} y_i$

Поскольку kNN не строит модель на этапе обучения, он называется ленивым алгоритмом (lazy learner). Алгоритм запоминает весь обучающий набор и выполняет действие над ним в момент предсказания.

Метрики расстояния

1. Евклидово расстояние: $d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
2. Манхэттенское расстояние: $d(x, y) = \sum_{i=1}^n |x_i - y_i|$
3. Расстояние Минковского: $d(x, y) = (\sum_{i=1}^n |x_i - y_i|^p)^{\frac{1}{p}}$

Вычислительные подходы

1. Brute Force: вычисляет расстояния до всех точек. Прост, но неэффективен для больших наборов данных.
2. K-D Tree: структура данных, разбивающая пространство, для организации точек в K -мерном пространстве: «Основная идея в том, что если точка A далека от точки B , а точка B близка к точке C , то мы знаем, что точки A и C далеки, без явного вычисления их расстояния». Эффективна для низкоразмерных данных.
3. Ball Tree: иерархическая структура данных на гиперсферах. Эффективна для высокоразмерных данных.

4.4.2 Достоинства

1. Прост в реализации
2. Не делает предположений о распределении данных (непараметрический)
3. Способен улавливать сложные границы решения

4.4.3 Недостатки

1. Вычислительно дорог для больших наборов данных
2. Чувствителен к масштабу признаков
3. Проклятие размерности: качество ухудшается на высокоразмерных данных

4.4.4 Выбор K

1. Нечётное K для бинарной классификации — чтобы избежать ничьих
2. Квадратный корень из N (где N — число объектов) как эмпирическое правило
3. Использовать кросс-валидацию для поиска оптимального K
4. Метод локтя: вычислить ошибку для разных значений K, построить график и найти «локоть» — точку, где ошибка начинает выходить на плато.

4.4.5 Пример кода

```
import numpy as np
from collections import Counter

class KNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def euclidean_distance(self, x1, x2):
        return np.sqrt(np.sum((x1 - x2) ** 2))

    def predict(self, X):
        predictions = [self._predict(x) for x in X]
        return np.array(predictions)

    def _predict(self, x):
        distances = [self.euclidean_distance(x, x_train) for x_train in self.X_train]
        k_indices = np.argsort(distances)[:self.k]
        k_nearest_labels = [self.y_train[i] for i in k_indices]
        most_common = Counter(k_nearest_labels).most_common(1)
        return most_common[0][0]
```

4.4.6 Ссылки

- [Sklearn documentation](#)

4.5 k-means кластеризация

k-means — алгоритм машинного обучения без учителя, используемый для разбиения набора данных на K непересекающихся подгрупп (кластеров). Для оценки используются метрики кластеризации. Не путать с kNN. kNN — алгоритм с учителем для классификации/регрессии, ищущий ближайших соседей; k-means — алгоритм без учителя для кластеризации, ищущий центроиды кластеров.

4.5.1 Как это работает

1. Выбрать число кластеров K
2. Случайно инициализировать K центроидов
3. Повторять до сходимости: - Назначить каждую точку данных ближайшему центроиду - Обновить центроиды, вычислив среднее всех точек, назначенных этому центроиду - Алгоритм сошёлся, когда назначения перестают меняться.
4. Инициализация: $\mu_1, \mu_2, \dots, \mu_K \in \mathbb{R}^n$
5. Шаг назначения: $c^{(i)} := \arg \min_j \|x^{(i)} - \mu_j\|^2$
6. Шаг обновления: $\mu_j := \frac{\sum_{i=1}^m 1_{c^{(i)}=j} x^{(i)}}{\sum_{i=1}^m 1_{c^{(i)}=j}}$

Где:

- μ_j — центроид кластера j
- $c^{(i)}$ — назначение кластера для точки i
- $x^{(i)}$ — i -я точка данных

4.5.2 Метрики расстояния

Обычно используется евклидово расстояние: $d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$

4.5.3 Выбор K

1. Метод локтя: построить график within-cluster sum of squares (WCSS) от K и найти «локоть»
2. Силуэт: измеряет, насколько объект похож на свой кластер по сравнению с другими кластерами
3. Gap Statistic: сравнивает суммарную внутрикластерную вариацию для разных значений k с её ожидаемыми значениями при нулевом референсном распределении данных

4.5.4 Достоинства

1. Простой
2. Хорошо масштабируется на большие наборы данных
3. Гарантирует сходимость
4. Можно делать warm-start с известными центроидами, если они доступны

4.5.5 Недостатки

1. Нужно задавать K заранее
2. Чувствителен к начальному размещению центроидов
3. Чувствителен к выбросам

4.5.6 Вариации

K-means++

K-means++ помогает сходимости за счёт улучшения инициализации. Он выбирает начальные центроиды кластеров через сэмплирование на основе эмпирического распределения вероятностей, отражающего вклад точек в общую inertia (внутрикластерную сумму квадратов).

Мини-батч K-means (k-means с батчами)

Вместо использования всего набора данных на каждой итерации мини-батч k-means использует небольшие случайные батчи данных. Процесс повторяется до сходимости или до достижения максимального числа итераций.

4.5.7 Пример кода

```
import numpy as np

class KMeans:
    def __init__(self, k=3, max_iters=100):
        self.k = k
        self.max_iters = max_iters

    def kmeans_plus_plus(self, X):
        centroids = [X[np.random.randint(X.shape[0])]]
        for _ in range(1, self.k):
            distances = np.min([np.sum((X - c) ** 2, axis=1) for c in centroids], axis=0)
            probabilities = distances / distances.sum()
            cumulative_probabilities = probabilities.cumsum()
            r = np.random.rand()
            for j, p in enumerate(cumulative_probabilities):
                if r < p:
                    centroids.append(X[j])
                    break
        return np.array(centroids)

    def fit(self, X):
        # Randomly initialize centroids
        self.centroids = X[np.random.choice(X.shape[0], self.k, replace=False)]
        # Or use K-means++ initialization
        self.centroids = self.kmeans_plus_plus(X)

        for _ in range(self.max_iters):
            # Assign points to closest centroid
            distances = np.sqrt(((X - self.centroids[:, np.newaxis])**2).sum(axis=2))
            labels = np.argmin(distances, axis=0)

            # Update centroids
            new_centroids = np.array([X[labels == i].mean(axis=0) for i in range(self.k)])

            # Check for convergence
            if np.all(self.centroids == new_centroids):
                break

            self.centroids = new_centroids

        return labels

    def predict(self, X):
        distances = np.sqrt(((X - self.centroids[:, np.newaxis])**2).sum(axis=2))
        return np.argmin(distances, axis=0)
```

4.5.8 Ссылки

- Stanford lecture

Раздел 5. Многорукие бандиты, SVM, снижение размерности, деревья и метрики расстояния

Классические алгоритмы, валидация, регуляризация, компромисс смещения-дисперсии.

5.1 Многорукие бандиты (multi-armed bandits)

Многорукий бандит — это задача последовательного принятия решений, в которой обучающийся агент многократно выбирает одно из k действий (рук), наблюдает стохастическую награду (reward) только для выбранной руки и адаптирует будущие выборы, балансируя между исследованием (exploration — сэмплирование недостаточно протестированных рук, чтобы узнать их ценность) и эксплуатацией (exploitation — выбор руки, которая в текущий момент считается лучшей).

flowchart LR

```
AB[A/B test] -->|adaptive allocation| MAB[Multi-armed<br/>bandit]
MAB -->|observed context| CB[Contextual<br/>bandit]
CB -->|state transitions| RL[Full reinforcement<br/>learning]
```

5.1.1 Постановка задачи

На каждом раунде t обучающийся выбирает руку $a_t \in 1, \dots, k$ и наблюдает награду $r_t \in \mathcal{D}$, где $\mathcal{D}$ — неизвестное распределение награды руки a_t . Цель обучающегося — минимизировать кумулятивный regret (сожаление): разрыв между наградой, которую собрал бы оракул, всегда выбирающий лучшую руку, и наградой, которую фактически собрал обучающийся. Regret — это цена незнания лучшей руки заранее, и он накапливается на каждом раунде, когда обучающийся выбирает неоптимально:

$$\text{Regret}(T) = T \cdot \mu^* - \sum_{t=1}^T \mathbb{E}[r_t]$$

где μ^* — средняя награда лучшей руки, а T — временной горизонт. Хорошая политика достигает $\text{Regret}(T) = O(\log T)$ (логарифмический regret) или $O(\sqrt{T})$ в зависимости от структуры задачи.

Логарифмический regret означает, что кумулятивный regret растёт как $\log T$, а не линейно с T , так что regret на раунд в итоге стремится к нулю по мере того, как обучающийся идентифицирует лучшую руку. UCB и Thompson sampling достигают логарифмического regret на стохастических бандитах с единственной лучшей рукой; epsilon-greedy с константой ϵ никогда не прекращает случайное исследование и накапливает линейный regret $O(T)$.

5.1.2 Основные алгоритмы

- Epsilon-greedy: выбирает лучшую из известных рук с вероятностью $1 - \epsilon$ и случайную руку с вероятностью ϵ . Прост; ϵ может затухать со временем. В матожидании линейный regret, если только ϵ не стремится к нулю.
- UCB (Upper Confidence Bound): выбирает руку, максимизирующую $\hat{\mu}_a + \sqrt{2 \log t / n_a}$, где $\hat{\mu}_a$ — эмпирическая средняя награда, а n_a — число выборов руки a . Детерминированный, логарифмический regret, один гиперпараметр-константа доверительного интервала.
- Thompson sampling: поддерживает байесовскую апостериорную оценку распределения награды каждой руки, тянет сэмпл из каждой апостериорной оценки и выбирает руку с наибольшим сэмплом. Логарифмический regret, конкурентоспособен с UCB или превосходит его эмпирически при несоответствии модели.

Все три масштабируются на k в сотнях; контекстуальные варианты расширяются на большие пространства действий за счёт обмена информацией между руками с похожим контекстом.

5.1.3 Контекстуальные бандиты

Когда перед каждым действием наблюдается контекст x_t (признаки пользователя, запрос, время суток), обучающийся целит в оптимальное действие при данном контексте, а не в одну лучшую руку. Модель награды принимает вид $\mu_a(x)$, параметризованный линейной или нейронной функцией.

Распространённые варианты:

- LinUCB: линейная модель награды с бонусом исследования в стиле UCB, основанным на ковариации оценки параметра.
- Контекстуальный Thompson sampling: апостериорная оценка по вектору параметров, сэмплирование и выбор лучшего действия при данном сэмпле.
- Нейронные бандиты: глубокая модель для $\mu_a(x)$ с исследованием через dropout, ансамбли или bootstrap.

Контекстуальные бандиты расширяют чистые бандиты до персонализированных решений, обуславливая выбор руки наблюдаемым контекстом.

5.1.4 Regret против оценки

Бандиты и A/B-тесты оптимизируют разные цели:

- Бандиты минимизируют кумулятивный regret во время обучения: потери от того, что лучшая рука не выбрана на каждом раунде.
- A/B-тесты минимизируют дисперсию оценки на конкретном контрасте: неопределённость относительно того, какая рука лучше, после фиксированного объёма выборки.

Выбор зависит от того, для чего проводится эксперимент. Бандиты используются, когда выбор на каждом раунде имеет значение (живой трафик, реальная цена исследования) или когда

непрерывное исследование является частью продуктового механизма. А/В-тесты — когда чистое решение «выкатить/убить» важнее, чем regret в период принятия решения.

5.1.5 Применения

- Холодный старт в рекомендательных системах: бандиты назначают новых пользователей или элементы кандидатным политикам, накапливая уверенность. См. «холодный старт».
- Распределение ставок в рекламных аукционах: адаптивное распределение ставок по креативам или стратегиям ставок.
- Оптимизация заголовков и посадочных страниц на контентных или коммерческих сайтах.
- Распределение в клинических испытаниях с адаптивной рандомизацией.
- Онлайн-обучение как упрощённая постановка reinforcement learning без переходов между состояниями.

5.1.6 Связь с reinforcement learning

Многорукий бандит — это задача reinforcement learning с единственным состоянием: действия существуют, награды стохастичны, но среда не переходит между состояниями. Каждый раунд независим от предыдущего. Бандиты — простейшая RL-постановка и распространённая площадка для тестирования алгоритмов исследования.

5.1.7 Типичные подводные камни

- Отложенные награды ломают наивные алгоритмы бандитов. Окна атрибуции должны моделироваться явно, иначе кредит достаётся не той руке.
- Нестационарные распределения наград делают теоретические оценки regret недействительными. Варианты со скользящим окном или дисконтированными наградами справляются с дрейфом.
- Коррелированные руки (общие признаки, похожий контент) означают, что независимые бандиты впустую тратят данные. Контекстуальные бандиты делят статистическую силу между руками с похожим контекстом.
- Бандиты не заменяют хорошо спроектированный А/В-тест, когда цель — подтверждённая оценка причинно-следственного эффекта для конкретного контраста.

5.1.8 Литература

- Sutton & Barto — *Reinforcement Learning: An Introduction* (2nd ed., 2018). Глава 2 покрывает бандиты в контексте RL.
- Lattimore & Szepesvári — *Bandit Algorithms* (Cambridge University Press, 2020). Комплексное современное изложение бандитов, теории regret и контекстуальных вариантов.
- Chapelle & Li — *An Empirical Evaluation of Thompson Sampling* (2011). Канонический эмпирический сравнительный анализ Thompson sampling и UCB.

5.2 SVM (Support Vector Machines)

Support Vector Machines (SVM) — это алгоритм обучения с учителем, используемый для классификации, регрессии и обнаружения выбросов. Support vector machines фокусируются только на точках, которые труднее всего различить, тогда как другие классификаторы обращают внимание на все точки.

Интуиция состоит в том, что если классификатор хорошо разделяет самые трудные точки (которые близки друг к другу), то и другие точки он будет разделять ещё лучше. SVM ищет ближайшие точки, так называемые «опорные векторы» (support vectors — точки подобны векторам, и лучшая разделяющая прямая «зависит от них» или «поддерживается ими»). Затем SVM проводит прямую, соединяющую эти точки. Лучшая разделяющая прямая перпендикулярна ей.

5.2.1 Математическая формулировка

Для задачи бинарной классификации:

1. Линейный SVM: минимизировать $\frac{1}{2}||w||^2$ при ограничении $y_i(w \cdot x_i + b) \geq 1$ для всех i .
2. SVM с мягким отступом (soft margin, допускающий некоторое количество ошибок классификации): минимизировать $\frac{1}{2}||w||^2 + C \sum_{i=1}^n \xi_i$ при ограничениях $y_i(w \cdot x_i + b) \geq 1 - \xi_i$ и $\xi_i \geq 0$ для всех i .

Задачу оптимизации SVM можно переписать как $\frac{1}{2}||w||^2 + C * \sum \max(0, 1 - y_i(w \cdot x_i + b))$ — сумма hinge-потерь по всем обучающим примерам.

Где:

- w — нормальный вектор к гиперплоскости
- b — смещение (bias)
- C — параметр регуляризации
- ξ_i — slack-переменные

5.2.2 Ядерный трюк (kernel trick)

Идея в том, что данные, не линейно разделимые в данном n -мерном пространстве, могут оказаться линейно разделимыми в пространстве большей размерности.

Ядерный трюк предоставляет решение. «Трюк» в том, что ядерные методы представляют данные только через набор попарных сравнений сходства между исходными наблюдениями x (с исходными координатами в пространстве меньшей размерности), вместо того чтобы явно применять преобразования $\phi(x)$ и представлять данные этими преобразованными координатами в пространстве признаков большей размерности.

В ядерных методах набор данных X представляется матрицей ядра размера $n \cdot n$ из попарных сравнений сходства, где элементы (i, j) определяются ядерной функцией: $k(x_i, x_j)$. Эта ядерная функция обладает специальным математическим свойством. Ядерная функция действует как модифицированное скалярное произведение.

1. Linear: $K(x_i, x_j) = x_i \cdot x_j$
2. Polynomial: $K(x_i, x_j) = (\gamma x_i \cdot x_j + r)^d$
3. RBF (Gaussian): $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$
4. Sigmoid: $K(x_i, x_j) = \tanh(\gamma x_i \cdot x_j + r)$

5.2.3 Преимущества

- Эффективен в пространствах высокой размерности
- Экономичен по памяти, поскольку использует только опорные векторы
- Универсален благодаря различным ядерным функциям
- Устойчив к переобучению

5.2.4 Недостатки

- Чувствителен к выбору ядра и параметра регуляризации
- Не даёт напрямую оценок вероятностей (требуется дополнительных методов вроде Platt Scaling, значительно увеличивающих вычислительные затраты)
- Может быть вычислительно затратен для больших наборов данных

5.2.5 Пример кода

```
import numpy as np

class SVM:
    def __init__(self, learning_rate=0.001, lambda_param=0.01, n_iters=1000):
        self.lr = learning_rate
        self.lambda_param = lambda_param
        self.n_iters = n_iters
        self.w = None
        self.b = None

    def fit(self, X, y):
        n_samples, n_features = X.shape
        y_ = np.where(y <= 0, -1, 1)

        self.w = np.zeros(n_features)
        self.b = 0

        for _ in range(self.n_iters):
            for idx, x_i in enumerate(X):
                condition = y_[idx] * (np.dot(x_i, self.w) - self.b) >= 1
                if condition:
                    self.w -= self.lr * (2 * self.lambda_param * self.w)
                else:
                    self.w -= self.lr * (2 * self.lambda_param * self.w - np.dot(x_i, y_))
                    self.b -= self.lr * y_[idx]

    def predict(self, X):
        linear_output = np.dot(X, self.w) - self.b
        return np.sign(linear_output)
```

5.2.6 Ссылки

- [A great visualization](#)
- [Sklearn documentation](#)
- [Duality and Geometry in SVM Classifiers](#)
- [The explanation of the kernel trick](#)

5.3 Снижение размерности

Снижение размерности — это процесс уменьшения числа признаков (измерений) в наборе данных при сохранении как можно большего объёма релевантной информации. Оно преобразует данные высокой размерности в представление меньшей размерности, которое схватывает существенную структуру и закономерности исходных данных.

5.3.1 Для чего используется снижение размерности

- Смягчение проклятия размерности
- Визуализация данных высокой размерности в 2D- или 3D-пространстве
- Снижение шума
- Эффективность по памяти и вычислениям
- Инженерия признаков

5.3.2 Основные подходы к снижению размерности

Линейные методы

- Principal Component Analysis (PCA): находит направления (главные компоненты), вдоль которых данные варьируются больше всего. Проецирует данные на эти направления, чтобы создать представление меньшей размерности.
- Linear Discriminant Analysis (LDA): находит линейные комбинации признаков, которые лучше всего разделяют различные классы.
- Independent Component Analysis (ICA): разделяет многомерные сигналы на аддитивные независимые компоненты. Предполагает, что наблюдаемые данные — это линейная смесь независимых источников сигналов.
- Factor Analysis: моделирует наблюдаемые переменные как линейные комбинации ненаблюдаемых латентных факторов плюс шум. Похож на PCA, но включает модель шума.

Нелинейные методы

- t-Distributed Stochastic Neighbor Embedding (t-SNE): сохраняет локальную структуру окрестности при отображении в меньшие размерности. Особенно эффективен для визуализации, но может быть вычислительно затратен.
- Uniform Manifold Approximation and Projection (UMAP): сохраняет как локальную, так и глобальную структуру лучше, чем t-SNE, и в целом быстрее. Основан на manifold learning и топологическом анализе данных.
- Autoencoders: нейронные сети, обучающиеся сжимать и восстанавливать данные. Сжатое представление в среднем слое служит представлением меньшей размерности.
- Kernel PCA: расширяет PCA до нелинейного снижения размерности с помощью ядерного трюка. Неявно отображает данные в пространство большей размерности, где применяется линейный PCA.

5.3.3 Ковариация

Ковариация измеряет совместную вариативность двух случайных величин. Она показывает направление линейной зависимости между ними. Для набора данных с признаками $X_1, X_2, \dots, X_p$ ковариация между признаками X_i и X_j :

$$\text{Cov}(X_i, X_j) = \frac{1}{n-1} \sum_{k=1}^n (x_{ki} - \bar{x}_i)(x_{kj} - \bar{x}_j)$$

Где:

- n — число наблюдений
- x_{ki} — k -е наблюдение признака i
- $\bar{x}_i$ — среднее признака i

Для набора данных X с n наблюдениями и p признаками ковариационная матрица C — это симметричная матрица размера $p \times p$:

$$C_{ij} = \text{Cov}(X_i, X_j)$$

В матричной форме, если X — центрированная матрица данных (со средним, вычтенным из каждого столбца):

$$C = \frac{1}{n-1} X^T X$$

5.3.4 PCA и ковариация

PCA фундаментально связан с ковариационной матрицей данных (в предположении, что данные центрированы). Главные компоненты — это собственные векторы ковариационной матрицы, а соответствующие собственные числа представляют дисперсию, схваченную каждой главной компонентой.

Как работает PCA

1. **Центрирование данных:** вычесть среднее из каждого признака
2. **Вычисление ковариационной матрицы:** $C = \frac{1}{n-1} X^T X$
3. **Поиск собственных чисел и собственных векторов:** $C\mathbf{v} = \lambda\mathbf{v}$
4. **Сортировка по собственным числам:** упорядочить собственные векторы по убыванию соответствующих собственных чисел. Собственный вектор с наибольшим собственным числом — это первая главная компонента. По сути, первая главная компонента находит направление с наибольшей вариативностью.
5. **Проекция данных:** преобразовать данные с помощью выбранных собственных векторов.

5.3.5 Собственные числа и собственные векторы

Собственный вектор — это направление, которое не меняется при применении линейного преобразования. Собственное число — это коэффициент масштабирования, применяемый вдоль направления собственного вектора.

Singular Value Decomposition (SVD) и PCA

PCA также можно выполнить с помощью SVD на (центрированной) матрице данных X напрямую, без явного формирования ковариационной матрицы. Если $X = USV^T$ — это SVD матрицы X :

- Главные компоненты — это столбцы V .
- Сингулярные числа в S связаны с собственными числами ковариационной матрицы ($\lambda = s^2 / (n - 1)$).

Использование SVD часто численно более устойчиво, особенно для данных высокой размерности.

5.3.6 Метрики оценки

1. Доля объяснённой дисперсии (Explained Variance Ratio)

Доля общей дисперсии, объяснённой выбранными компонентами:

$$\text{EVR} = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^p \lambda_i}$$

2. Ошибка восстановления

Среднеквадратическая ошибка между исходными и восстановленными данными:

$$\text{MSE} = \frac{1}{n} \|\mathbf{X} - \mathbf{X}_{\text{reconstructed}}\|_F^2$$

5.3.7 Практические соображения

- Стандартизируйте признаки перед применением PCA, если они имеют разные масштабы
- PCA работает лучше всего, когда признаки коррелированы

5.3.8 Ссылки

- [A Tutorial on Principal Component Analysis](#)
- [Scikit-learn PCA Documentation](#)
- [The Elements of Statistical Learning - Chapter 14](#)
- [Pattern Recognition and Machine Learning - Chapter 12](#)

5.4 Компромисс смещения-дисперсии (bias-variance trade-off)

Компромисс смещения-дисперсии — это фундаментальное понятие машинного обучения, описывающее баланс между способностью модели подгоняться под обучающие данные (низкое смещение) и её способностью обобщать на новые, ранее не виденные данные (низкая дисперсия).

5.4.1 Смещение (bias)

- Смещение — это разница между средним предсказанием модели и корректным значением, которое мы пытаемся предсказать.

- Иное объяснение: представим, что мы обучаем модели на разных наборах данных; смещение измеряет, насколько далеко предсказания этих разных моделей отстоят от корректных значений.
- Модель с высоким смещением слишком проста и недообучается на данных, имеет сильные модельные предположения.
- Примеры моделей с высоким смещением: линейная регрессия, логистическая регрессия, linear discriminant analysis.

5.4.2 Дисперсия (variance)

- Дисперсия — это вариативность предсказания модели для заданной точки данных или величина, показывающая разброс наших данных.
- Иное объяснение: представим, что мы обучаем модели на разных наборах данных; дисперсия измеряет, насколько предсказания для определённого наблюдения различаются между разными моделями.
- Модель с высокой дисперсией слишком сложна и переобучается на данных, плохо обобщает на ранее не виденные данные.
- Примеры моделей с высокой дисперсией: решающие деревья (когда глубокие), kNN с малым k, SVM (Support Vector Machines).

5.4.3 Компромисс

- По мере уменьшения смещения дисперсия, как правило, растёт, и наоборот. Цель — найти «золотую середину», которая минимизирует и смещение, и дисперсию.
- Этот компромисс по сложности — причина того, почему существует компромисс между смещением и дисперсией. Алгоритм не может одновременно быть и более сложным, и менее сложным.
- Bagging (случайный лес) даёт более высокое смещение и более низкую дисперсию: усреднение нескольких решающих деревьев нацелено на снижение дисперсии ценой небольшого роста смещения. Два независимых дерева будут иметь одинаковое смещение (и их среднее будет таким же), но модели делают разные ошибки, так что их усреднение снижает общую ошибку — дисперсию.
- Boosting (градиентный бустинг) даёт более низкое смещение и более высокую дисперсию: модели сложны, поэтому склонны к переобучению, но каждая модель оптимизирована (стартует из лучших точек), так что они менее склонны вносить смещение.

5.4.4 Разложение полной ошибки

$$TotalError = Bias^2 + Variance + IrreducibleError$$

Где:

- $Bias^2$: насколько далеко предсказания модели отстоят от корректных значений.
- $Variance$: насколько предсказания для заданной точки варьируются между различными реализациями модели.

- *Irreducible Error*: шум в истинной зависимости, который невозможно снизить никакой моделью (неустранимая ошибка).

$$Err(x) = E[(Y - \hat{f}(x))^2] = (E[\hat{f}(x)] - f(x))^2 + E[(\hat{f}(x) - E[\hat{f}(x)])^2] + \sigma_e^2$$

<https://mlu-explain.github.io/bias-variance/>

5.4.5 Недообучение и переобучение

- Недообучение: модели с высоким смещением и низкой дисперсией. Они не способны схватить лежащую в основе закономерность данных.
- Переобучение: модели с низким смещением и высокой дисперсией. Они захватывают шум вместе с лежащей в основе закономерностью данных.

5.4.6 Стратегии управления компромиссом смещения-дисперсии

1. Кросс-валидация
2. Регуляризация
3. Ансамблевые методы: - Bagging: снижает дисперсию (например, случайный лес) - Boosting: снижает смещение, может увеличить дисперсию (например, градиентный бустинг)
4. Отбор и инженерия признаков
5. Настройка сложности модели (например, глубина дерева, число скрытых слоёв)

5.4.7 Ссылки

- Bias-Variance Tradeoff Explanation
- MLU Explain: Bias-Variance

5.5 Случайный лес (Random Forest)

Случайный лес — это метод ансамблевого обучения (bagging), который строит на этапе обучения множество решающих деревьев и выдаёт класс, являющийся модой классов (классификация), либо среднее предсказание (регрессия) отдельных деревьев.

5.5.1 Как работает

- Создание множества решающих деревьев на bootstrap-выборках (с возвращением) обучающих данных.
- Для каждого разбиения в каждом дереве рассматривается только случайное подмножество признаков.
- Для классификации используется голосование большинством деревьев; для регрессии — среднее предсказание.

5.5.2 Преимущества

- Снижает переобучение по сравнению с отдельными решающими деревьями.
- Хорошо справляется с данными высокой размерности благодаря сэмплингованию.

5.5.3 Недостатки

- Менее интерпретируем, чем одиночное решающее дерево.
- Вычислительно более затратен, чем одиночное решающее дерево.

5.5.4 Предпосылки для хорошей производительности

- Наличие реального сигнала в признаках.
- Низкая корреляция между предсказаниями (и ошибками) отдельных деревьев.
Решающее дерево по определению имеет высокую дисперсию, так что использование случайного сэмплингования обеспечивает низкую корреляцию между отдельными деревьями.

5.5.5 Ссылки

- [Explained.ai: Random Forest](#)

5.6 Решающее дерево (Decision Tree)

Решающее дерево — это алгоритм обучения с учителем, используемый как для задач классификации, так и для задач регрессии. Он создаёт модель, которая предсказывает значение целевой переменной, обучаясь простым правилам решения, выведенным из признаков данных.

5.6.1 Структура

- Корневой узел: верхний узел, представляющий весь набор данных.
- Внутренние узлы: узлы, представляющие признаки набора данных и используемые для принятия решений.
- Ветви: связи между узлами, представляющие правила решения.
- Листья (leaf nodes): терминальные узлы, дающие финальный вывод (метка класса (мода обучающих наблюдений) или значение (среднее обучающих наблюдений)).

5.6.2 Процесс обучения

1. Отбор признака: выбор лучшего атрибута для разбиения данных.
2. Определение точки разбиения: выбор лучшей точки разбиения для выбранного признака.
3. Разбиение: разделение набора данных по выбранному признаку и точке разбиения: левый дочерний узел содержит точки, в которых значение признака меньше точки разбиения, правый дочерний узел — со значениями больше точки разбиения.

4. Рекурсия: повторение шагов 1–3 для каждого дочернего узла, пока не будут выполнены критерии остановки.

5.6.3 Критерии разбиения

- Деревья классификации:
 - Gini impurity: $G = 1 - \sum_k (p_k)^2$
 - Энтропия: $S = - \sum_{i=1}^N p_i \log_2 p_i$
- Деревья регрессии:
 - Дисперсия (эквивалент MSE): $D = \frac{1}{\ell} \sum_{i=1}^{\ell} (y_i - \frac{1}{\ell} \sum_{j=1}^{\ell} y_j)^2$

5.6.4 Критерии остановки

- Достигнута максимальная глубина
- Минимальное число наблюдений в узле
- Минимальное снижение impurity
- Все наблюдения в узле принадлежат одному классу

5.6.5 Ансамблевые методы на основе решающих деревьев

- Случайный лес
- Градиентный бустинг

5.6.6 Преимущества

- Легко понять и интерпретировать
- Требуется мало предобработки данных
- Может работать как с числовыми, так и с категориальными данными
- Легко визуализируется

5.6.7 Недостатки

- Может создавать чрезмерно сложные деревья, которые плохо обобщают (переобучение)
- Может быть неустойчивым; небольшие вариации в данных могут привести к совершенно другому дереву
- Смещён в сторону признаков с большим числом уровней (для категориальных переменных)
- Не может предсказывать за пределами диапазона обучающих данных (для задач регрессии)

5.6.8 Регуляризация

- Обрезка (pruning): удаление ветвей, дающих мало предсказательной силы
- Установка минимального числа наблюдений, требуемого в листе
- Установка максимальной глубины дерева
- Установка максимального числа признаков, рассматриваемых для разбиения

5.6.9 Важность признаков

Решающие деревья дают встроенную важность признаков: важность вычисляется на основе того, насколько каждый признак уменьшает взвешенный impurity.

5.6.10 Пример вычисления энтропии

20 шаров: 9 синих, 11 жёлтых. $S_0 = -\frac{9}{20} \log_2 \frac{9}{20} - \frac{11}{20} \log_2 \frac{11}{20} \approx 1$ Теперь 13 шаров: 8 синих и 5 жёлтых: $S_1 = -\frac{5}{13} \log_2 \frac{5}{13} - \frac{8}{13} \log_2 \frac{8}{13} \approx 0.96$

5.6.11 Ссылки

- [Explained.ai: Decision Tree](#)
- [A visual introduction to machine learning: Decision Tree example](#)

5.7 Расчёт расстояний

- Косинусная схожесть измеряет косинус угла между двумя векторами, фактически схватывая сходство их ориентации и игнорируя их длину.
- Евклидово расстояние вычисляет прямолинейное расстояние между концами двух векторов в векторном пространстве. Сильно чувствительно к длине вектора.
- Манхэттенское расстояние вычисляет сумму абсолютных разностей между компонентами двух векторов. Может быть более устойчивым к выбросам, чем евклидово расстояние.
- Скалярное произведение (dot product) вычисляет сумму произведений соответствующих компонент векторов. По сути, это ненормализованная (зависящая от длин векторов) косинусная схожесть.
- Jaccard index (jaccard similarity) вычисляет размер пересечения, делённый на размер объединения (изначально предназначен для множеств). Хорош для разреженных бинарных данных.
- Hamming distance считает число позиций, в которых два вектора различаются.

Раздел 6. Валидация моделей и регуляризация

6.1 Валидация (cross-validation, splits, leakage)

Валидация модели — это процесс оценки того, насколько хорошо обученная модель машинного обучения работает на ранее не виденных данных. Она необходима для оценки способности модели к обобщению и для проверки того, что модель не переобучена и не недообучена. Валидация подразумевает разделение датасета на две или более частей, обучение модели на одной из них и вычисление метрик на всех частях.

Назначение

- Оценка качества модели на новых данных, обеспечение надёжности и устойчивости модели
- Обнаружение переобучения и недообучения
- Сравнение разных моделей или алгоритмических подходов
- Подбор гиперпараметров

Подходы к валидации

Разделение train/validation/test

- Классический способ валидации — разделение датасета на три части. Обучающая выборка используется для обучения модели, валидационная — для оптимизации качества модели, а тестовая — для несмещённой оценки качества. Качество на тестовой выборке следует проверять только после завершения экспериментов на валидационной.
- Иногда используются только две части — train и validation/test

Bootstrapping

- Данные сэмплятся с возвращением, чтобы получить несколько подвыборок
- Полезен для оценки дисперсии модели и построения доверительных интервалов

Кросс-валидация

- Многократное разделение данных на обучающую и валидационную выборки — случайным образом либо по определённым принципам.
- Обычно метрики на train и validation вычисляются для каждого разбиения, а затем считаются среднее значение и стандартное отклонение метрики
- Часто используется отдельная holdout/test-выборка для несмещённой оценки качества

- Для относительно небольших датасетов кросс-валидация предпочтительнее одиночного разделения train/validation/test. Однако когда данных миллионы строк, допустимо использовать одиночное разбиение

Adversarial validation

- Это отдельная техника — она применяется для проверки способности модели к обобщению путём построения задачи бинарной классификации между обучающей и тестовой выборками.
- Работает так: объединяем train и test с метками 0 и 1 соответственно, обучаем бинарный классификатор и измеряем AUC.
- Высокий AUC указывает на сдвиг распределения между train и test; низкий AUC означает, что выборки похожи.
- Признаки с высокой feature importance могут иметь наибольший сдвиг распределения

Способы разделения данных

- Random splitting: разделение данных случайным образом
- Stratified: разделение данных с сохранением одинакового процента каждого класса в каждой подвыборке (стратификация)
- Leave-One-Out: разделение, при котором в валидации остаётся только один объект
- Group: разделение, при котором все объекты одной группы попадают только в одну подвыборку. Например, можно гарантировать, что данные конкретного пользователя окажутся только в одной подвыборке, чтобы избежать утечки данных
- Временной ряд: сохраняет временную упорядоченность данных, полезно при наличии временных зависимостей

Ссылки

- [Explained.ai: The Importance of Data Splitting](#)
- [Sklearn documentation](#)

6.2 Регуляризация (L1, L2, ElasticNet, early stopping и др.)

Регуляризация — это техника, применяемая в машинном обучении для предотвращения переобучения путём добавления штрафного члена к функции потерь. Она препятствует обучению более сложной или гибкой модели, отдавая предпочтение более простой и лучше обобщающейся.

Регуляризация — это набор техник, используемых в машинном обучении для предотвращения переобучения и улучшения обобщения моделей. Распространённый способ применения регуляризации — добавление штрафного члена к функции потерь, но существует множество других методов, в том числе не зависящих от модели. В общем случае регуляризация направлена на:

1. Предотвращение переобучения
2. Улучшение обобщения модели
3. Работу с мультиколлинеарностью в задачах регрессии
4. Отбор признаков (в случае L1-регуляризации)

Распространённые типы регуляризации

L1-регуляризация (Lasso)

- Добавляет к функции потерь сумму абсолютных значений коэффициентов: $Loss + \lambda * \sum |w|$
- Склонна порождать разреженные модели и поэтому может использоваться для отбора признаков
- Может уменьшать коэффициенты строго до нуля

L2-регуляризация (Ridge)

- Добавляет к функции потерь сумму квадратов коэффициентов: $Loss + \lambda * \sum w^2$
- Уменьшает все коэффициенты
- Хорошо справляется с мультиколлинеарностью

Elastic Net

- Комбинация L1- и L2-регуляризации: $Loss + \lambda_1 * \sum |w| + \lambda_2 * \sum w^2$
- Балансирует преимущества L1 и L2
- В отличие от lasso, который может выбрать один признак из группы произвольно, склонен отбирать группы коррелированных переменных совместно

Типы регуляризации, специфичные для модели

- L1, L2, elastic net в линейных моделях
- Pruning и снижение сложности в моделях на основе деревьев
- Параметр C (обратный к силе регуляризации) в SVM

Типы регуляризации, не зависящие от модели

- Early stopping: останавливает обучение, когда качество на валидационной выборке начинает ухудшаться
- Data augmentation: искусственно увеличивает размер обучающей выборки за счёт генерации новых изменённых объектов
- Noise injection: добавление случайного шума к входам или весам в процессе обучения

Типы регуляризации, специфичные для нейросетей

- Dropout: случайным образом отключает часть выходных нейронов во время обучения
- Weight decay (L2): добавляет к функции потерь штрафной член, пропорциональный сумме квадратов весов
- Batch normalization: нормализует вход каждого слоя по батчу, уменьшает internal covariate shift
- Layer normalization: нормализует вход каждого слоя по признакам
- Label smoothing: заменяет жёсткие метки на мягкие вероятности
- Gradient clipping: ограничивает величину градиентов в процессе обратного распространения ошибки
- Stochastic depth: случайным образом отключает целые слои во время обучения

[!warning] Будьте осторожны, применяя сразу много типов регуляризации вслепую. Бывают случаи, когда разные типы регуляризации неожиданно взаимодействуют между собой.

Weight decay (L2) и batch normalization: при совместном использовании с BN L2-штраф больше не оказывает своего исходного регуляризирующего эффекта. Вместо этого он по сути становится эквивалентен адаптивной корректировке скорости обучения! [Источник](#)

Dropout и batch normalization: применяйте BN до dropout, а не после. [Ссылка на обсуждение](#).

6.3 Валидация для временных рядов

Это один из видов валидации, который заслуживает отдельного объяснения из-за высокой вариативности и сложности.

Основная идея проста: имитировать реальный мир и сохранять временную взаимосвязь между подвыборками — обучающий датасет содержит объекты за период строго до валидационного датасета, а валидационный — за период строго до тестового. Например, обучающий датасет содержит данные за недели 1–4, валидационный — за неделю 5, а тестовый — за неделю 6.

Но можно пойти ещё дальше. В примере выше нет промежутка между обучающими и валидационными данными, но между обучающими и тестовыми данными есть промежуток в одну неделю. Чтобы обеспечить одинаковый промежуток между подвыборками, можно использовать только недели 1–3 для первичного обучения и подбирать гиперпараметры модели на неделе 5; после завершения подбора можно переобучить модель с оптимальными параметрами на неделях 2–4 и измерить качество на неделе 6 как на тестовых данных.

Существует два основных подхода к временной валидации.

Forward Chaining

Постепенно увеличивает обучающую выборку, сохраняя постоянным размер валидационной. Подходит для имитации реального сценария, когда данные накапливаются постепенно (expanding window).


```
gantt
  title Forward Chaining
  dateFormat X
  axisFormat %s

  section Fold 1
  Train      :0, 5
  Validation  :5, 7

  section Fold 2
  Train      :0, 7
  Validation  :7, 9

  section Fold 3
  Train      :0, 9
  Validation  :9, 11
```

Sliding Window

Использует окно фиксированного размера, которое скользит по датасету. Подходит для обнаружения сезонных паттернов или когда более релевантны недавние данные.

```
gantt
  title Sliding Window
  dateFormat X
  axisFormat %s

  section Fold 1
  Train      :0, 5
  Validation  :5, 7

  section Fold 2
  Train      :2, 7
  Validation  :7, 9

  section Fold 3
  Train      :4, 9
  Validation  :9, 11
```

Раздел 7. Метрики и функции потерь

7.1 Введение в метрики и функции потерь

Заметки по метрикам оценки качества и функциям потерь, применяемым в разных областях машинного обучения.

Функции потерь

Описывают, как модель учится — это оптимизационный сигнал во время обучения.

- «Общие функции потерь» — cross-entropy, MSE/MAE, KL-дивергенция, L1/L2/Elastic Net регуляризация.
- «NLP-функции потерь» — NLL, perplexity, CTC, triplet, contrastive, RLHF (PPO/DPO).
- «Функции потерь для компьютерного зрения» — focal, dice, IoU/Jaccard, perceptual, adversarial, SSIM.

Метрики оценки

Описывают, как качество модели измеряется — оценка производительности после обучения.

- «F1-метрика» — среднее гармоническое precision и recall, с вариантами macro/micro/weighted.
- «Матрица ошибок» — true/false positives и negatives для задач классификации.
- «Регрессионные метрики» — MSE, RMSE, MAE, R^2 и их варианты.
- «Метрики рекомендательных систем» — NDCG, MAP, MRR, hit rate, покрытие, разнообразие.
- «Метрики компьютерного зрения» — IoU, mAP, pixel accuracy, FID, SSIM.
- «NLP-метрики» — BLEU, ROUGE, METEOR, perplexity, BERTScore.
- «Метрики кластеризации» — silhouette score, Davies-Bouldin, Calinski-Harabasz, purity.

7.2 Регрессионные метрики

Регрессионные метрики — это количественные меры, используемые для оценки качества регрессионных моделей, которые предсказывают непрерывные значения, а не дискретные классы.

Какую метрику когда использовать

Метрика	Когда использовать
MAE	Интерпретируемая ошибка в единицах целевой переменной; менее чувствительна к выбросам.
MSE	Сильно штрафует большие ошибки; дифференцируема (часто используется как функция потерь).
RMSE	Тот же масштаб штрафа, что у MSE, но в единицах целевой переменной; сопоставима с MAE.
MAPE	Процентная ошибка. Для положительных целевых значений, не близких к нулю.
R^2	Доля объяснённой дисперсии (1 = идеально, 0 = предсказание среднего, <0 = хуже).
Adjusted R^2	R^2 со штрафом за добавление неинформативных предикторов.
Pearson r	Сила и направление линейной связи (от -1 до +1).

Метрики на основе ошибок

Mean Absolute Error (MAE)

Среднее абсолютных разностей между предсказанными и фактическими значениями. Легко интерпретируется — выражена в тех же единицах, что и целевая переменная.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Mean Squared Error (MSE)

Среднее квадратов разностей между предсказанными и фактическими значениями. Более чувствительна к выбросам. Дифференцируема, поэтому часто используется как функция потерь.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE)

Квадратный корень из MSE, возвращающий метрику ошибки к исходному масштабу целевой переменной. Более чувствительна к выбросам, чем MAE, но измеряется в тех же единицах, что и целевая переменная.

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Mean Absolute Percentage Error (MAPE)

Среднее абсолютных процентных ошибок. MAPE в 10% означает, что в среднем предсказание отклоняется на 10% от истинного значения. Смещена в сторону недооценки. Лучше использовать для положительных целевых значений, не близких к нулю.

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Метрики качества подгонки

Коэффициент детерминации (R^2)

Доля дисперсии зависимой переменной, которая может быть предсказана независимыми переменными. 1 означает идеальное предсказание, 0 — что качество модели эквивалентно предсказанию среднего. Может быть отрицательным, если модель хуже горизонтальной линии. $R^2 = 0.75$ означает, что 75% дисперсии целевой переменной можно объяснить признаками модели. R^2 имеет тенденцию расти (или оставаться неизменным) при добавлении признаков, даже если эти признаки бесполезны. Модель с большим R^2 будет иметь меньший MSE, и наоборот.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} = 1 - \frac{\text{SSE}}{\text{SST}}$$

Adjusted R²

Модификация R², учитывающая число предикторов в модели. Штрафует за добавление предикторов, не улучшающих модель. Adjusted R² всегда меньше или равен R².

$$\text{Adjusted } R^2 = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

Корреляция, ковариация и R²

Ковариация измеряет, как две переменные изменяются вместе (совместную изменчивость двух переменных).

$$\text{Cov}(X, Y) = \frac{1}{n - 1} \sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})$$

Ковариация может принимать значения от $-\infty$ до $+\infty$: - Положительные значения указывают, что переменные склонны двигаться в одном направлении. - Отрицательные значения указывают, что они склонны двигаться в противоположных направлениях. - Значения, близкие к нулю, указывают на слабую линейную связь.

Коэффициент **корреляции** Пирсона стандартизирует ковариацию, деля её на произведение стандартных отклонений обеих переменных. Он измеряет силу и направление линейной связи между двумя непрерывными переменными.

$$r = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2 \sum_{i=1}^n (Y_i - \bar{Y})^2}}$$

Где σ_X и σ_Y — стандартные отклонения X и Y . Эта стандартизация ограничивает коэффициент корреляции диапазоном от -1 до $+1$:

- $r = +1$ — идеальная положительная линейная связь.
- $r = -1$ — идеальная отрицательная линейная связь.
- $r = 0$ — отсутствие линейной связи.

В простой линейной регрессии (с одной независимой переменной) R² равен квадрату коэффициента корреляции:

$$R^2 = r^2$$

В множественной регрессии R² обобщает эту концепцию:

$$R^2 = 1 - \frac{\sum_{i=1}^n (Y_i - \hat{Y}_i)^2}{\sum_{i=1}^n (Y_i - \bar{Y})^2}$$

Здесь R² равен квадрату множественного коэффициента корреляции — корреляции между наблюдаемыми и предсказанными значениями.

Ссылки

- [Scikit-learn Regression Metrics](#)
- [Paper: R-squared is More Informative than SMAPE, MAE, MAPE, MSE and RMSE](#)

- The Little Book of ML Metrics

7.3 Метрики рекомендательных систем

Метрики рекомендательных систем — это количественные меры, используемые для оценки качества и эффективности алгоритмов рекомендаций. Эти метрики помогают понять, насколько хорошо система предсказывает предпочтения пользователей, ранжирует элементы и предоставляет полезные рекомендации.

Какую метрику когда использовать

Метрика	Когда использовать
Precision@k	Пользователь видит только несколько рекомендаций — они должны быть высокорелевантными.
Recall@k	Достать как можно больше релевантных элементов из большого каталога.
Hit Rate@k	Общая эффективность системы — попадает ли хоть одна рекомендация?
MAP@k	Ранжированные рекомендации, где важен порядок и нужна средняя точность по пользователям.
NDCG@k	Ранжированные рекомендации с градуированной релевантностью — позиции вверху важнее.
MRR@k	Первая хорошая рекомендация важнее всего (поиск, верх ленты).
ILD@k	Защита от filter-bubble — разнообразие внутри одного списка.
Novelty@k	Подталкивать пользователей к непопулярным элементам.
Serendipity@k	Неожиданно и при этом релевантно — приятные открытия.
Coverage	Здоровье длинного хвоста — какая доля каталога вообще рекомендуется.
CTR	Онлайн-поведение кликов по рекомендованным элементам.
Conversion Rate	Последующее действие (покупка, регистрация) на рекомендацию.
User Satisfaction	Опросы / явная обратная связь.

Precision@k

Доля релевантных элементов среди топ-k рекомендаций. Полезна, когда пользователь видит лишь несколько рекомендаций и мы хотим, чтобы эти немногие были высокорелевантными.

$$\text{Precision@k} = \frac{\text{Number of relevant items in top-k recommendations}}{k}$$

Recall@k

Доля релевантных элементов, присутствующих в топ-k рекомендациях. Полезна, когда мы хотим извлечь как можно больше релевантных элементов из большого каталога.

$$\text{Recall@k} = \frac{\text{Number of relevant items in top-k recommendations}}{\text{Total number of relevant items}}$$

Hit Rate@k

Доля пользователей, для которых хотя бы один релевантный элемент появляется в их топ-k рекомендаций. Хороша для оценки общей эффективности системы. Не различает один и несколько релевантных рекомендаций.

$$\text{Hit Rate@k} = \frac{\text{Number of users with at least one hit in top-k}}{\text{Total number of users}}$$

Mean Average Precision (MAP@k)

Среднее значение average precision (AP) по всем пользователям, где AP — это среднее значений precision на каждой релевантной позиции в ранжированных рекомендациях. Полезна для ранжированных рекомендаций, где важен порядок.

$$\text{AP@k} = \frac{1}{\min(m, k)} \sum_{i=1}^k \text{Precision@i} \cdot \text{rel}(i)$$

Где: - m — количество релевантных элементов для пользователя. - $\text{rel}(i)$ — индикаторная функция (1, если элемент на позиции i релевантен; 0 в противном случае).

$$\text{MAP@k} = \frac{1}{|U|} \sum_{u \in U} \text{AP@k}(u)$$

Normalized Discounted Cumulative Gain (NDCG@k)

Измеряет качество ранжирования, присваивая больший вес релевантным элементам, находящимся выше в списке, и нормализуя по идеальному ранжированию. Штрафует за релевантные элементы, появляющиеся ниже по списку.

$$\text{DCG@k} = \sum_{i=1}^k \frac{2^{\text{rel}(i)} - 1}{\log_2(i + 1)}$$

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}}$$

Mean Reciprocal Rank (MRR@k)

Среднее значение reciprocal rank первого релевантного элемента по всем пользователям. Полезна, когда первая хорошая рекомендация важнее всего (поисковые системы).

$$\text{MRR@k} = \frac{1}{|U|} \sum_{u \in U} \frac{1}{\text{rank}_u}$$

Diversity (разнообразие)

Измеряет, насколько разнообразны рекомендованные элементы по различным измерениям. Помогает предотвратить эффект «filter bubble».

Intra-List Diversity (ILD@k) — среднее попарное различие между элементами в списке рекомендаций.

$$\text{ILD@k} = \frac{2}{k(k-1)} \sum_{i=1}^{k-1} \sum_{j=i+1}^k \text{dist}(i, j)$$

Где $\text{dist}(i, j)$ — расстояние или различие между элементами i и j .

Novelty (новизна)

Измеряет, насколько необычны или незнакомы пользователям рекомендованные элементы. Помогает пользователям открывать новый контент за пределами популярных элементов.

$$\text{Novelty@k} = \frac{1}{k} \sum_{i=1}^k -\log_2 \frac{|\text{Users who interacted with item } i|}{|\text{Total users}|}$$

Serendipity

Измеряет, насколько неожиданными и при этом релевантными являются рекомендации. Стремится радовать пользователей открытиями, которые они не нашли бы самостоятельно.

$$\text{Serendipity@k} = \frac{1}{k} \sum_{i=1}^k \text{unexp}(i) \cdot \text{rel}(i)$$

Где: - $\text{unexp}(i)$ — неожиданность элемента i (часто вычисляется как различие от профиля пользователя). - $\text{rel}(i)$ — релевантность элемента i .

Coverage (покрытие)

Item Coverage — доля всех доступных элементов, рекомендуемых хотя бы одному пользователю. Помогает предотвратить проблему «длинного хвоста», когда многие элементы никогда не рекомендуются.

$$\text{Item Coverage} = \frac{|\text{Items recommended to at least one user}|}{|\text{All available items}|}$$

User Coverage — доля пользователей, получающих хотя бы одну рекомендацию.

$$\text{User Coverage} = \frac{|\text{Users receiving at least one recommendation}|}{|\text{All users}|}$$

Conversion Rate

Процент рекомендаций, приводящих к желаемому действию (например, клику, покупке).

$$\text{Conversion Rate} = \frac{\text{Number of recommendations resulting in conversion}}{\text{Total number of recommendations}}$$

Click-Through Rate (CTR)

Отношение кликов к показам рекомендованных элементов.

$$\text{CTR} = \frac{\text{Number of clicks on recommendations}}{\text{Number of recommendation impressions}}$$

User Satisfaction

Прямое измерение удовлетворённости пользователей рекомендациями, часто собираемое через опросы или механизмы обратной связи.

Ссылки

- [Evaluating Recommendation Systems: A Guide](#)
- [Ranking Metrics for Information Retrieval Systems](#)

7.4 NLP-метрики

NLP-метрики оценивают задачи от машинного перевода и реферирования до ответов на вопросы и разметки последовательностей. Большинство из них сводится к совпадению n-грамм, эмбеddинг-схожести или precision/recall в стиле классификации, адаптированной к типу выхода.

Какую метрику когда использовать

Метрика	Когда использовать
BLEU	Перевод / генерация; precision по n-граммам.
ROUGE-N / L / S	Реферирование; recall по n-граммам, LCS или skip-биграмм.
METEOR	Перевод с учётом синонимов и стемминга.
chrF	F-мера по n-граммам на уровне символов; устойчива для морфологически богатых языков.
BERTScore	Семантическое сходство через эмбеddинги BERT.
MoverScore	Семантическое расстояние через Earth Mover's на контекстных эмбеddингах.
Perplexity	Языковое моделирование — насколько хорошо модель предсказывает последовательность.
Bits per Character (BPC)	То же, что и perplexity, но на уровне символов.
Exact Match (EM)	QA — совпадает ли предсказанный ответ точно?
QA F1	QA — F1 на уровне слов (bag-of-words).
Answer Accuracy	QA с выбором из вариантов.
MRR	Ранг первого правильного ответа (QA / IR).
Entity-level F1	NER на уровне сущностей.
Span-based F1	NER с точным совпадением спанов.
CoNLL Score	NER, усреднённый по типам сущностей.

Метрики машинного перевода и генерации текста

Используются для перевода, реферирования, описания изображений и диалогов.

BLEU (Bilingual Evaluation Understudy)

Измеряет precision совпадений n-грамм между кандидатом-переводом и референс-переводами. Умеренно коррелирует с человеческой оценкой; плохо улавливает беглость и семантический смысл. Предпочитает более короткие предложения.

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

Где: - p_n — модифицированная precision по n-граммам. - w_n — вес для precision n-грамм (обычно равномерные веса). - BP — brevity penalty (штраф за краткость), штрафующий короткие переводы. - N — максимальный размер n-граммы (обычно 4).

METEOR (Metric for Evaluation of Translation with Explicit ORdering)

F-мера на основе совпадений слов с учётом синонимов, стемминга и порядка слов.

$$\text{METEOR} = F_{mean} \cdot (1 - \text{Penalty})$$

Где: - F_{mean} — взвешенное среднее гармоническое precision и recall. - Penalty учитывает фрагментацию (плохой порядок слов).

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

Измеряет recall по n-граммам (а иногда precision / F1) между сгенерированным и референс-текстами.

- **ROUGE-N** — совпадение n-грамм.

$$\text{ROUGE-N} = \frac{\sum_{S \in \{\text{References}\}} \sum_{\{\text{gram}_n \in S\} \text{Count}} (\text{gram}_n)}{\sum_{\{\text{gram}_n \in S\} \text{Count}} (\text{gram}_n)}$$

- **ROUGE-L** — наибольшая общая подпоследовательность (LCS). Улавливает сходство структуры на уровне предложения.
- **ROUGE-S** — совпадение skip-биграмм (пары слов в порядке предложения с допустимыми пропусками). ROUGE-SU расширяет ROUGE-S добавлением униграмм.

chrF

F-мера по n-граммам на уровне символов.

$$\text{chrF} = (1 + \beta^2) \cdot \frac{\text{chrP} \cdot \text{chrR}}{\beta^2 \cdot \text{chrP} + \text{chrR}}$$

Где: - chrP — precision по символьным n-граммам. - chrR — recall по символьным n-граммам. - β определяет важность recall (обычно $\beta = 2$).

BERTScore

Использует эмбединги BERT для вычисления сходства между кандидатом и референс-переводом на уровне токенов.

MoverScore

Использует контекстные эмбединги и Earth Mover's Distance для измерения семантического расстояния между сгенерированным и референс-текстами.

Perplexity (перплексия)

Измеряет, насколько хорошо модель предсказывает выборку. Меньшая перплексия означает лучшее предсказание. Это экспонента от среднего отрицательного log-likelihood последовательности.

$$\text{Perplexity} = 2^{-\frac{1}{N} \sum_{i=1}^N \log_2 P(w_i | w_1, \dots, w_{i-1})}$$

Где: - $P(w_i | w_1, \dots, w_{i-1})$ — условная вероятность слова w_i при заданных предыдущих словах.
- N — количество слов.

Bits Per Character (BPC)

Аналог perplexity, измеряемый на уровне символов.

$$\text{BPC} = -\frac{1}{N} \sum_{i=1}^N \log_2 P(c_i | c_1, \dots, c_{i-1})$$

Где: - $P(c_i | c_1, \dots, c_{i-1})$ — условная вероятность символа c_i при заданных предыдущих символах. - N — общее число символов.

Метрики ответов на вопросы (QA)

Exact Match (EM)

Бинарная мера, показывающая, точно ли совпадает предсказанный ответ с ground truth. Может также вычисляться как процент предсказаний, точно совпадающих с одним из эталонных ответов.

$$\text{EM} = \frac{1}{N} \sum_{i=1}^N 1(\text{prediction}_i = \text{groundtruth}_i)$$

F1 Score

F1 на уровне слов между предсказанием и ground truth, при этом и предсказание, и эталон трактуются как bags of words.

Answer Accuracy

Для QA с выбором из вариантов — доля вопросов, на которые дан правильный ответ.

Mean Reciprocal Rank (MRR)

Для QA-моделей, возвращающих ранжированный список ответов, — среднее значение reciprocal rank правильного ответа.

$$\text{MRR} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i}$$

Метрики разметки последовательностей

Распознавание именованных сущностей (NER) и разметка частей речи (POS-теггинг) включают идентификацию и классификацию именованных сущностей или частей речи в тексте.

Entity-Level F1 Score

F1, вычисленная на уровне сущностей, а не на уровне токенов.

Span-Based F1 Score

F1 на основе точного совпадения спанов сущностей.

Partial Matching Metrics

- **Partial Precision / Recall** — учитывают частичное перекрытие между предсказанными и истинными сущностями.
- **Type-Based Evaluation** — отдельная оценка классификации типов сущностей и определения границ сущностей.

CoNLL Score

Среднее F1-оценок по всем типам сущностей.

Ссылки

- [ROUGE: A Package for Automatic Evaluation of Summaries](#)
- [BLEU: a Method for Automatic Evaluation of Machine Translation](#)
- [HuggingFace Evaluate Library](#)

7.5 Метрики кластеризации

Метрики кластеризации — это количественные меры, используемые для оценки качества и производительности алгоритмов кластеризации.

Какую метрику когда использовать

Метрика	Когда использовать
Silhouette	Нет ground truth; насколько хорошо каждая точка соответствует своему кластеру по сравнению с ближайшим.
Davies-Bouldin	Нет ground truth; меньше = лучше разделение.
Calinski-Harabasz	Нет ground truth; больше = чётче определённые кластеры.
Dunn	Нет ground truth; плотные кластеры, далеко друг от друга.
Inertia	Компактность. Используется в методе локтя при выборе k .
Adjusted Rand Index (ARI)	Есть ground truth; сходство относительно случайного baseline.
Normalized Mutual Information (NMI)	Есть ground truth; взаимная информация, нормализованная по энтропии.

Метрика	Когда использовать
Fowlkes-Mallows	Есть ground truth; геометрическое среднее precision и recall.
Homogeneity / Completeness / V-measure	Есть ground truth; чистота классов, чистота кластеров и их среднее гармоническое.

Внутренние метрики — intrinsic (без ground truth)

Эти метрики оценивают качество кластеризации только на основе данных и результата кластеризации, без необходимости в истинных метках.

Silhouette Coefficient (коэффициент силуэта)

Измеряет, насколько объект похож на свой кластер по сравнению с другими кластерами. Значения от -1 до 1 : $+1$ = хорошее разделение и сцеплённость, 0 = образец близок к границе решения между кластерами, -1 = плохое разделение (слишком близко к соседнему кластеру).

$$\text{Silhouette}(i) = \frac{b(i) - a(i)}{\max a(i), b(i)}$$

Где: $a(i)$ — среднее расстояние между точкой i и всеми другими точками того же кластера. $b(i)$ — среднее расстояние между точкой i и всеми точками ближайшего соседнего кластера.

Silhouette score для кластеризации — это среднее значение коэффициента силуэта по всем образцам:

$$\text{Silhouette Score} = \frac{1}{n} \sum_{i=1}^n \text{Silhouette}(i)$$

Davies-Bouldin Index

Измеряет среднее сходство между кластерами, где сходство — это отношение внутрикластерных расстояний к межкластерным. Меньшие значения указывают на лучшую кластеризацию (0 — минимум).

$$\text{DB} = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{d(c_i, c_j)} \right)$$

Где: k — число кластеров. σ_i — среднее расстояние точек в кластере i до его центроида. $d(c_i, c_j)$ — расстояние между центроидами кластеров i и j .

Calinski-Harabasz Index (Variance Ratio Criterion)

Отношение межкластерной дисперсии к внутрикластерной. Большие значения указывают на лучше определённые кластеры.

$$\text{CH} = \frac{\text{Tr}(B_k)/(k-1)}{\text{Tr}(W_k)/(n-k)}$$

Где: B_k — матрица межкластерной дисперсии. W_k — матрица внутрикластерной дисперсии. Tr — след матрицы. n — общее число образцов. k — число кластеров.

Dunn Index

Отношение минимального межкластерного расстояния к максимальному внутрикластерному. Большие значения указывают на лучшую кластеризацию.

$$\text{Dunn} = \frac{\min_{i \neq j} d(c_i, c_j)}{\max_{1 \leq k \leq K} \text{diam}(c_k)}$$

Где: - $d(c_i, c_j)$ — расстояние между кластерами i и j . - $\text{diam}(c_k)$ — диаметр кластера k (максимальное расстояние между любыми двумя точками кластера).

Inertia (Within-cluster Sum of Squares)

Сумма квадратов расстояний образцов до ближайшего центра кластера. Меньшие значения указывают на более компактные кластеры.

$$\text{Inertia} = \sum_{i=1}^n \min_{c_j \in C} \|x_i - c_j\|^2$$

Где: - x_i — точка данных. - c_j — центр кластера j . - C — множество всех центров кластеров.

Внешние метрики — требуется ground truth

Эти метрики сравнивают результаты кластеризации с известными истинными метками.

Adjusted Rand Index (ARI)

Измеряет сходство между двумя кластеризациями с поправкой на случайность. Значения от -1 до 1; 1 = полное совпадение, 0 = случайное назначение, отрицательные = хуже случайного.

$$\text{ARI} = \frac{\sum_{ij} \binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}{\frac{1}{2} \left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2} \right] - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2} \right] / \binom{n}{2}}$$

Где: - n_{ij} — количество объектов и в классе i , и в кластере j . - a_i — количество объектов в классе i . - b_j — количество объектов в кластере j . - n — общее число объектов.

Normalized Mutual Information (NMI)

Измеряет взаимную информацию между назначениями кластеризации и ground truth, нормализованную средней энтропией обеих. Значения от 0 до 1; 1 = полное совпадение.

$$\text{NMI}(U, V) = \frac{2 \times I(U, V)}{H(U) + H(V)}$$

Где: - $I(U, V)$ — взаимная информация между кластеризациями U и V . - $H(U)$ и $H(V)$ — энтропии U и V .

Fowlkes-Mallows Score

Геометрическое среднее precision и recall. Значения от 0 до 1; 1 = полное совпадение.

$$FMI = \sqrt{\frac{TP}{TP + FP} \times \frac{TP}{TP + FN}}$$

Где TP, FP и FN получены из подсчёта пар точек: - **True Positive (TP)** — пара в одном кластере в обеих кластеризациях. - **False Positive (FP)** — пара в одном кластере в предсказанной кластеризации, но не в ground truth. - **False Negative (FN)** — пара в разных кластерах в предсказанной кластеризации, но в одном кластере в ground truth.

Homogeneity, Completeness и V-measure

- **Homogeneity (однородность)** — каждый кластер содержит только членов одного класса (значения от 0 до 1).
- **Completeness (полнота)** — все члены класса назначены в один кластер (значения от 0 до 1).
- **V-measure** — среднее гармоническое homogeneity и completeness (значения от 0 до 1).

$$V\text{-measure} = 2 \times \frac{\text{homogeneity} \times \text{completeness}}{\text{homogeneity} + \text{completeness}}$$

Contingency Matrix (таблица сопряжённости)

Таблица, показывающая распределение точек данных по предсказанным кластерам и истинным классам. Сама по себе не является метрикой, но служит основой для многих внешних метрик.

Определение оптимального числа кластеров

- **Метод локтя (Elbow Method)** — построение зависимости inertia от числа кластеров и поиск точки «локтя».
- **Метод силуэта (Silhouette Method)** — выбор числа кластеров, максимизирующего silhouette score.
- **Gap Statistic** — сравнение внутрикластерной дисперсии с ожидаемой при нулевом референсном распределении.

Ссылки

- [Scikit-learn documentation on clustering metrics](#)
- [Wikipedia: Cluster Analysis](#)

7.6 Метрики компьютерного зрения

Метрики компьютерного зрения оценивают модели в задачах детекции, сегментации, генерации, 3D и оценки позы. Большинство сводится к мере перекрытия в стиле precision/recall, адаптированной к типу предсказания (bounding box, маска, ключевая точка, облако точек, изображение).

Какую метрику когда использовать

Метрика	Когда использовать
IoU	Перекрытие bounding box или маски с ground truth.

Метрика	Когда использовать
AP	Площадь под precision-recall для класса при заданном пороге IoU.
mAP	Среднее AP по классам (и часто по порогам IoU).
Pixel Accuracy	Доля правильно классифицированных пикселей — доминируется крупными классами.
mIoU	Среднее IoU по классам (стандартная метрика для семантической сегментации).
FWIoU	mIoU, взвешенный по частоте класса.
Dice Coefficient	Эквивалент F1 для масок сегментации.
Mask AP	AP, вычисленный по маскам вместо bounding box.
Panoptic Quality (PQ)	Паноптическая сегментация — segmentation quality × recognition quality.
FID	Качество/разнообразие генеративных изображений относительно реального распределения.
Inception Score (IS)	Чёткость и разнообразие генеративных изображений.
SSIM	Перцептуальное сходство через luminance/contrast/structure.
PSNR	Качество реконструкции (шумоподавление, super-resolution).
LPIPS	Перцептуальное сходство через глубокие признаки.
Chamfer / EMD	Расстояние между облаками точек.
PCK / MPJPE / OKS	Точность ключевых точек позы.

Метрики детекции объектов

Модели детекции объектов предсказывают bounding box вокруг объектов и классифицируют их.

Intersection over Union (IoU)

Измеряет перекрытие между предсказанным и ground truth bounding box.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

Average Precision (AP)

Площадь под кривой Precision-Recall для конкретного класса, вычисленная при заданном пороге IoU.

$$\text{AP} = \int_0^1 p(r) dr$$

Где $p(r)$ — precision на уровне recall r .

Mean Average Precision (mAP)

Среднее значение AP по всем классам объектов, часто вычисляемое при нескольких порогах IoU.

$$mAP = \frac{1}{n} \sum_{i=1}^n AP_i$$

Метрики семантической сегментации

В семантической сегментации каждый пиксель принадлежит одному классу.

Pixel Accuracy

Доля правильно классифицированных пикселей среди всех пикселей. Может доминироваться крупными классами (например, фоном).

$$\text{Pixel Accuracy} = \frac{\text{Number of correctly classified pixels}}{\text{Total number of pixels}}$$

Mean Intersection over Union (mIoU)

Среднее IoU по всем классам.

$$IoU_c = \frac{TP_c}{TP_c + FP_c + FN_c}$$

$$mIoU = \frac{1}{n_c} \sum_{c=1}^{n_c} IoU_c$$

Где: - TP_c — число true positive пикселей для класса c . - FP_c — число false positive пикселей для класса c . - FN_c — число false negative пикселей для класса c . - n_c — число классов.

Frequency Weighted IoU (FWIoU)

Взвешенная версия mIoU, учитывающая дисбаланс классов.

$$FWIoU = \frac{1}{\sum_{k=1}^{n_c} t_k} \sum_{c=1}^{n_c} t_c \cdot IoU_c$$

Где t_c — общее число пикселей, действительно принадлежащих классу c .

Dice Coefficient (коэффициент Дайса)

Эквивалент F1 для масок сегментации.

$$Dice_{class} = \frac{2 \cdot TP_{class}}{(2 \cdot TP_{class} + FP_{class} + FN_{class})} = \frac{2 \cdot IoU}{(IoU + 1)}$$

Метрики instance-сегментации

Instance-сегментация включает как семантическую сегментацию, так и различение экземпляров (разделение отдельных объектов).

Mask AP

Average precision, вычисленный на основе IoU между предсказанными и ground truth масками вместо bounding box.

Panoptic Quality (PQ)

Объединяет качество распознавания и качество сегментации для задач паноптической сегментации.

$$\text{PQ} = \underbrace{\frac{\sum_{(p,g) \in TP} \text{IoU}(p,g)}{|TP|}}_{\text{segmentation quality (SQ)}} \times \underbrace{\left(\frac{|TP|}{|TP| + |FP| + |FN|} \right)}_{\text{recognition quality (RQ)}}$$

Где: - p — предсказанный сегмент. - g — ground truth сегмент. - TP, FP, FN — true positives, false positives и false negatives.

Метрики генерации и синтеза изображений

Эти метрики оценивают качество, разнообразие и реалистичность сгенерированных изображений.

Fréchet Inception Distance (FID)

Измеряет расстояние между распределением признаков сгенерированных и реальных изображений, извлечённых предобученной сетью Inception. Сравнивает среднее и ковариацию этих распределений признаков. Меньшие значения указывают на более реалистичные сгенерированные изображения.

$$\text{FID} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2\sqrt{\Sigma_r \Sigma_g})$$

Где: - μ_r и μ_g — средние представления признаков реальных и сгенерированных изображений.
- Σ_r и Σ_g — ковариационные матрицы представлений признаков.

Inception Score (IS)

Измеряет качество (чёткость, узнаваемость предобученной сетью Inception) и разнообразие сгенерированных изображений.

$$\text{IS} = \exp(\mathbb{E}_x[\text{KL}(p(y|x)||p(y))])$$

Где: - $p(y|x)$ — условное распределение классов для изображения x . - $p(y)$ — маргинальное распределение классов.

Structural Similarity Index (SSIM)

Измеряет перцептуальное различие между двумя изображениями на основе яркости (luminance), контраста и структуры. Диапазон от -1 до 1 (или от 0 до 1); 1 = полное сходство. Более согласована с человеческим восприятием, чем PSNR/MSE.

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

Где: - μ_x и μ_y — средние значения пикселей. - σ_x^2 и σ_y^2 — дисперсии. - σ_{xy} — ковариация. - c_1 и c_2 — константы для избежания деления на ноль.

Peak Signal-to-Noise Ratio (PSNR)

Измеряет качество реконструированных изображений в таких задачах, как шумоподавление или super-resolution. Отношение максимально возможной мощности сигнала к мощности искажающего шума, влияющего на его точность. Основана на MSE.

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right)$$

Где: - MAX_I — максимально возможное значение пикселя. - MSE — mean squared error между изображениями.

Learned Perceptual Image Patch Similarity (LPIPS)

Измеряет перцептуальное сходство через глубокие признаки предобученных сетей (VGG, AlexNet). Согласована с человеческим восприятием лучше, чем попиксельные метрики, такие как MSE.

Метрики 3D-зрения

Метрики для оценки 3D-реконструкции, оценки глубины и обработки облаков точек.

Оценка глубины

- **Mean Absolute Error (MAE)** — среднее абсолютное различие между предсказанными и ground truth значениями глубины.
- **Root Mean Squared Error (RMSE)** — квадратный корень из среднего квадратов разностей.
- **Threshold Accuracy** — процент пикселей, для которых отношение предсказанной глубины к ground truth находится в пределах порога t (обычно $t \in 1.25, 1.25^2, 1.25^3$).

Облако точек

- **Chamfer Distance** — среднее расстояние от каждой точки одного облака до её ближайшего соседа в другом облаке.

$$\text{CD}(S_1, S_2) = \frac{1}{|S_1|} \sum_{x \in S_1} \min_{y \in S_2} \|x - y\|_2 + \frac{1}{|S_2|} \sum_{y \in S_2} \min_{x \in S_1} \|y - x\|_2$$

- **Earth Mover's Distance (EMD)** — минимальная «стоимость» преобразования одного облака точек в другое.
- **F-Score** — среднее гармоническое precision и recall при заданном пороге расстояния.

3D-реконструкция

- **Volumetric IoU** — intersection over union для 3D-объёмов.
- **Surface-to-Surface Distance** — среднее расстояние между реконструированной и ground truth поверхностями.

Метрики оценки позы человека

Percentage of Correct Keypoints (PCK)

Процент предсказанных ключевых точек, попадающих в пределы порога расстояния от ground truth ключевых точек.

Mean Per Joint Position Error (MPJPE)

Среднее евклидово расстояние между предсказанными и ground truth позициями суставов.

Object Keypoint Similarity (OKS)

Аналог IoU, но для ключевых точек — учитывает видимость и масштаб ключевых точек.

$$\text{OKS} = \frac{\sum_i \exp(-d_i^2 / (2s^2 k_i^2)) \delta(v_i > 0)}{\sum_i \delta(v_i > 0)}$$

Где: - d_i — евклидово расстояние между предсказанной и ground truth ключевой точкой i . - s — масштаб объекта. - k_i — константа для каждой ключевой точки. - v_i — флаг видимости ключевой точки i .

Ссылки

- [COCO Evaluation Metrics](#)
- [Scikit-image Comparison Metrics](#)
- [PyTorch Vision Metrics](#)

7.7 Общие функции потерь (классификация и регрессия)

Функции потерь (также называемые целевыми функциями или функциями стоимости) — это математические меры ошибки между предсказанными и фактическими значениями. Они количественно отражают, насколько хорошо работает модель, и обеспечивают оптимизационный сигнал для обучения.

Для функций потерь, специфичных для конкретных областей, см. «NLP-функции потерь» и «Функции потерь для компьютерного зрения». Для метрик оценки см. «Метрики и функции потерь».

Какую функцию потерь когда использовать

Функция потерь	Когда использовать
Binary Cross-Entropy (BCE)	Бинарная классификация.
Categorical Cross-Entropy (CCE)	Многоклассовая классификация с единственным правильным классом.
Label Smoothing CE	Уменьшение переуверенности; регуляризация вероятностей классов.
MSE	Регрессия; сильно штрафует большие ошибки; дифференцируема.
RMSE	MSE в собственных единицах целевой переменной.
MAE	Регрессия, менее чувствительна к выбросам, чем MSE.
KL Divergence	Сопоставление распределений — VAE, knowledge distillation.
L1 / Lasso	Разреженные веса; отбор признаков.
L2 / Ridge	Штраф за большие веса; стабильное обучение.
Elastic Net	L1 + L2 совмещённые.

Cross-Entropy Loss

Cross-entropy loss (или log-loss) измеряет качество модели классификации, выход которой — значение вероятности в диапазоне от 0 до 1.

Binary Cross-Entropy (BCE)

$$L_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Categorical Cross-Entropy (CCE)

Используется для многоклассовых задач, где каждый образец принадлежит единственному классу.

$$L_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c})$$

Где: - y — ground truth метка. - $\hat{y}$ — предсказанная вероятность. - N — количество образцов. - C — количество классов.

Label Smoothing Cross-Entropy

Помогает предотвратить переуверенность, заменяя one-hot закодированный ground truth смесью исходных меток и равномерного распределения.

$$\tilde{y}_{i,c} = (1 - \alpha) \cdot y_{i,c} + \alpha \cdot \frac{1}{C}$$

Mean Squared Error (MSE)

Среднее квадратов ошибок между предсказанными и фактическими значениями.

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE)

Квадратный корень из MSE — в тех же единицах, что и целевая переменная.

Mean Absolute Error (MAE)

Абсолютные разности вместо квадратов — менее чувствительна к выбросам.

$$L_{\text{MAE}} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Kullback-Leibler Divergence (KL-дивергенция)

Измеряет, насколько одно распределение вероятностей отклоняется от второго, ожидаемого распределения.

$$L_{KL} = \sum_i P(i) \log \left(\frac{P(i)}{Q(i)} \right)$$

Где: - P — истинное распределение. - Q — аппроксимированное распределение.

Применения: вариационные автоэнкодеры (VAE), knowledge distillation, сопоставление распределений.

Варианты: - **Reverse KL Divergence** — меняет порядок распределений, что приводит к иному поведению. - **Jensen-Shannon Divergence** — симметризованная и сглаженная версия KL-дивергенции.

Регуляризационные функции потерь

Обычно добавляются к основной функции потерь для предотвращения переобучения и улучшения обобщения.

L1 Regularization (Lasso)

$$L_{L1} = \lambda \sum_{i=1}^n |w_i|$$

Свойства: - Поощряет разреженные веса (многие веса становятся точно нулевыми). - Менее чувствительна к выбросам, чем L2. - Может использоваться для отбора признаков.

L2 Regularization (Ridge)

$$L_{L2} = \lambda \sum_{i=1}^n w_i^2$$

Свойства: - Сильнее штрафует большие веса. - Редко обнуляет веса полностью. - Даёт более стабильные решения, чем L1-регуляризация.

Elastic Net

$$L_{\text{ElasticNet}} = \lambda_1 \sum_{i=1}^n |w_i| + \lambda_2 \sum_{i=1}^n w_i^2$$

Свойства: - Объединяет L1- и L2-регуляризацию. - Может отбирать группы коррелированных признаков. - Более устойчива, чем L1 или L2 по отдельности.

Ссылки

- [PyTorch Loss Functions Documentation](#)
- [TensorFlow Loss Functions Guide](#)
- [A Gentle Introduction to Cross-Entropy for Machine Learning](#)

7.8 NLP-функции потерь

Функции потерь, обычно используемые в задачах NLP. Для общих функций потерь (cross-entropy, MSE, KL-дивергенция) см. «Общие функции потерь». Для метрик оценки NLP см. «NLP-

метрики».

Какую функцию потерь когда использовать

Функция потерь	Когда использовать
Negative Log-Likelihood (NLL)	Языковое моделирование, предсказание последовательностей.
Perplexity	Оценочная форма NLL — геометрическое среднее обратных вероятностей.
CTC	Sequence-to-sequence без явного выравнивания (речь, OCR).
Triplet	Метрическое обучение; похожие объекты близко, непохожие далеко.
Contrastive	Сиамские сети; обучение дискриминативных признаков.
PPO	Policy-gradient RL с клиппированными обновлениями для стабильности.
DPO	Дообучение LLM на предпочтениях без reward-модели.

Negative Log-Likelihood (NLL) Loss

Обычно используется в языковом моделировании и предсказании последовательностей.

$$L_{\text{NLL}} = -\frac{1}{N} \sum_{i=1}^N \log(p(y_i|x_i))$$

Где $p(y_i|x_i)$ — предсказанная вероятность истинного токена/класса.

Применения: языковое моделирование, машинный перевод, генерация текста, предсказание последовательностей.

Perplexity (перплексия)

Экспоненциальное преобразование среднего отрицательного log-likelihood, делающее его интерпретируемым как взвешенное среднее число альтернатив, относительно которых модель не уверена. Perplexity — это метрика оценки, но минимизация NLL эквивалентна минимизации перплексии.

$$\text{Perplexity} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log p(y_i|y_{<i}) \right)$$

Где $p(y_i|y_{<i})$ — вероятность i -го токена при заданных предыдущих токенах.

Применения: языковое моделирование, оценка генерации текста, распознавание речи.

Connectionist Temporal Classification (CTC) Loss

Выравнивает sequence-to-sequence данные без необходимости предварительно сегментированных обучающих данных или явных выравниваний.

$$L_{\text{CTC}} = -\log \left(\sum_{\pi \in \mathcal{A}^{-1}(y)} \prod_{t=1}^T p(\pi_t | x) \right)$$

Где: - $\mathcal{A}^{-1}(y)$ — множество всех возможных выравниваний, соответствующих целевой последовательности y . - $p(\pi_t | x)$ — вероятность выравнивания π в момент времени t при заданном входе x .

Применения: распознавание речи, распознавание рукописного текста, выравнивание белковых последовательностей.

Triplet Loss

Обучает эмбединги, в которых похожие объекты находятся ближе друг к другу, а непохожие — дальше.

$$L_{\text{triplet}} = \max(d(a, p) - d(a, n) + \text{margin}, 0)$$

Где: - a — образец-якорь. - p — положительный образец, похожий на якорь. - n — отрицательный образец, непохожий на якорь. - d — функция расстояния (обычно евклидова или косинусная). - margin — гиперпараметр.

Применения: sentence-эмбединги, схожесть документов, распознавание лиц, поиск изображений.

Contrastive Loss

Используется для обучения дискриминативных признаков путём сближения похожих образцов и отдаления непохожих.

$$L_{\text{contrastive}} = (1 - Y) \cdot \frac{1}{2} \cdot D^2 + Y \cdot \frac{1}{2} \cdot \max(0, \text{margin} - D)^2$$

Где: - Y равно 0 для непохожих пар и 1 для похожих пар. - D — расстояние между образцами.

Применения: схожесть предложений, обучение текстовых эмбедингов, сиамские сети для сравнения документов.

Функции потерь для Reinforcement Learning from Human Feedback (RLHF)

PPO (Proximal Policy Optimization) Loss

$$L_{\text{PPO}} = \mathbb{E} [\min(r_t(\theta) \cdot A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A_t)]$$

Где: - $r_t(\theta)$ — отношение вероятности новой политики к вероятности старой политики. - A_t — оценка advantage. - ϵ — гиперпараметр, ограничивающий обновления политики.

Direct Preference Optimization (DPO) Loss

$$L_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{p_{\theta}(y_w | x)}{p_{\theta}(y_l | x)} - \beta \log \frac{p_{\theta_{\text{ref}}}(y_w | x)}{p_{\theta_{\text{ref}}}(y_l | x)} \right) \right]$$

Где: - π_θ — обучаемая политика. - π_{ref} — референсная политика. - (x, y_w, y_l) — вход, предпочтительный выход и неpreferируемый выход. - β — гиперпараметр.

Применения: дообучение языковых моделей на основе человеческих предпочтений, выравнивание больших языковых моделей с человеческими ценностями, улучшение выходов языковых моделей по конкретным критериям.

Ссылки

- [PyTorch Loss Functions Documentation](#)
- [TensorFlow Loss Functions Guide](#)

7.9 Функции потерь для компьютерного зрения

Функции потерь, обычно используемые в задачах компьютерного зрения. Для общих функций потерь (cross-entropy, MSE, KL-дивергенция) см. «Общие функции потерь». Для метрик оценки в CV см. «Метрики компьютерного зрения».

Какую функцию потерь когда использовать

Функция потерь	Когда использовать
Focal	Дисбаланс классов в детекции или сегментации.
Dice	Перекрывание масок сегментации — медицинская визуализация, семантическая/instance-сегментация.
IoU / Jaccard	Качество bounding box, детекция.
Perceptual	Супервизия на уровне признаков для super-resolution, переноса стиля, image-to-image translation.
Adversarial	Обучение GAN — генератор против дискриминатора.
SSIM	Восстановление изображений, сжатие, super-resolution — структурное сходство.

Focal Loss

Борется с дисбалансом классов, понижая вклад «лёгких» примеров.

$$L_{\text{focal}} = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

Где: - p_t — вероятность правильного класса. - α_t — балансирующий коэффициент. - γ — фокусирующий параметр.

Применения: детекция объектов (RetinaNet), сегментация с несбалансированными классами, анализ медицинских изображений.

Dice Loss

Основана на коэффициенте Дайса, измеряющем перекрытие между предсказанной и ground truth сегментацией.

$$L_{\text{Dice}} = 1 - \frac{2 \sum_i^N p_i g_i}{\sum_i^N p_i^2 + \sum_i^N g_i^2}$$

Где: - p_i — предсказанная вероятность. - g_i — ground truth бинарная маска.

Применения: сегментация медицинских изображений, семантическая сегментация, instance-сегментация.

Варианты: - **Tversky Loss** — обобщение dice loss, позволяющее регулировать precision и recall. - **Combo Loss** — комбинация dice loss и взвешенной cross-entropy.

IoU (Intersection over Union) / Jaccard Loss

Основана на метрике IoU; напрямую оптимизирует качество предсказаний bounding box.

$$L_{\text{IoU}} = 1 - \frac{\text{area of overlap}}{\text{area of union}}$$

Применения: детекция объектов, instance-сегментация, регрессия bounding box.

Perceptual Loss

Сравнивает высокоуровневые представления признаков, извлечённые предобученной CNN, вместо попиксельных различий.

$$L_{\text{perceptual}} = \sum_j \lambda_j \frac{1}{C_j H_j W_j} \sum_{c,h,w} (\Phi_j(l)_{c,h,w} - \Phi_j(\hat{l})_{c,h,w})^2$$

Где: - Φ_j — карта признаков с j -го слоя предобученной сети. - I — ground truth изображение. - $\hat{I}$ — сгенерированное изображение. - C_j, H_j, W_j — размерности карты признаков.

Применения: super-resolution, перенос стиля, image-to-image translation, генерация изображений.

Adversarial Loss

Происходит из Generative Adversarial Networks (GAN) и заключается в минимаксной игре между генератором и дискриминатором.

$$L_{\text{adv}} = \mathbb{E}_x [\log(1 - D(G(z)))] + \mathbb{E}_z [\log D(x)]$$

Где: - D — дискриминатор. - G — генератор. - p_{data} — распределение реальных данных. - p_z — распределение шума.

Применения: генерация изображений, image-to-image translation, доменная адаптация, text-to-image генерация.

Варианты: - **WGAN Loss** — использует расстояние Вассерштейна для более стабильных градиентов. - **LSGAN Loss** — использует наименьшие квадраты вместо log-likelihood для более стабильного обучения. - **Hinge Loss** — альтернативная формулировка, показавшая хорошие результаты для генерации изображений.

SSIM (Structural Similarity Index) Loss

Измеряет структурное сходство между изображениями, фокусируясь на структурной информации, яркости и контрасте.

$$L_{\text{SSIM}} = 1 - \text{SSIM}(x, y)$$

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

Где: - μ_x, μ_y — средние значения пикселей. - σ_x^2, σ_y^2 — дисперсии. - σ_{xy} — ковариация. - C_1, C_2 — константы для избежания деления на ноль.

Применения: восстановление изображений, super-resolution, сжатие изображений, оценка качества изображений.

Ссылки

- [Understanding Dice Loss for Segmentation](#)
- [Perceptual Losses for Real-Time Style Transfer and Super-Resolution](#)
- [Focal Loss for Dense Object Detection](#)
- [GAN Loss Functions: A Comparison](#)